Chapter 1

High-Performance CPU

1.1 Overview

ESP32-P4 is powered by Espressif's second-generation 32-bit CPU core complex based upon RISC-V Instruction Set Architecture (ISA). The CPU Core complex consists of dual RISC-V CPU cores with a dedicated core-local interrupt controller (CLIC), a debug block, and a core-local interrupt (CLINT) timer. Figure 1.1 shows the block diagram of the CPU core complex.

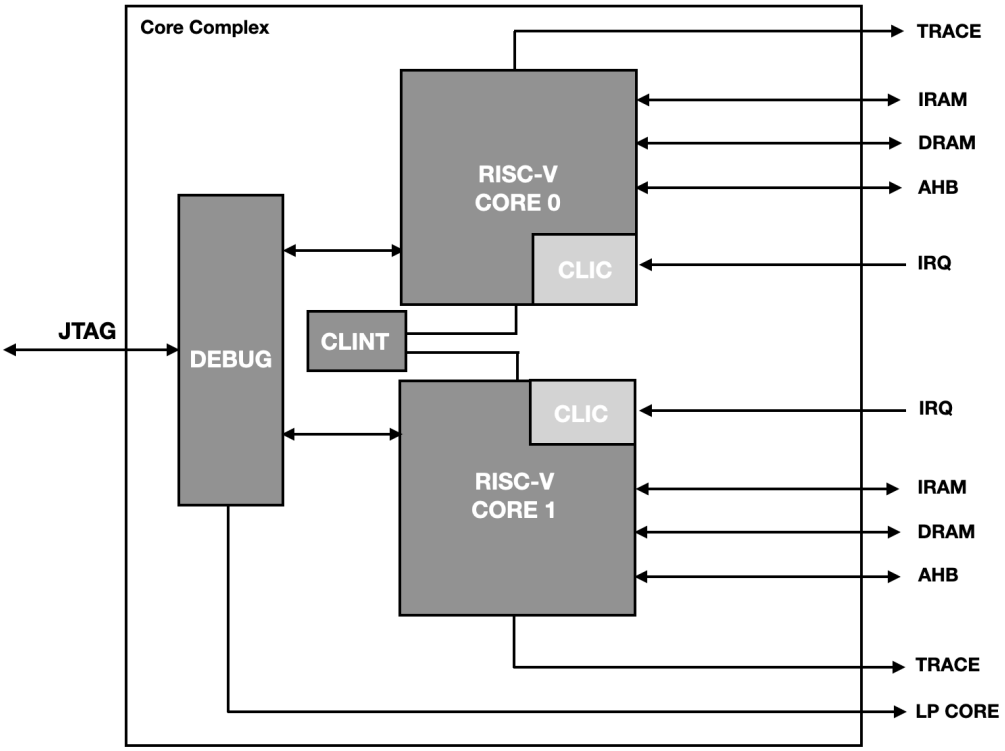


Figure 1.1-1. CPU Core Complex Block Diagram

Each RISC-V core in CPU core complex uses a 5-stage, in-order, scalar pipeline architecture optimized for area, power, and performance. Each RISC-V core has a built-in dedicated core-local interrupt controller (CLIC) and system bus (SYS BUS) interfaces for memory and peripheral access. The CLINT timer is shared between two cores. The DEBUG block provides a connection between the JTAG interface and the dual cores. It also supports LP core debug. Each core also provides an instruction trace interface for offline debugging.

1.2 Features

Each RISC-V CPU core in the CPU core complex implements the following standard RISC-V extensions:

- Mandatory base integer (I)
- Multiplication/Division (M)
- Atomic (A)
- Floating (F)
- Compressed (C)
- Code size reduction (Zc)
- Bit Manipulation (Zb)

In addition to the above standard extensions, each RISC-V CPU core also implements custom instructions (X) which can be broadly classified in the following two categories:

- Custom instructions to accelerate AI algorithms performance
- Custom hardware loop instructions to further improve performance and reduce code size for iterative AI algorithms execution

Apart from the above RISC-V instruction extensions support, each CPU core supports the features below:

- RISC-V RV32IMAFCX ISA with a five-stage pipeline that supports an operating clock frequency up to 360 MHz
- Compatible with RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA, Version 2.2
- Compatible with RISC-V Instruction Set Manual, Volume II: RISC-V Privileged Architecture, Version 1.10
- Compatible with Core-Local Interrupt Controller (CLIC) RISC-V Privileged Architecture Extensions (10/10/2023)
- Compatible with RISC-V External Debug Support Version 0.13.2
- Zero wait cycle access to on-chip SRAM and cache for program and data access over IRAM/DRAM interface
- 32-bit AHB system bus for peripheral access
- Core-local interrupts (CLINT) dedicated for each privilege mode
- User (U) privilege mode execution
- Interrupt delegation to user mode
- Bus error response
- Standard physical memory protection (PMP) configurable up to 32 regions and custom attributes (PMA) configurable up to 16 regions for security
- Branch target buffer (BTB) with BHT and RAS for performance improvement
- Debug module (DM) compliant with the specification RISC-V External Debug Support Version 0.13.2 with external debugger support over an industry-standard JTAG/USB port

- Debugger with a direct system bus access (SBA) to memory and peripherals via Core 0 data bus
- Support for instruction trace for offline debug
- Hardware trigger compliant with the specification RISC-V External Debug Support Version 0.13.2 with up to 3 breakpoints/watchpoints
- Fast GPIO support (up to 8)
- Configurable events for core performance metrics

1.3 Terminology

To better illustrate the functionality of the High-Performance CPU, the following terms are used in this chapter.

ABI	Application binary interface
branch	An instruction that conditionally changes the execution flow
CLIC	Core-local interrupt controller
CLINT	Core-local interrupt
CSR	Control and status register
delta	A change in the program counter that is other than the difference between two instructions placed consecutively in memory
DTM	Debug transport module
FPR	Floating-point register
GPIO	General-purpose input/output
GPR	General-purpose register
HWLP	Hardware loop
hart	RISC-V hardware thread
LSB	Least significant bit
LR	Load reserved
MSB	Most significant bit
PMP	Physical memory protection
retire	The final stage of executing an instruction, when the machine state is updated
RO	Read-only field
R/W	Readable and writable field
SC	Store conditional
SoC	System on chip
WO	Write-only field
WARL	Write any read legal
W1	Write One

1.4 Address Map

The table below shows the address map of various regions accessible by CPU for instruction, data, and system bus peripherals.

Table 1.4-1. CPU Address Map

Region	Description	Start Address	End Address	Access Type
CPU	CLIC/CLINT registers	0x2000_0000	0x2FFF_FFFF	R/W
IRAM/DRAM	Instruction/Data region	0x3000_0000	0x3FEF_FFFF	R/W
CPU peripherals		0x3FF0_0000	0x3FF1_FFFF	R/W
IRAM/DRAM	Instruction/Data region	0x3FF2_0000	0x4FFF_FFFF	R/W
CPU peripherals	HP APB peripherals	0x5000_0000	0x500F_FFFF	R/W
LP RAM/ROM		0x5010_0000	0x5010_FFFF	R/W
LP APB peripherals		0x5011_0000	0x5012_FFFF	R/W
AHB	AHB peripheral region	*Default	*Default	R/W

***Default:** Addresses not matching any of the specified ranges above are accessed through AHB bus by a CPU core.

1.5 Configuration and Status Registers (CSRs)

1.5.1 Register Summary

Below is a list of CSRs implemented for each HP CPU core. All the implemented CSRs follow the standard mapping of bit fields as described in the RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10. It must be noted that even among the standard CSRs, not all bit fields have been implemented, limited by the subset of features implemented in the CPU. Refer to the next section for detailed description of the subset of fields implemented under each of these CSRs.

Name	Description	Address	Access
Machine Information CSRs			
mvendorid	Machine vendor ID	0xF11	RO
marchid	Machine architecture ID	0xF12	RO
mimpid	Machine implementation ID	0xF13	RO
mhartid	Machine hart ID	0xF14	RO
Machine Trap Setup CSRs			
mstatus	Machine mode status	0x300	R/W
misa [1]	Machine ISA	0x301	R/W
mideleg	Machine interrupt delegation register (INACTIVE IN CLIC MODE)	0x303	RO
mie	Machine interrupt enable register (INACTIVE IN CLIC MODE)	0x304	RO
mtvec [2]	Machine trap vector	0x305	R/W
mcounteren	Machine counter enable	0x306	R/W
mtvt	Machine vector interrupt base address (Refer to CLIC specifications)	0x307	R/W

[1] Although misa is specified as having both read and write access (R/W), its fields are hardwired and thus write has no effect. This is what would be termed WARL (Write Any Read Legal) in RISC-V terminology

[2] mtvec Holds trap vector configuration consisting of vector base address and a vector mode

Name	Description	Address	Access
Machine Trap Handling CSRs			
mscratch	Machine scratch	0x340	R/W
mepc	Machine trap program counter	0x341	R/W
mcause [3]	Machine trap cause	0x342	R/W
mtval	Machine trap value	0x343	R/W
mip	Machine interrupt pending (INACTIVE IN CLIC MODE)	0x344	RO
mnxti	Interrupt handler address and enable modifier (Refer to CLIC specifications)	0x345	R/W
mintstatus	Current interrupt levels (Refer to CLIC specifications)	0xFB1	RO
mintthresh	Interrupt threshold (Refer to CLIC specifications)	0x347	R/W
mscratchcsw	Conditional scratch swap on priv mode change (Refer to CLIC specifications)	0x348	R/W
mscratchcswl	Conditional scratch swap on level change (Refer to CLIC specifications)	0x349	R/W
mclicbase	Machine mode interrupt controller base address register (CUSTOM) (Refer to CLIC specifications)	0x350	RO
User Trap Setup CSRs			
ustatus	User mode status	0x000	R/W
utvec [4]	User trap vector	0x005	R/W
utvt	User vector interrupt base address (Refer to CLIC specifications)	0x007	R/W
User Trap Handling CSRs			
uscratch	User scratch	0x040	R/W
uepc	User trap program counter	0x041	R/W
ucause [5]	User trap cause	0x042	R/W
unxti	Interrupt handler address and enable modifier (Refer to CLIC specifications)	0x045	R/W
uintthresh	Interrupt threshold (Refer to CLIC specifications)	0x047	R/W
uclicbase	User mode interrupt controller base address (CUSTOM) (Refer to CLIC specifications)	0x050	RO
uintstatus	Current interrupt levels (Refer to CLIC specifications)	0xCB1	RO
Physical Memory Protection (PMP) CSRs			
pmpcfg0	Physical memory protection configuration	0x3A0	R/W
pmpcfg1	Physical memory protection configuration	0x3A1	R/W
pmpcfg2	Physical memory protection configuration	0x3A2	R/W
pmpcfg3	Physical memory protection configuration	0x3A3	R/W
pmpcfg4	Physical memory protection configuration	0x3A4	R/W
pmpcfg5	Physical memory protection configuration	0x3A5	R/W
pmpcfg6	Physical memory protection configuration	0x3A6	R/W
pmpcfg7	Physical memory protection configuration	0x3A7	R/W

[3] External interrupt IDs reflected in mcause include even those IDs which have been reserved by RISC-V standard for core internal sources.
[4] utvec holds trap vector configurations, including the vector base address and the vector mode
[5] External interrupt IDs reflected in mcause include even those IDs that have been reserved by the RISC-V standard for core internal sources.

Name	Description	Address	Access
pmpaddr*n* (*n*: 0-31)	Physical memory protection address register	0x3B0+0x1**n*	R/W
Trigger Module CSRs (shared with Debug Mode)			
tselect	Trigger select register	0x7A0	R/W
tdata1	Trigger abstract data 1	0x7A1	R/W
tdata2	Trigger abstract data 2	0x7A2	R/W
tdata3	Trigger abstract data 3	0x7A3	R/W
tinfo	Trigger information	0x7A4	R/W
tcontrol	Global trigger control	0x7A5	R/W
mcontext	Machine mode context register	0x7A8	R/W
Debug Mode CSRs (Accessible only in Debug Mode)			
dcsr	Debug control and status	0x7B0	R/W
dpc	Debug PC	0x7B1	R/W
dscratch0	Debug scratch register 0	0x7B2	R/W
dscratch1	Debug scratch register 1	0x7B3	R/W
dmcs2	Debug mode control status register 2	0x32	R/W
Machine Counter Setup			
mcountinhibit	Machine counter inhibit register	0x320	R/W
mhpmevent8	Machine performance-monitoring event selector	0x308	R/W
mhpmevent9	Machine performance-monitoring event selector	0x329	R/W
mhpmevent13	Machine performance-monitoring event selector	0x32D	R/W
Machine Counter/Timers			
mcycle	Machine cycle counter	0xB00	R/W
minstret	Machine instructions-retired counter	0xB02	R/W
mhpmcounter8	Machine performance-monitoring counter	0xB08	R/W
mhpmcounter9	Machine performance-monitoring counter	0xB09	R/W
mhpmcounter13	Machine performance-monitoring counter	0xB0D	R/W
mcycleh	Upper 32 bits of mcycle	0xB80	R/W
minstreth	Upper 32 bits of minstret	0xB82	R/W
mhpmcounter8h	Upper 32 bits of mhpmcounter8	0xB88	R/W
mhpmcounter9h	Upper 32 bits of mhpmcounter9	0xB89	R/W
mhpmcounter13h	Upper 32 bits of mhpmcounter13	0xB8D	R/W
Unprivileged/User Mode Floating-Point CSRs			
fflags	Floating-point accrued exceptions	0x001	R/W
frm	Floating-point dynamic rounding mode	0x002	R/W
fcsr	Floating-point control and status register (frm + fflags)	0x003	R/W
Unprivileged/User Mode Counter/Timers			
cycle	Cycle Counter for RDCYCLE instruction	0xC00	RO
time	Timer for RDTIME instruction	0xC01	RO
instret	Instructions-retired counter for RDINSTRET instruction	0xC02	RO
hpmcounter8	Performance-monitoring counter	0xC08	RO
hpmcounter9	Performance-monitoring counter	0xC09	RO
hpmcounter13	Performance-monitoring counter	0xC0D	RO
cycleh	Upper 32 bits of cycle	0xC80	RO

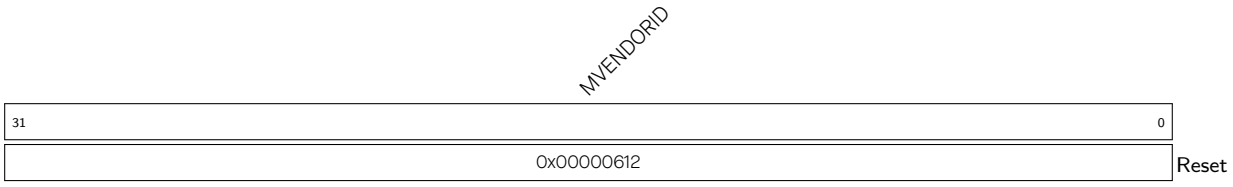
Name	Description	Address	Access
instreth	Upper 32 bits of instret	0xC82	RO
hpmcounter8h	Upper 32 bits of hpmcounter8	0xC88	RO
hpmcounter9h	Upper 32 bits of hpmcounter9	0xC89	RO
hpmcounter13h	Upper 32 bits of hpmcounter13	0xC8D	RO
Custom System CSRs (Custom)			
Floating-Point User Mode CSR (Custom)			
fxcr	Floating-point extended control register	0x800	R/W
GPIO Access CSRs (Custom)			
cpu_gpio_oen	GPIO output enable	0x803	R/W
cpu_gpio_in	GPIO input value	0x804	RO
cpu_gpio_out	GPIO output value	0x805	R/W
Extension Control and Status CSRs (Custom)			
mext_ill_reg	Machine extension illegal register	0x7F0	R/W
mhwloop_state_reg	Machine mode HWLP extension state register	0x7F1	R/W
mext_pie_status	Machine mode PIE extension status register	0x7F2	R/W
Zc Extension CSRs			
jvt	Machine table jump base vector and control register	0x017	R/W
Machine Extension Status CSRs (Custom)			
mexstatus	Machine extension control and status register	0x7E1	R/W
mhint	Machine implicit operation register	0x7C5	R/W
Physical Memory Attributes Checker (PMAC) CSRs (Custom)			
pma_cfg*n* (*n*: 0-15)	Physical memory attribute configuration register	0xBC0+0x1**n*	R/W
pma_addr*n* (*n*: 0-15)	Physical memory attribute address register	0xBD0+0x1**n*	R/W
Machine Hardware Loop CSRs (Custom)			
mhloop0_start_addr	32-bit start address for loop 0	0x7C6	R/W
mhloop0_end_addr	32-bit start address for loop 0	0x7C7	R/W
mhloop0_count	Iteration count for hardware loop 0	0x7C8	R/W
mhloop1_start_addr	32-bit start address for loop 1	0x7C9	R/W
mhloop1_end_addr	32-bit end address for loop 1	0x7CA	R/W
mhloop1_count	Iteration count for hardware loop 1	0x7CB	R/W
User Hardware Loop CSRs (Custom)			
uhloop0_start_addr	32-bit start address for loop 0	0x8C0	R/W
uhloop0_end_addr	32-bit end address for loop 0	0x8C1	R/W
uhloop0_count	Iteration count for hardware loop 0	0x8C2	R/W
uhloop1_start_addr	32-bit start address for loop 1	0x8C3	R/W
uhloop1_end_addr	32-bit end address for loop 1	0x8C4	R/W
uhloop1_count	Iteration count for hardware loop 1	0x8C5	R/W
uhwloop_state_reg	User HWLP state register	0x8C6	R/W
Bus Error Exception Context CSRs (Custom)			
ldpc0	Load bus error PC register 0	0xBE0	R/W
ldpc1	Load bus error PC register 1	0xBE1	R/W
ldtval0	Load bus error access address register 0	0xBE8	R/W
ldtval1	Load bus error access address register 1	0xBE9	R/W

Name	Description	Address	Access
stpc0	Store bus error PC register 0	0xBF0	R/W
stpc1	Store bus error PC register 1	0xBF1	R/W
stpc2	Store bus error PC register 2	0xBF2	R/W
sttval0	Store bus error access address register 0	0xBF8	R/W
sttval1	Store bus error access address register 1	0xBF9	R/W
sttval2	Store bus error access address register 2	0xBFA	R/W

Note that if write/set/clear operation is attempted on any of the CSRs which are read-only (RO), as indicated in the above table, the CPU will generate an illegal instruction exception.

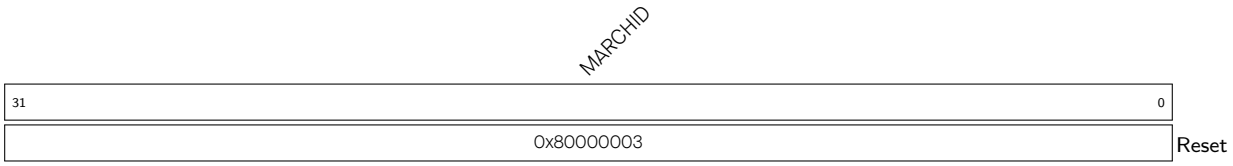
1.5.2 Register Description

Register 1.1. mvendorid (0xF11)



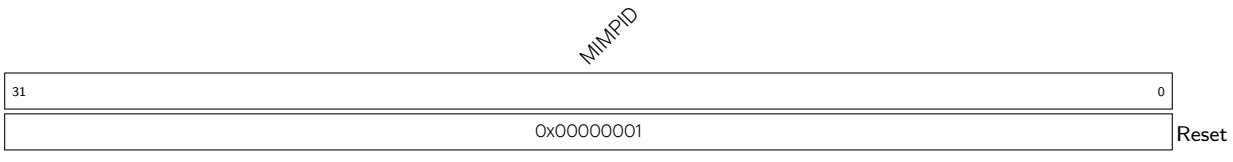
MVENDORID Represents Vendor ID. (RO)

Register 1.2. marchid (0xF12)



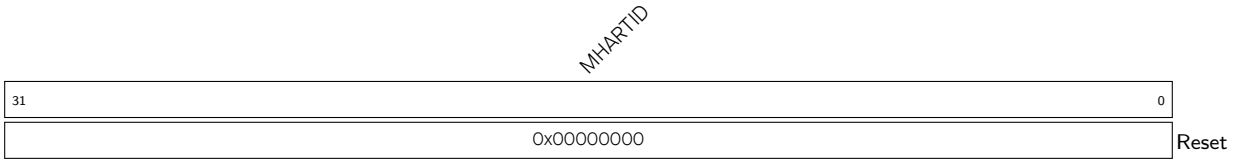
MARCHID Represents Architecture ID. (RO)

Register 1.3. mimpid (0xF13)



MIMPID Represents Implementation ID. (RO)

Register 1.4. mhartid (0xF14)



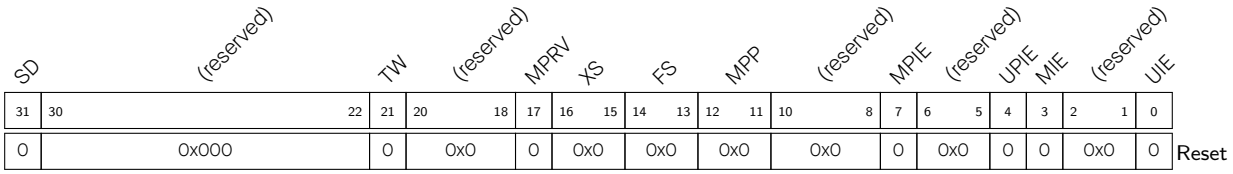
MHARTID Represents Hart ID.

0: Hart 0

1: Hart 1

(RO)

Register 1.5. mstatus (0x300)



SD Automatically set when either the FS or XS fields signal the presence of some dirty state that will require saving extension context to memory. (RO)

TW Configures whether the WFI (wait for interrupt) instruction can execute in less privileged modes.
0: WFI can execute in lower privilege modes.
1: WFI can only be executed in machine mode, and if executed in a less privileged mode, it will trigger an illegal instruction exception.
(R/W)

MPRV Configures whether to apply mstatus.MPP as the effective privilege mode, in which loads and stores execute, instead of the actual privilege mode in which the CPU is executing.
0: Not apply
1: Apply
Note that instruction protection is unaffected by this bit.
(R/W)

XS Represents the combined status of the custom extension states, including the HWLP and the PIE extensions.
0x0: All extension are Off
0x1: None dirty or clean, some on
0x2: None dirty, some clean
0x3: Some dirty
Note that in order to modify the state of either of these extensions, the corresponding CSRs mext_hwlp_status and mext_pie_status must be used.
(RO)

FS Represents the status of the floating-point unit, including the floating-point registers f0–f31 and the CSRs fcsr, frm, and fflags.
0x0: Off
0x1: Initial
0x2: Clean
0x3: Dirty
(R/W)

MPP Configures machine previous privilege mode (before trap).
0x0: User mode
0x3: Machine mode
Note: Only the lower bit is writable. Any write to the higher bit is ignored as it is directly tied to the lower bit.
(R/W)

Continued on the next page...

Register 1.5. mstatus (0x300)

Continued from the previous page...

MPIE Configures whether to enable the machine previous interrupt (before trap).

0: Disable

1: Enable

(R/W)

UPIE Configures whether to enable the user previous interrupt (before trap).

0: Disable

1: Enable

(R/W)

MIE Configures whether to enable the global machine interrupt.

0: Disable

1: Enable

(R/W)

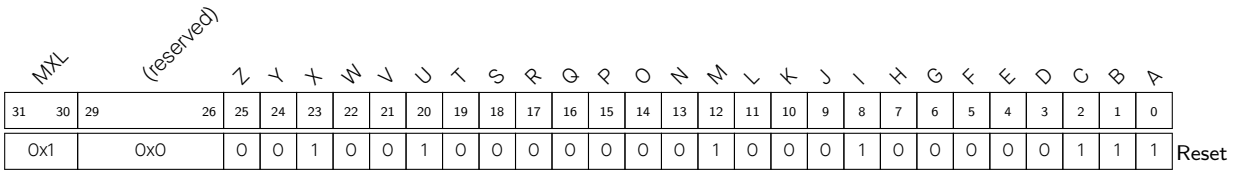
UIE Configures whether to enable the global user interrupt.

0: Disable

1: Enable

(R/W)

Register 1.6. misa (0x301)



MXL Machine XLEN = 1 (32-bit). (RO)

Z Reserved = 0. (RO)

Y Reserved = 0. (RO)

X Non-standard extensions present = 1. (RO)

W Reserved = 0. (RO)

V Reserved = 0. (RO)

U User mode implemented = 1. (RO)

T Reserved = 0. (RO)

S Supervisor mode implemented = 0. (RO)

R Reserved = 0. (RO)

Q Quad-precision floating-point extension = 0. (RO)

P Reserved = 0. (RO)

O Reserved = 0. (RO)

N User-level interrupts supported = 0. (RO)

M Integer Multiply/Divide extension = 1. (RO)

L Reserved = 0. (RO)

K Reserved = 0. (RO)

J Reserved = 0. (RO)

I RV32I base ISA = 1. (RO)

H Hypervisor extension = 0. (RO)

G Additional standard extensions present = 0. (RO)

F Single-precision floating-point extension = 0. (RO)

E RV32E base ISA = 0. (RO)

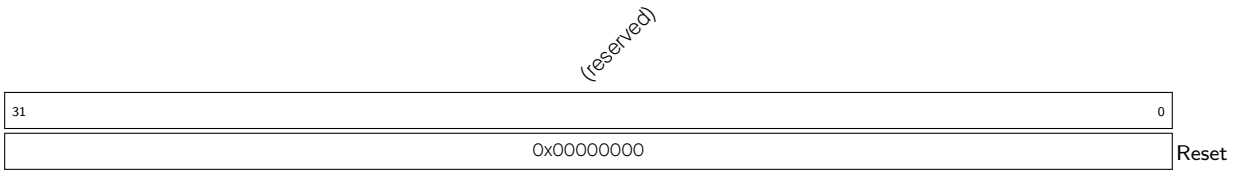
D Double-precision floating-point extension = 0. (RO)

C Compressed Extension = 1. (RO)

B Bit Manipulation = 1. (RO)

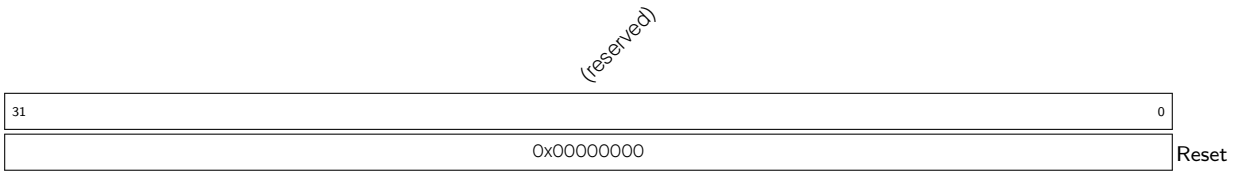
A Atomic Extension = 1. (RO)

Register 1.7. mideleg (0x303)



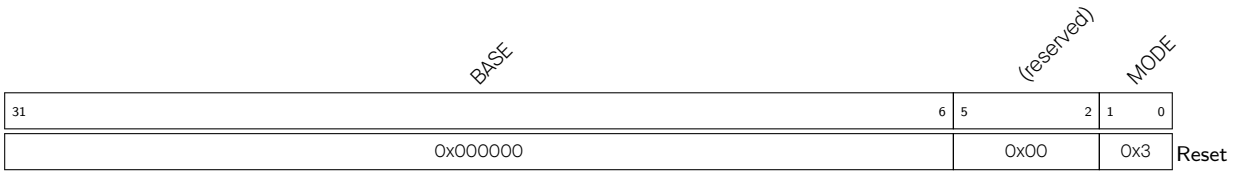
mideleg All bits are hardwired to 0 since only CLIC mode is available. (RO)

Register 1.8. mie (0x304)



mie All bits are hardwired to 0 since only CLIC mode is available. (RO)

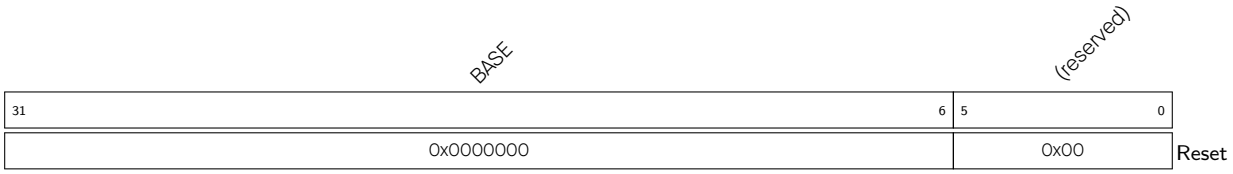
Register 1.9. mtvec (0x305)



BASE Configures the higher 26 bits of exception and non-vectored machine mode interrupt base address aligned to 64 bytes. (R/W)

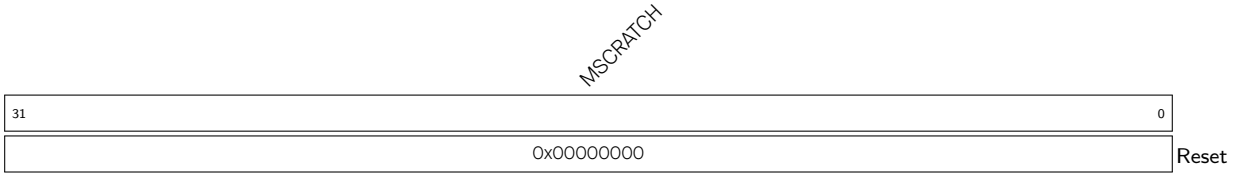
MODE Represents whether machine mode interrupts are operating in CLIC mode or vectored/non-vectored CLINT mode. Only CLIC mode **0x3** is available. (RO)

Register 1.10. mtvt (0x307)



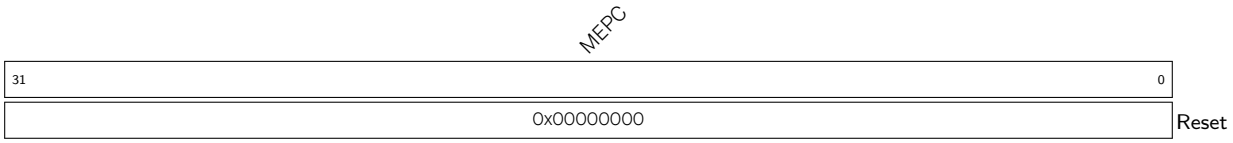
BASE Configures the higher 26 bits of CLIC machine mode interrupt vector base address aligned to 64 bytes. (R/W)

Register 1.11. mscratch (0x340)



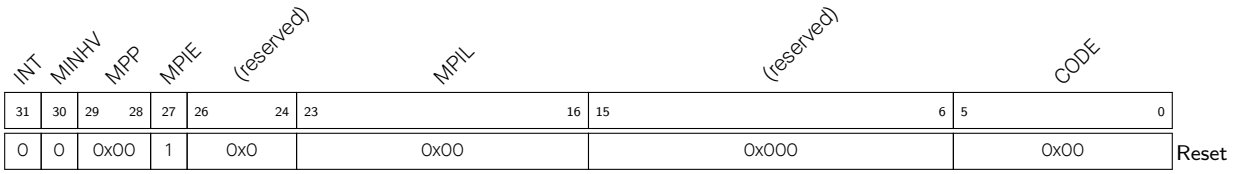
MSCRATCH Configures machine scratch information for custom use. (R/W)

Register 1.12. mepc (0x341)



MEPC Configures the machine trap/exception program counter. This is automatically updated with address of the instruction which was about to be executed while CPU encountered the most recent trap. (R/W)

Register 1.13. mcause (0x342)



INT Represents whether the CPU entered a trap due to interrupts.

0: The trap occurred due to an exception.

1: The trap occurred due to an interrupt.

(R/W)

MINHV Represents whether mepc is an address of an instruction or an address of a vector table entry.

1: Address of a table entry

0: Address of an instruction

(R/W)

MPP Represents the previous privilege mode, same as mstatus.MPP. (R/W)

MPIE Represents the previous interrupt enable, same as mstatus.MPIE. (R/W)

MPIL Represents the previous 8-bit interrupt level. (R/W)

CODE Represents the unique ID of the most recent exception or interrupt due to which CPU entered trap. Possible exception IDs are:

0x01: Instruction access fault

0x02: Illegal instruction

0x03: Hardware breakpoint/watchpoint or EBREAK

0x05: PMP load access fault

0x06: Misaligned store address or AMO address

0x07: Store access or AMO access fault

0x08: ECALL from U mode

0x0b: ECALL from M mode

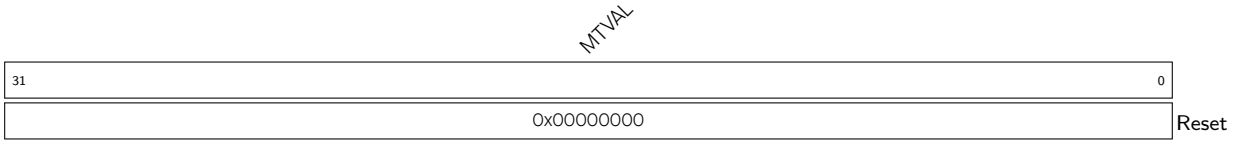
0x1f: Illegal PIE instruction

Other exception IDs are reserved.

Note: Exception ID 0x0 (instruction access misaligned) is not present because CPU always masks the lowest bit of the address during instruction fetch.

(R/W)

Register 1.14. mtval (0x343)



MTVAL Represents the machine trap value. This is automatically updated with an exception dependent data which may be useful for handling that exception.

Data is to be interpreted depending upon exception IDs:

0x01: Faulting virtual address of instruction

0x02: Faulting instruction opcode

0x05: Faulting data address of load operation

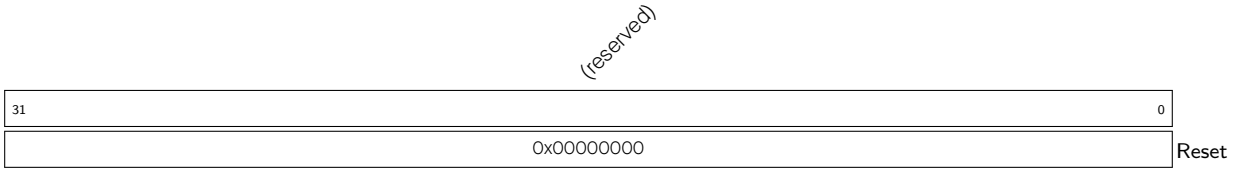
0x07: Faulting data address of store operation

0x1F: Faulting instruction opcode related to illegal PIE instruction

Note: The value of this register is not valid for other exception IDs and interrupts.

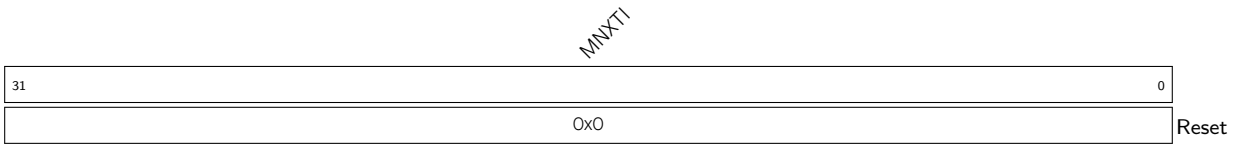
(R/W)

Register 1.15. mip (0x344)



All bits are hardwired to 0 since only CLIC mode is available. (RO)

Register 1.16. mnxti (0x345)



MNXTI Represents the next machine mode interrupt entry address and configures the interrupt enable status.

This CSR can be used by software to service the next horizontal interrupt for the same privilege mode when it has greater level than the saved interrupt context (held in mcause.MPIL) and greater level than the interrupt threshold of the corresponding privilege mode, without incurring the full cost of an interrupt pipeline flush and context save/restore.

This CSR is designed to be accessed using CSRRSI/CSRRCI instructions, where the value read is a pointer to an entry in the trap handler table and the write back updates the interrupt-enable status. In addition, writes to the mnxti have side-effects that update the interrupt context state.

A read of the mnxti CSR using CSRR returns either zero, indicating there is no suitable interrupt to service or that the system is not in a CLIC mode, or returns a non-zero address of the entry in the trap handler table for software trap vectoring.

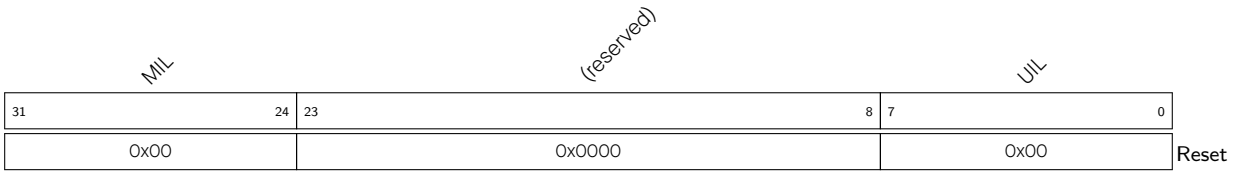
If the CSR instruction that accesses mnxti includes a write, the mstatus CSR is the one used for the read-modify-write portion of the operation, while the mcause register's exccode field and the mintstatus.MIL field can also be updated with the new interrupt id and level. If the interrupt is edge-triggered, then the pending bit is also zeroed.

The mnxti CSR is intended to be used inside an interrupt handler after an initial interrupt has been taken and mcause and mepc registers updated with the interrupted context and the ID of the interrupt.

If the pending interrupt is edge-triggered, hardware will automatically clear the corresponding pending bit when the CSR instruction that accesses mnxti includes a write. However, if the CSR instruction does not include write side effects, then no state update on any CSR occurs and thus the interrupt pending bit is not zeroed. This behavior allows software to optimize the selection and execution of interrupts using mnxti.

(R/W)

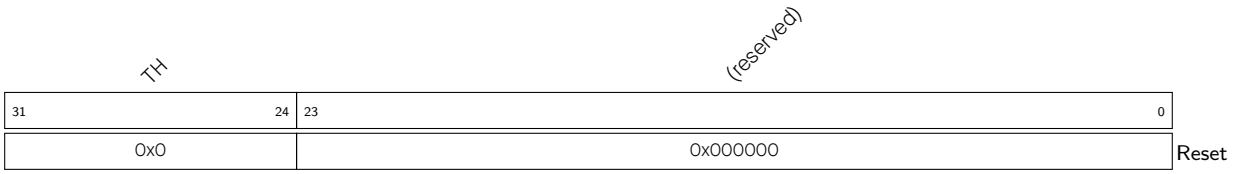
Register 1.17. mintstatus (0xFB1)



MIL Represents the active 8-bit interrupt level for current machine mode interrupt. (RO)

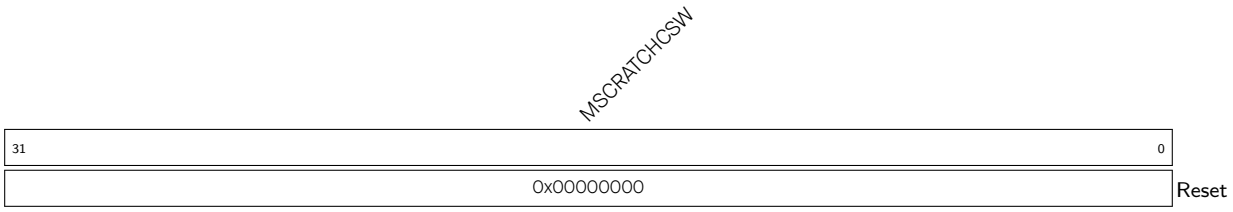
UIL Hardwired to 0x0 as user mode interrupts are not supported. (RO)

Register 1.18. mintthresh (0x347)



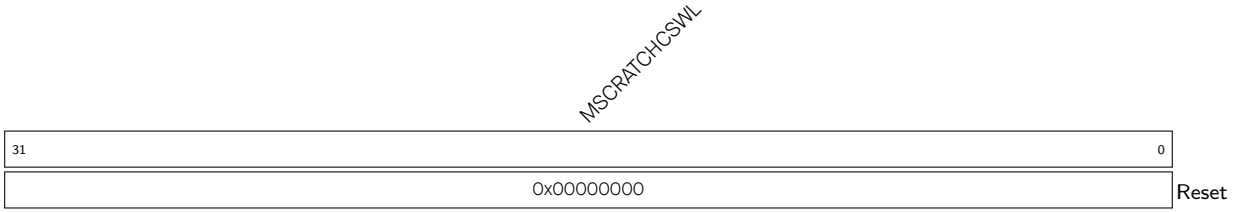
TH Configures the 8-bit level threshold of machine mode interrupts.
Note that the effective threshold level for machine mode interrupts is the maximum of mintthresh.TH and mintstatus.MIL. All machine mode pending interrupts with levels less than or equal to the effective threshold level are not allowed to preempt the execution.
(R/W)

Register 1.19. mscratchcsw (0x348)



MSCRATCHCSW Configures the MSCRATCH value by conditionally swapping its value with RS1, based on the current privilege mode and the previous privilege mode in MPP.
This is the conditional scratch swap CSR, which allows performing a scratch value swap conditionally, based on privilege mode change, in a single instruction.
When using CSRRW instruction to access this CSR, the value written into RD is either that of MSCRATCH, if MPP is different than the current privilege mode, or RS1 if MPP is the same as the current privilege mode. The MSCRATCH CSR value is updated with the original value of RS1 only if there is a privilege mode difference.
(R/W)

Register 1.20. mscratchcswl (0x349)



MSCRATCHCSWL Configures the MSCRATCH value by conditionally swapping its value with RS1, based on the current interrupt level in mintstatus.MIL and the previous interrupt level in mcause.MPIL.

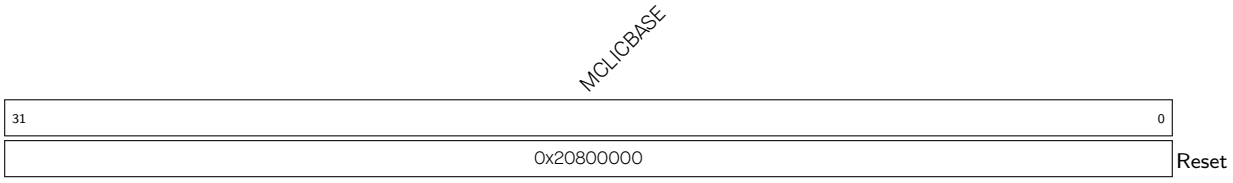
This is the conditional scratch swap CSR, which allows performing a scratch value swap conditionally, based on interrupt level change, in a single instruction.

When using CSRRW instruction to access this CSR, the value written into RD is either that of MSCRATCH, if only one of mcause.MPIL and mintstatus.MIL is equal to zero, or RS1 for all other cases.

The value of MSCRATCH is updated with the initial value of RS1 only if one of mcause.MPIL and mintstatus.MIL is equal to zero.

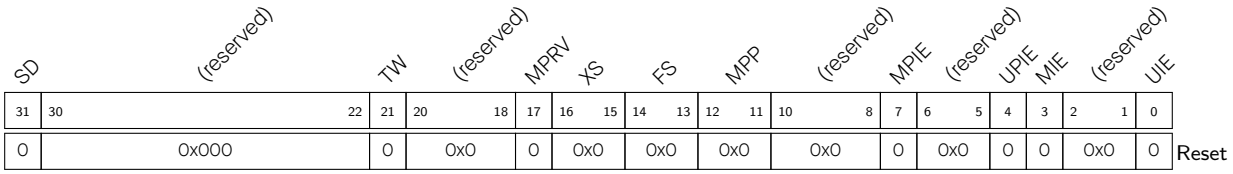
(R/W)

Register 1.21. mclicbase (0x350)



MCLICBASE Represents the base address of the CLIC memory-mapped registers for machine mode. (RO)

Register 1.22. ustatus (0x000)



SD Automatically set when either the FS or XS fields signal the presence of some dirty state that will require saving extension context to memory. (RO)

TW Represents whether the WFI (wait for interrupt) instruction can execute in less privileged modes.
0: WFI can execute in lower privilege modes.
1: WFI can only be executed in machine mode, and if executed in a less privileged mode, it will trigger an illegal instruction exception.
(RO)

MPRV Represents whether to apply ustatus.MPP as the effective privilege mode, in which loads and stores execute, instead of the actual privilege mode in which the CPU is executing.
0: Not apply
1: Apply
Note that instruction protection is unaffected by this bit.
(RO)

XS Represents the combined status of the custom extension states, including the HWLP and the PIE extensions.
0x0: All extension are Off
0x1: None dirty or clean, some on
0x2: None dirty, some clean
0x3: Some dirty
Note that in order to modify the state of either of these extensions, the corresponding CSRs mext_hwlp_status and mext_pie_status must be used.
(RO)

FS Configures the status of the floating-point unit, including the floating-point registers f0–f31 and the CSRs fcsr, frm, and fflags.
0x0: Off
0x1: Initial
0x2: Clean
0x3: Dirty
(R/W)

MPP Represents machine previous privilege mode (before trap).
0x0: User mode
0x3: Machine mode
Note: Only the lower bit is writable. Any write to the higher bit is ignored as it is directly tied to the lower bit.
(RO)

Continued on the next page...

Register 1.22. ustatus (0x000)

Continued from the previous page...

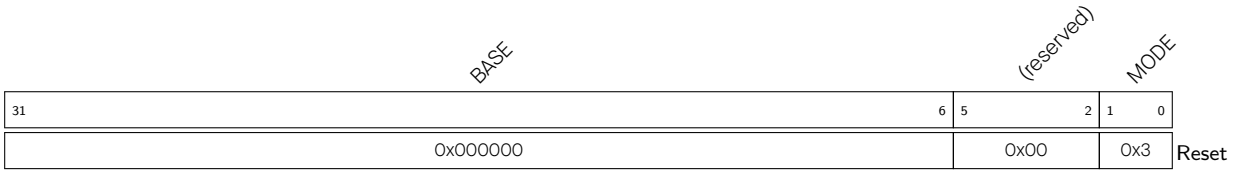
MPIE Represents machine previous interrupt enable status (before trap).
0: Disabled
1: Enabled
(RO)

UPIE Represents user previous interrupt enable status (before trap).
0: Disabled
1: Enabled
(RO)

MIE Represents global machine interrupt enable status.
0: Disabled
1: Enabled
(RO)

UIE Represents global user mode interrupt enable status.
0: Disabled
1: Enabled
(RO)

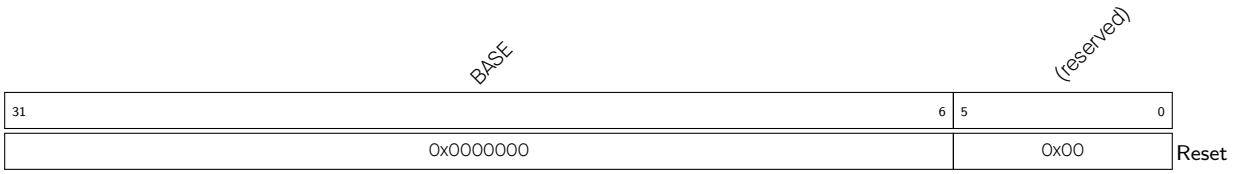
Register 1.23. utvec (0x005)



BASE Configures the higher 26 bits of exception and non-vectored user mode interrupt base address aligned to 64 bytes. (R/W)

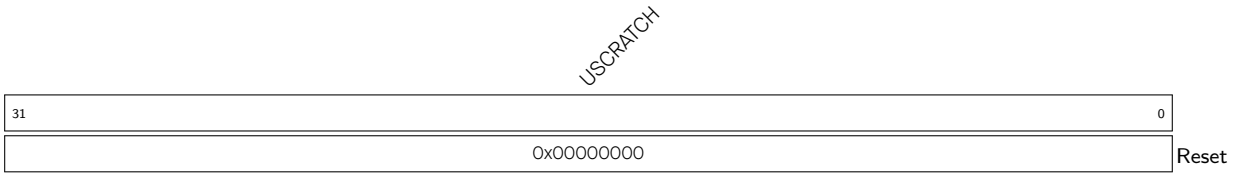
MODE Represents whether user mode interrupts are operating in CLIC mode or vectored/non-vectored CLINT mode. Only CLIC mode **0x3** is available. (RO)

Register 1.24. utvt (0x007)



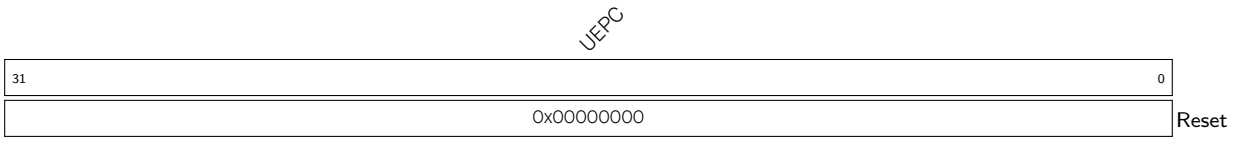
BASE Configures the higher 26 bits of CLIC user mode interrupt vector base address aligned to 64 bytes. (R/W)

Register 1.25. uscratch (0x040)



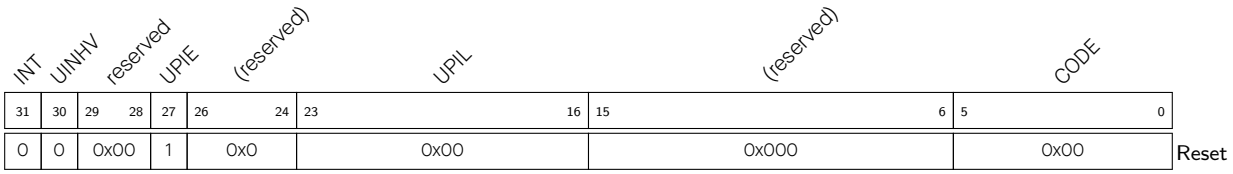
USCRATCH Configures user scratch information for custom use. (R/W)

Register 1.26. uepc (0x041)



UEPC Configures the user trap/exception program counter. This is automatically updated with address of the instruction which was about to be executed while CPU encountered the most recent trap. (R/W)

Register 1.27. ucause (0x042)



INT Represents whether the CPU entered a trap due to interrupts.

0: The trap occurred due to an exception.

1: The trap occurred due to an interrupt.

(R/W)

UINHV Represents whether uepc is an address of an instruction or an address of a vector table entry.

1: Address of a table entry

0: Address of an instruction

(R/W)

UPIE Represents the previous interrupt enable, same as ustatus.UPIE. (R/W)

UPIL Represents the previous 8-bit interrupt level. (R/W)

CODE Represents the unique ID of the most recent exception or interrupt due to which CPU entered trap. Possible exception IDs are:

0x01: Instruction access fault

0x02: Illegal instruction

0x03: Hardware breakpoint/watchpoint or EBREAK

0x05: PMP load access fault

0x06: Misaligned store address or AMO address

0x07: Store access or AMO access fault

0x08: ECALL from U mode

0x0b: ECALL from M mode

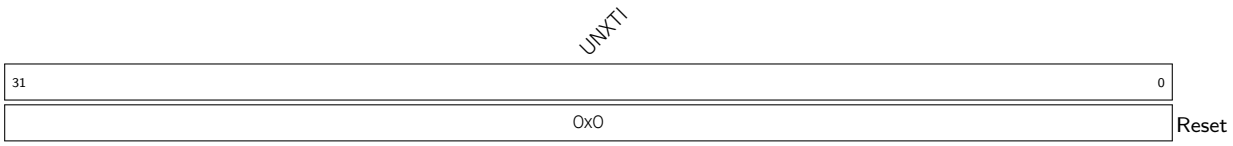
0x1f: Illegal PIE instruction

Other exception IDs are reserved.

Note: Exception ID 0x0 (instruction access misaligned) is not present because CPU always masks the lowest bit of the address during instruction fetch.

(R/W)

Register 1.28. unxti (0x045)



UNXTI Represents the next user mode interrupt entry address and configures the interrupt enable status.

This CSR can be used by software to service the next horizontal interrupt for the same privilege mode when it has greater level than the saved interrupt context (held in ucause.UPIL) and greater level than the interrupt threshold of the corresponding privilege mode, without incurring the full cost of an interrupt pipeline flush and context save/restore.

This CSR is designed to be accessed using CSRRSI/CSRRCI instructions, where the value read is a pointer to an entry in the trap handler table and the write back updates the interrupt-enable status. In addition, writes to the unxti have side-effects that update the interrupt context state.

A read of the unxti CSR using CSRR returns either zero, indicating there is no suitable interrupt to service or that the system is not in a CLIC mode, or returns a non-zero address of the entry in the trap handler table for software trap vectoring.

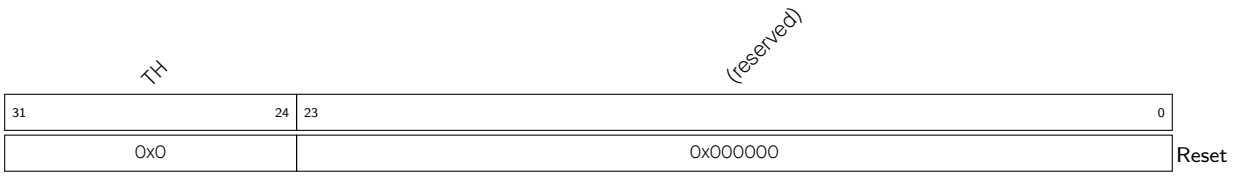
If the CSR instruction that accesses unxti includes a write, the ustatus CSR is the one used for the read-modify-write portion of the operation, while the ucause register's exccode field and the uintstatus.UIL field can also be updated with the new interrupt id and level. If the interrupt is edge-triggered, then the pending bit is also zeroed.

The unxti CSR is intended to be used inside an interrupt handler after an initial interrupt has been taken and ucause and uepc registers updated with the interrupted context and the ID of the interrupt.

If the pending interrupt is edge-triggered, hardware will automatically clear the corresponding pending bit when the CSR instruction that accesses unxti includes a write. However, if the CSR instruction does not include write side effects, then no state update on any CSR occurs and thus the interrupt pending bit is not zeroed. This behavior allows software to optimize the selection and execution of interrupts using unxti.

(R/W)

Register 1.29. uintthresh (0x047)

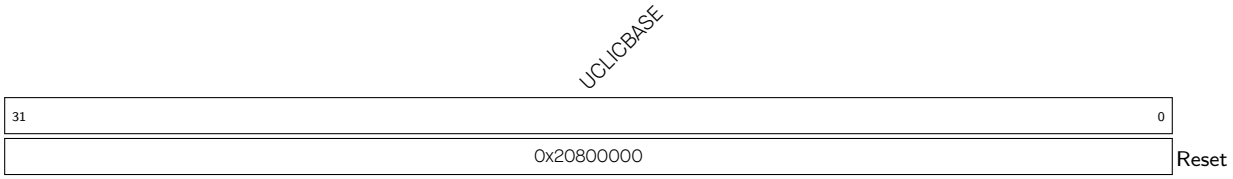


TH Configures the 8-bit level threshold of user mode interrupts.

Note that the effective threshold level for user mode interrupts is the maximum of uintthresh.TH and uintstatus.UIL. All user mode pending interrupts with levels less than or equal to the effective threshold level are not allowed to preempt the execution.

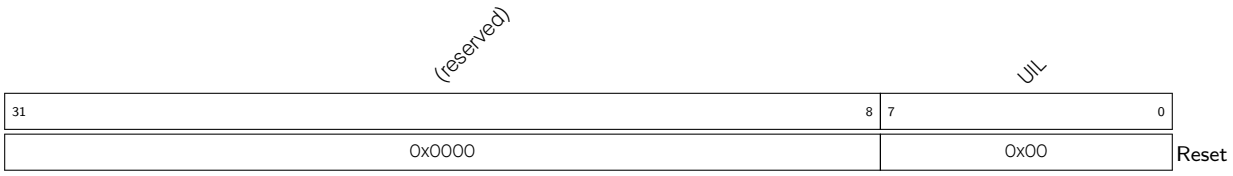
(R/W)

Register 1.30. uclicbase (0x050)



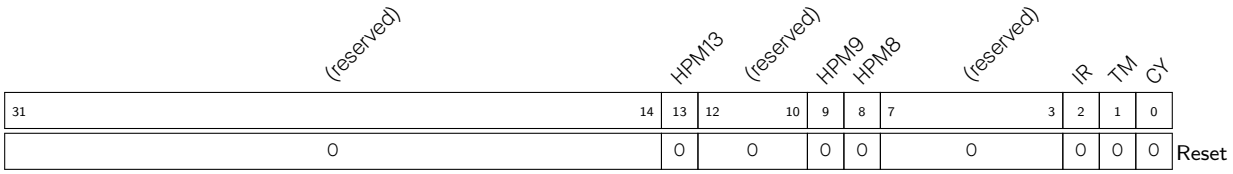
UCLICBASE Represents the base address of the CLIC memory-mapped registers for user mode. (RO)

Register 1.31. uintstatus (0xCB1)



UIL Represents the active 8-bit interrupt level for the current user mode interrupt. (RO)

Register 1.32. mcounteren (0x306)



HPM13 Configures inline]Configures users counters instead of machine counters access from user mode. Bhanu: Updated hpmcounter13 access from user mode.

0: Accessing hpmcounter13 CSR in user mode will generate illegal instruction exception.

1: Accessing hpmcounter13 CSR in user mode is valid.

(R/W)

HPM9 Configures hpmcounter9 access from user mode.

0: Accessing hpmcounter9 CSR in user mode will generate illegal instruction exception.

1: Accessing hpmcounter9 CSR in user mode is valid.

(R/W)

HPM8 Configures hpmcounter8 access from user mode.

0: Accessing hpmcounter8 CSR in user mode will generate illegal instruction exception.

1: Accessing hpmcounter8 CSR in user mode is valid.

(R/W)

CY Configures access of cycle counter from user mode.

0: Accessing cycle CSR in user mode will generate illegal instruction exception.

1: Accessing cycle CSR in user mode is valid.

(R/W)

TM Configures access of time counter from user mode.

0: Accessing time CSR in user mode will generate illegal instruction exception.

1: Accessing time CSR in user mode is valid.

(R/W)

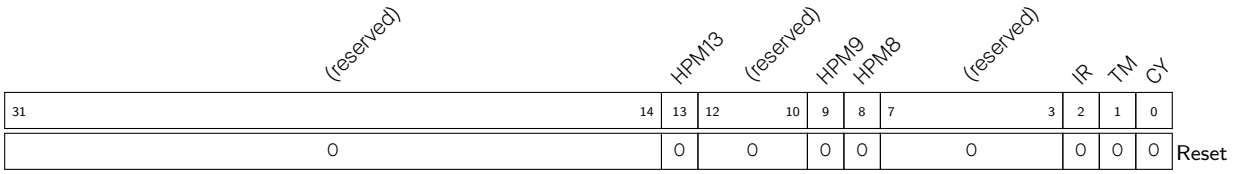
IR Configures access of instret counter from user mode.

0: Accessing instret CSR in user mode will generate illegal instruction exception.

1: Accessing instret CSR in user mode is valid.

(R/W)

Register 1.33. mcountinhibit (0x320)



HPM13 Configures the incrementing of mhpmcounter13.

0: Continue incrementing the mhpmcounter13 counter.

1: Stop incrementing the mhpmcounter13 counter.

(R/W)

HPM9 Configures the incrementing of mhpmcounter9.

0: Continue incrementing the mhpmcounter9 counter.

1: Stop incrementing the mhpmcounter9 counter.

(R/W)

HPM8 Configures the incrementing of mhpmcounter8.

0: Continue incrementing the mhpmcounter8 counter.

1: Stop incrementing the mhpmcounter8 counter.

(R/W)

IR Configures the incrementing of the instret counter.

0: Continue incrementing the instret counter.

1: Stop incrementing the instret counter.

(R/W)

TM Configures the incrementing of the time counter.

0: Continue incrementing the time counter.

1: Stop incrementing the time counter.

(R/W)

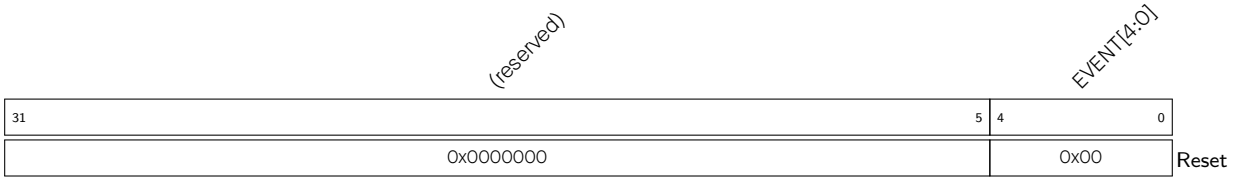
CY Configures the incrementing of the cycle counter.

0: Continue incrementing the cycle counter.

1: Stop incrementing the cycle counter.

(R/W)

Register 1.34. mhpmevent8 (0x328)

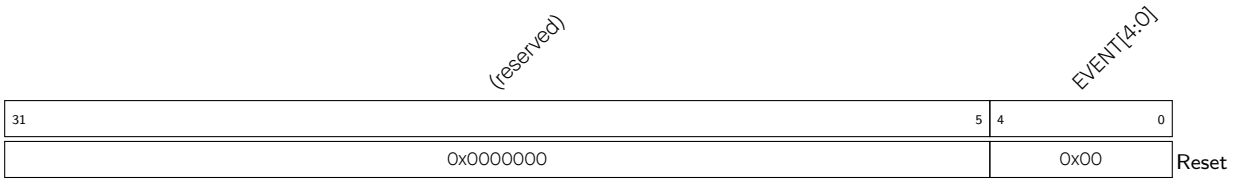


EVENT[4:0] Event selector for mhpmcounter8.

The only valid event value for this counter is 0x6, for conditional branch mispredictions.

(R/W)

Register 1.35. mhpmevent9 (0x329)

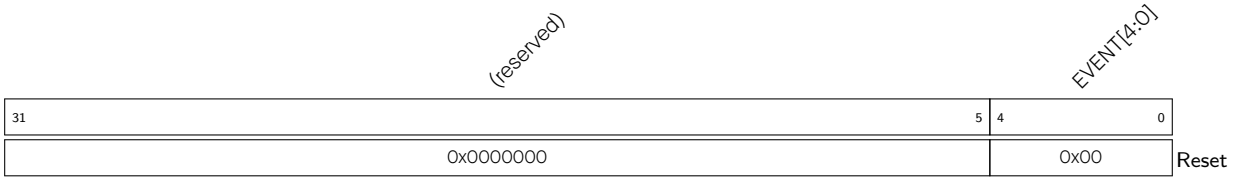


EVENT[4:0] Event selector for mhpmcounter9.

The only valid event value for this counter is 0x7, for conditional branch instructions.

(R/W)

Register 1.36. mhpmevent13 (0x32D)

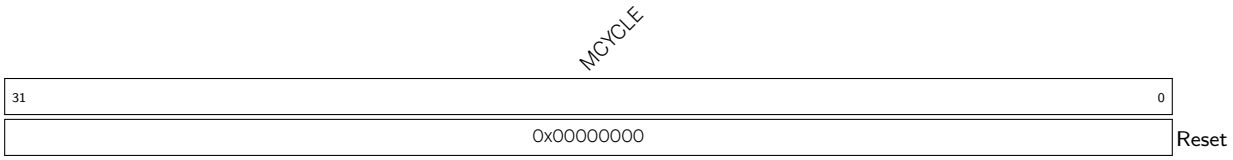


EVENT[4:0] Event selector for mhpmcounter13.

The only valid event value for this counter is 0xB, for store instructions.

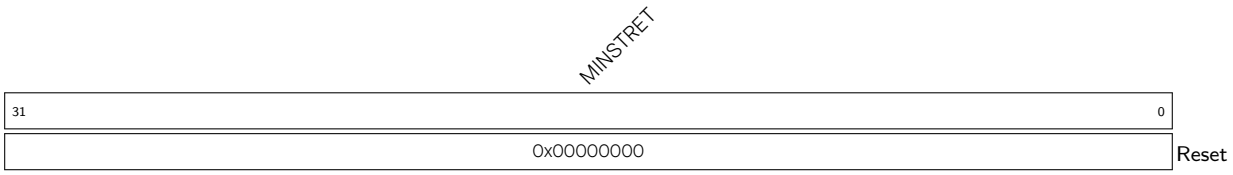
(R/W)

Register 1.37. mcycle (0xB00)



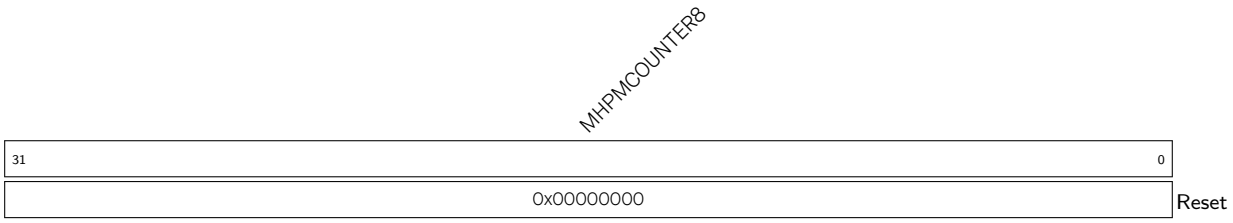
MCYCLE Represents the lower 32 bits of the machine mode cycle counter. (R/W)

Register 1.38. minstret (0xB02)



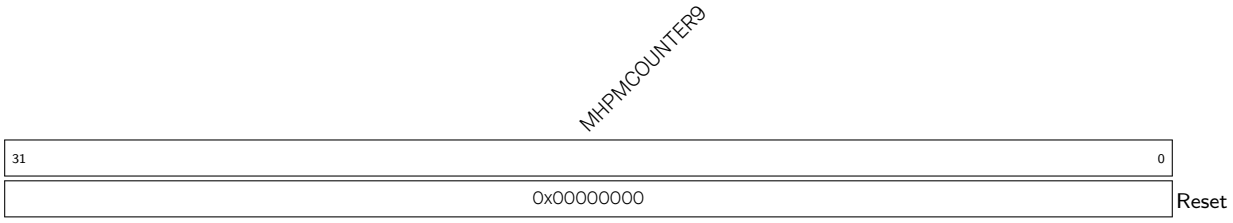
MINSTRET Represents the lower 32 bits of the machine instructions retired counter. (R/W)

Register 1.39. mhpmcounter8 (0xB08)



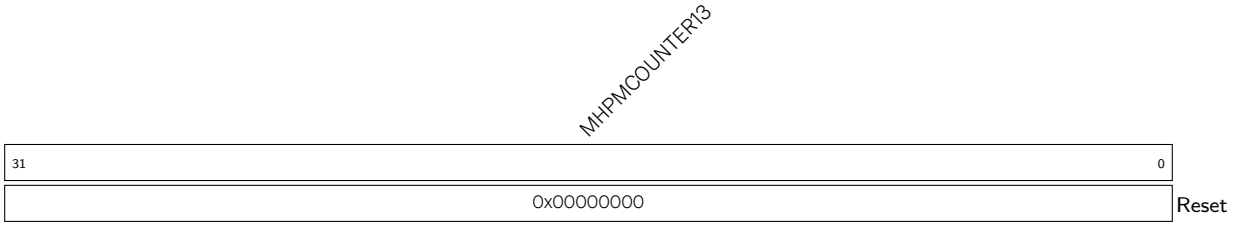
MHPMCOUNTER8 Represents the lower 32 bits of the machine performance-monitoring counter8. (R/W)

Register 1.40. mhpmcounter9 (0xB09)



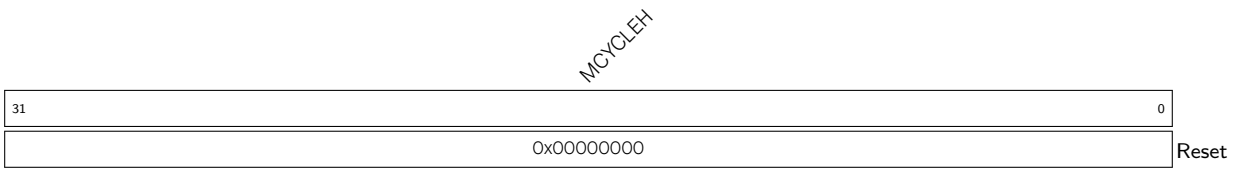
MHPMCOUNTER9 Represents the lower 32 bits of the machine performance-monitoring counter9. (R/W)

Register 1.41. mhpmcounter13 (0xB0D)



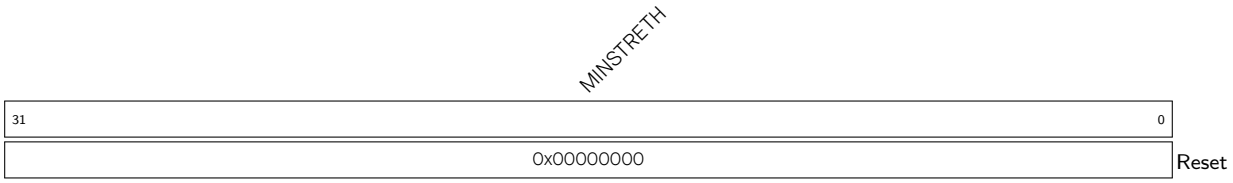
MHPMCOUNTER13 Represents the lower 32 bits of the machine performance-monitoring counter13. (R/W)

Register 1.42. mcycleh (0xB80)



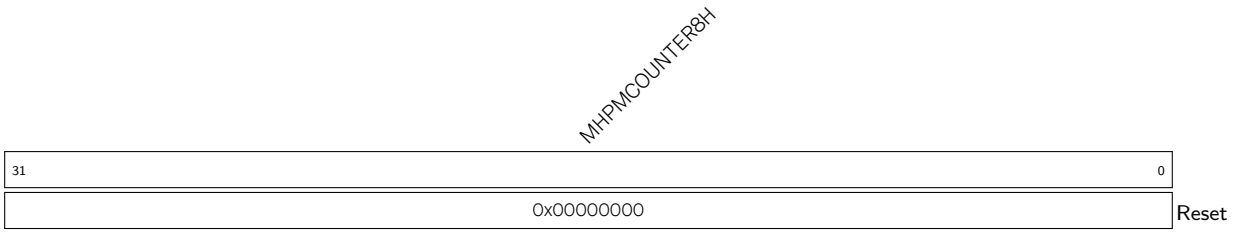
MCYCLEH Represents the upper 32 bits of mcycle counter. (R/W)

Register 1.43. minstreth (0xB82)



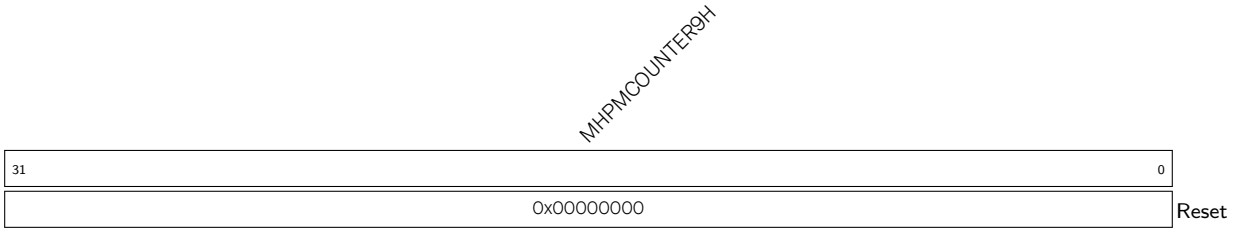
MINSTRETH Represents the upper 32 bits of minstret counter. (R/W)

Register 1.44. mhpmcounter8h (0xB88)



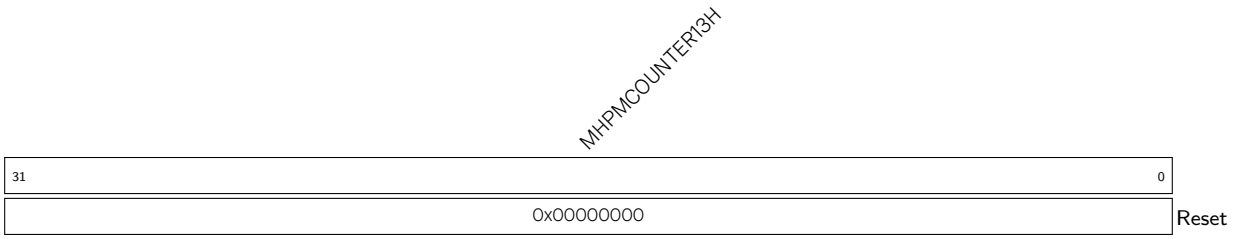
MHPMCOUNTER8H Represents the upper 32 bits of mhpmcounter8 counter. (R/W)

Register 1.45. mhpmcounter9h (0xB89)



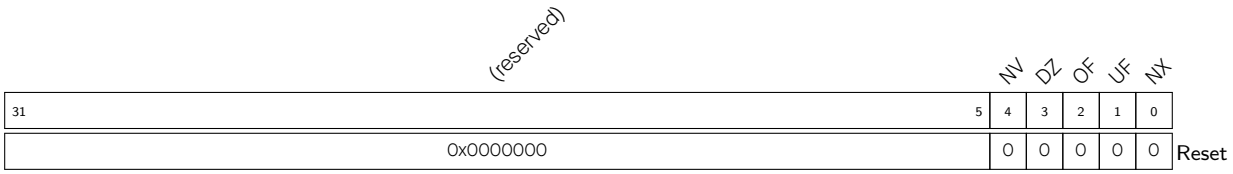
MHPMCOUNTER9H Represents the upper 32 bits of mhpmcounter9 counter. (R/W)

Register 1.46. mhpmcounter13h (0xB8D)



MHPMCOUNTER13H Represents the upper 32 bits of mhpmcounter13 counter. (R/W)

Register 1.47. fflags (0x001)



NV Represents the floating-point invalid operation flag. (R/W)

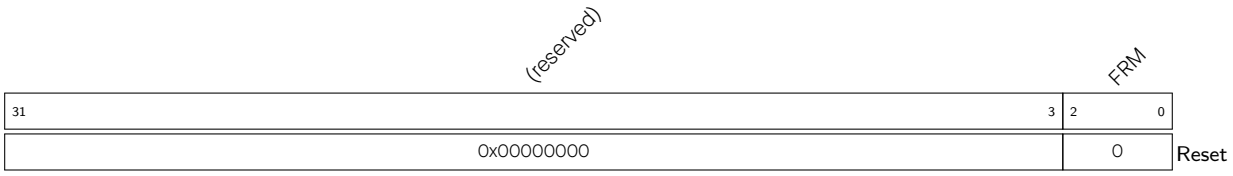
DZ Represents the floating-point divide-by-zero flag. (R/W)

OF Represents the floating-point overflow flag. (R/W)

UF Represents the floating-point underflow flag. (R/W)

NX Represents the floating-point inexact flag. (R/W)

Register 1.48. frm (0x002)



FRM Configures the floating-point rounding mode.

Valid rounding mode values:

0x0: RNE (Round to nearest, ties to even)

0x1: RTZ (Round towards zero)

0x2: RDN (Round down towards negative infinity)

0x3: RUP (Round up towards positive infinity)

0x4: RMM (Round to nearest, ties to maximum magnitude)

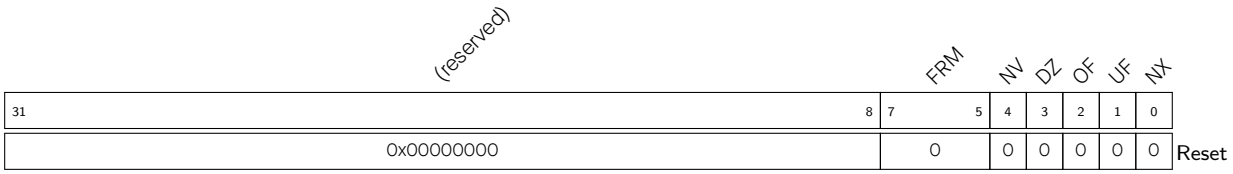
0x5: Reserved (Invalid. Reserved to future use)

0x6: Reserved (Invalid. Reserved to future use)

0x7: DYN (In Instruction's rm field, it selects dynamic rounding mode. In the rounding mode register, it is invalid)

(R/W)

Register 1.49. fcsr (0x003)



FRM Configures the floating-point rounding mode.

Valid rounding mode values:

0x0: RNE (Round to nearest, ties to even)

0x1: RTZ (Round towards zero)

0x2: RDN (Round down towards negative infinity)

0x3: RUP (Round up towards positive infinity)

0x4: RMM (Round to nearest, ties to maximum magnitude)

0x5: Reserved (Invalid. Reserved to future use)

0x6: Reserved (Invalid. Reserved to future use)

0x7: DYN (In Instruction's rm field, it selects dynamic rounding mode. In the rounding mode register, it is invalid)

(R/W)

NV Represents the floating-point invalid operation flag. (R/W)

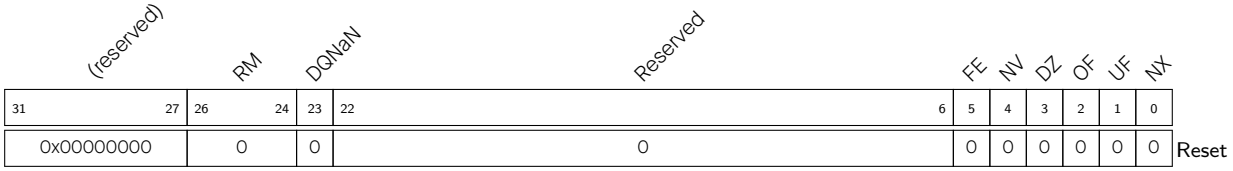
DZ Represents the floating-point divide-by-zero flag. (R/W)

OF Represents the floating-point overflow flag. (R/W)

UF Represents the floating-point underflow flag. (R/W)

NX Represents the floating-point inexact flag. (R/W)

Register 1.50. fxsr (0x800)



RM Mapped to frm bits. Write operation to this field will be reflected on fcsr register at the same time. (R/W)

DQNaN Configures the calculated value of QNaN output.

0: The calculated value of QNaN output is the fixed value specified in RISC-V, i.e., 0x7FC00000.

1: The calculated QNaN value is consistent with the IEEE 754 standard.

(R/W)

FE Floating-point exception accumulation bit. Mapped to FE bits. Write operation to this field will be reflected on fcsr register at the same time.

0: No floating-point exception occurred

1: Floating-point exception occurred

(R/W)

NV Represents the invalid operand exception flag. Mapped to NV bits. Write operation to this field will be reflected on fcsr register at the same time. (R/W)

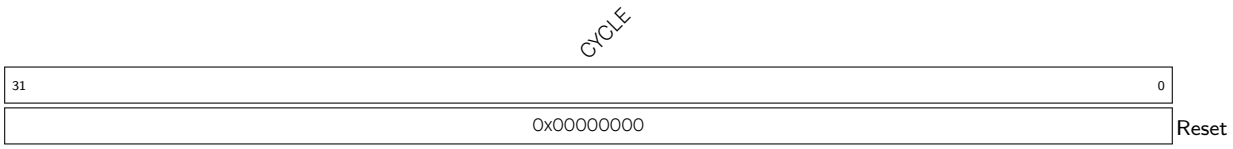
DZ Represents the divide-by-zero exception flag. Mapped to DZ bits. Write operation to this field will be reflected on fcsr register at the same time. (R/W)

OF Represents the overflow exception flag. Mapped to OF bits. Write operation to this field will be reflected on fcsr register at the same time. (R/W)

UF Represents the underflag. Mapped to UF bits. Write operation to this field will be reflected on fcsr register at the same time. (R/W)

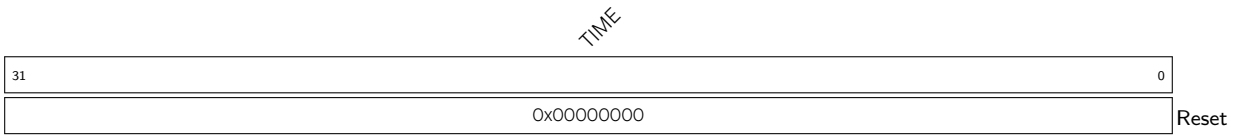
NX Represents the inexact flag. Mapped to NX bits. Write operation to this field will be reflected on fcsr register at the same time. (R/W)

Register 1.51. cycle (0xC00)



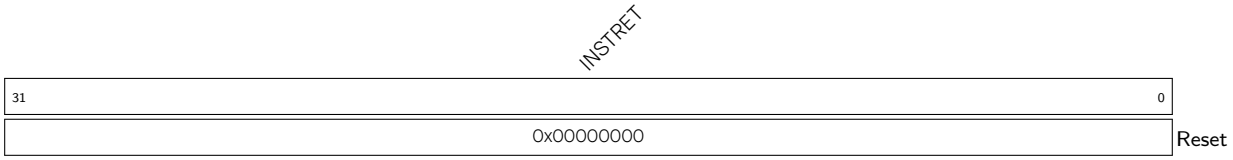
CYCLE Read-only mirror of mcycle. (RO)

Register 1.52. time (0xC01)



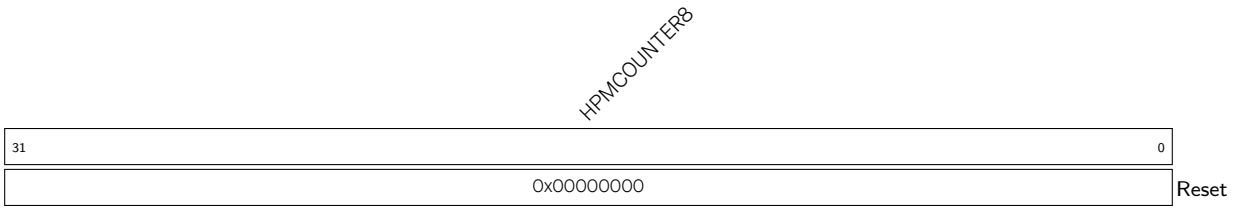
TIME Represents the timer value for RDTIME instruction. (RO)

Register 1.53. instret (0xC02)



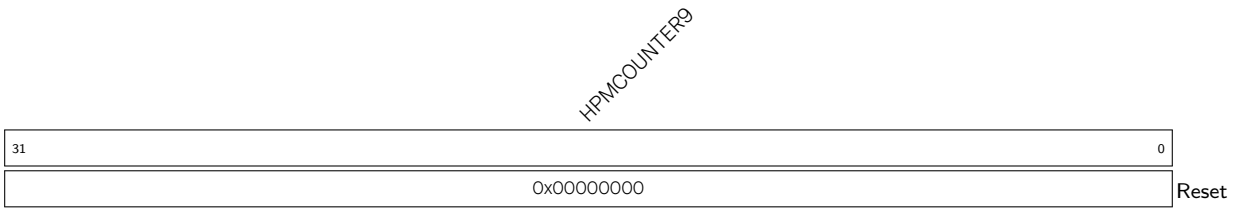
INSTRET Read-only mirror of minstret. (RO)

Register 1.54. hpmcounter8 (0xC08)



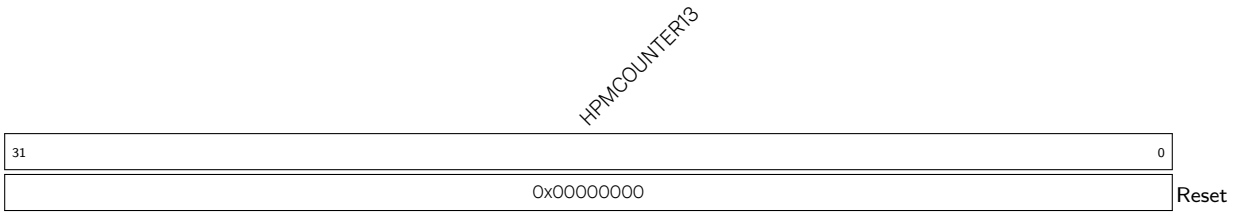
HPMCOUNTER8 Read-only mirror of mhpmcounter8. (RO)

Register 1.55. hpmcounter9 (0xC09)



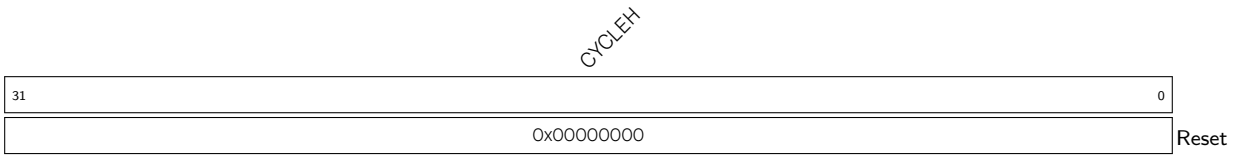
HPMCOUNTER9 Read-only mirror of mhpmcounter9. (RO)

Register 1.56. hpmcounter13 (0xC0D)



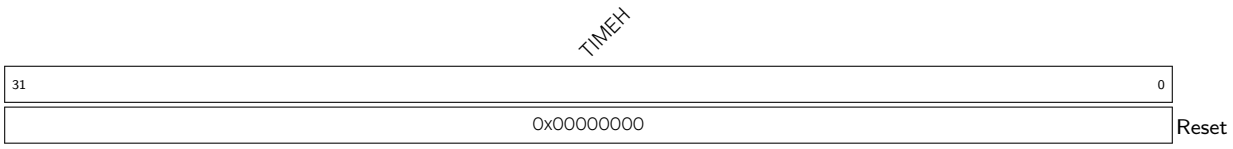
HPMCOUNTER13 Read-only mirror of mhpmcounter13. (RO)

Register 1.57. cycleh (0xC80)



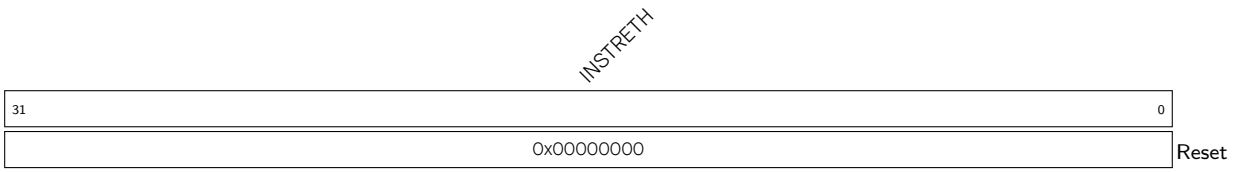
CYCLEH Read-only mirror of mcycleh. (RO)

Register 1.58. timeh (0xC81)



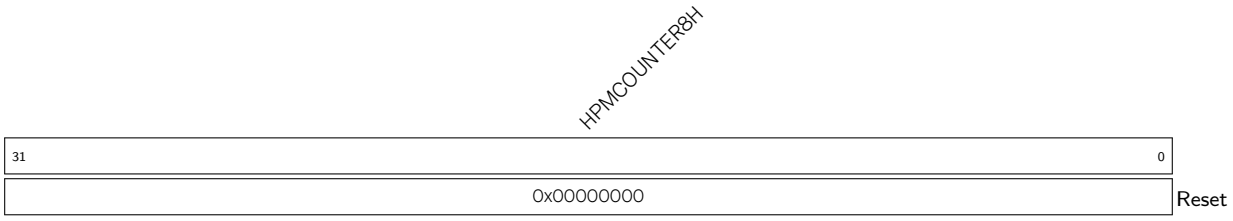
TIMEH Represents the upper 32 bits of the timer for RDTIME instruction. (RO)

Register 1.59. instreth (0xC82)



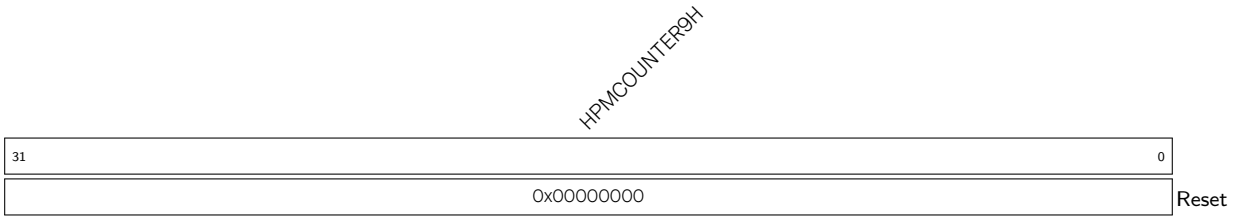
INSTRETH Read-only mirror of minstreth. (RO)

Register 1.60. hpmcounter8h (0xC88)



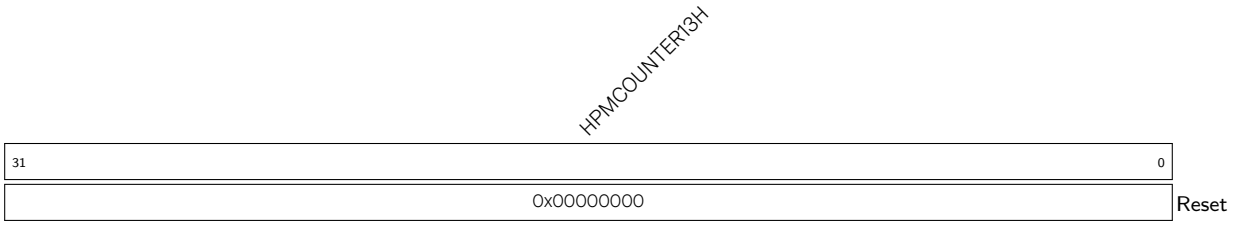
HPMCOUNTER8H Read-only mirror of mhpmcounter8h. (RO)

Register 1.61. hpmcounter9h (0xC89)



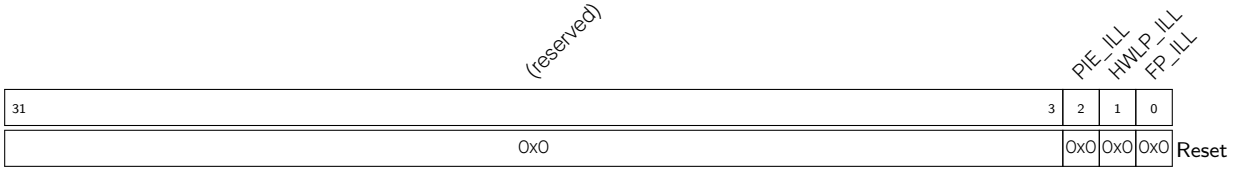
HPMCOUNTER9H Read-only mirror of mhpmcounter9h. (RO)

Register 1.62. hpmcounter13h (0xC8D)



HPMCOUNTER13H Read-only mirror of mhpmcounter13h. (RO)

Register 1.63. mext_ill (0x7F0)

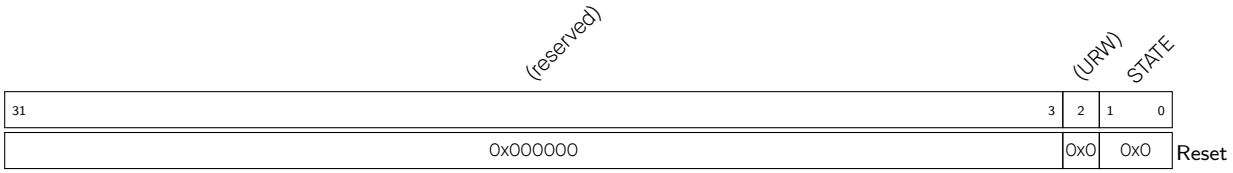


PIE_ILL Represents whether an illegal instruction exception occurred due to the CPU attempting to execute instructions belonging to the disabled PIE extension. (R/W)

HWLP_ILL Represents whether an illegal instruction exception occurred due to CPU attempting to execute instructions belonging to the disabled HWLP extension. (R/W)

FP_ILL Represents whether an illegal instruction exception occurred due to the CPU attempting to execute instructions belonging to the disabled RV32F extension, i.e. mstatus.FS=0x0. (R/W)

Register 1.64. mext_hwlp_status (0x7F1)



URW Represents whether to enable user-mode access for user-mode hardware loop CSRs.

0: Access is disabled

1: Access is enabled

(R/W)

STATE Configures state of the HWLP extension:

0x0: OFF (Accessing HWLP instructions/CSRs will trigger an illegal instruction exception)

0x1: INITIAL

0x2: CLEAN

0x3: DIRTY

Assuming state is not OFF, whenever HWLP CSRs are modified or HWLP instructions are executed the state will be updated to DIRTY. SW can set the state as INITIAL or CLEAN depending on the context. As specified in RISCV standard: When an extension's status is set to OFF, any instruction that attempts to read or write the corresponding state will cause an illegal instruction exception.

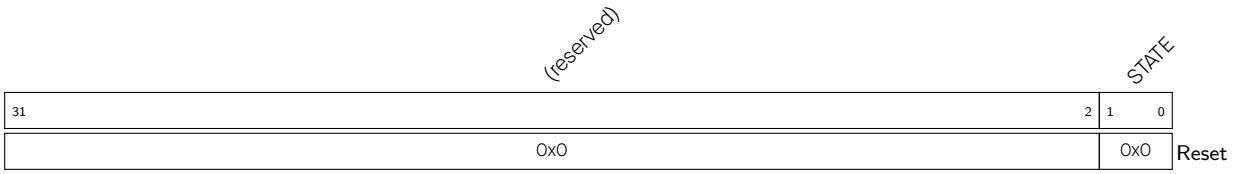
When the status is INITIAL, the corresponding state should have an initial constant value.

When the status is CLEAN, the corresponding state is potentially different from the initial value, but matches the last value stored on a context swap.

When the first iteration of HWLP is complete, the state changes to DIRTY. When the status is DIRTY, the corresponding state has potentially been modified since the last context save.

(R/W)

Register 1.65. mext_pie_status (0x7F2)



STATE Configures the state of the PIE extension:

0x0: OFF (Executing PIE instructions will trigger illegal instruction exception)

0x1: INITIAL

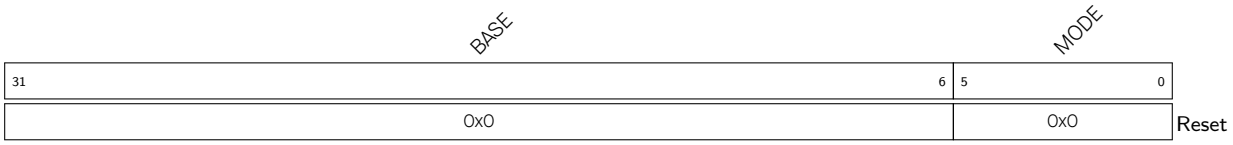
0x2: CLEAN

0x3: DIRTY

This is the extension state bits for the PIE extension, with similar functionality as the mstatus.FS bits of RV32F. Please refer to the "RISC-V Volume II - Privileged Architecture" for the standard interpretation of the STATE values.

(R/W)

Register 1.66. jvt (0x017)



BASE Represents jump table base address. (R/W)

MODE Represents configurable modes. This field is reserved to define the usage of this register. Currently, only one usage is defined, i.e., jump table. Other usages are reserved for future use).

0x0: The JTV_BASE is used as jump table

Others: reserved for future standard use

(R/W)

Register 1.67. mexstatus (0x7E1)

EXPT_VLD This bit is automatically set upon entering a trap handler.

This bit is only writable from debug mode.

(RO)

LOCKUP This bit is automatically set when double exceptions occur.

Upon lockup, the core will stall and wait for the debugger to resolve the lockup.

This bit is only writable from debug mode.

(RO)

BUS_ERR This bit is automatically set upon entering a load/store fault trap due to a bus error exception.

Upon entering the trap handler, this bit must be read to find if the load/store fault is due to a bus error.

When mexstatus.NMFT is enabled, this bit gets cleared automatically on executing MRET instruction.

When mexstatus.NMFT is disabled and mexstatus.PBEXE, this bit must be cleared manually before resuming execution, else it may re-trigger a pending bus error exception.

(R/W)

PBEXE Configures whether to enable the pending bus-error exception.

0: Disable

1: Enable

When mexstatus.PBEXE is enabled and mexstatus.NMFT is disabled, a synchronous exception is generated when mexstatus.BUS_ERR is set.

If mexstatus.NMFT is enabled, mexstatus.PBEXE has no effect.

Upon entering a trap the value of mexstatus.PBEXE is pushed into mexstatus.PPBEXE and mexstatus.PBEXE is set to 0, automatically.

Upon exiting a trap via MRET, the value of mexstatus.PPBEXE is popped back into mexstatus.PBEXE.

(R/W)

PPBEXE Configures whether to enable the previous pending bus-error exception.

0: Disable

1: Enable

Upon entering a trap, the value of mexstatus.PBEXE is pushed into mexstatus.PPBEXE.

Upon exiting a trap via MRET, the value of mexstatus.PPBEXE is popped back into mexstatus.PBEXE and mexstatus.PPBEXE is set to 1, automatically.

(R/W)

Continued on the next page...

Register 1.67. mexstatus (0x7E1)

Continued from the previous page...

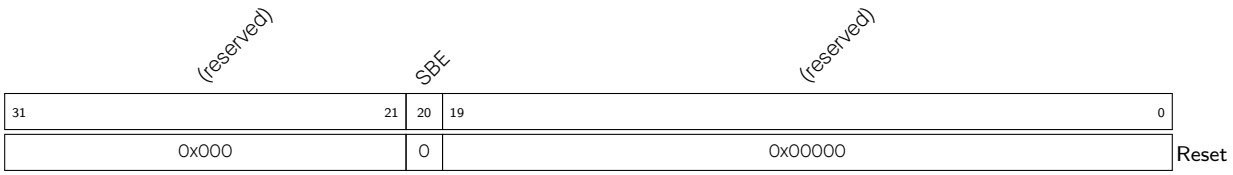
PMP_ERR This bit is automatically set upon entering load/store fault trap due to a PMP/PMA exception.
Upon entering the trap handler, this bit must be read to find if the load/store fault is due to PMP/PMA error.
This bit is cleared automatically upon MRET.
(R/W)

FETCH_ERR This bit is automatically set upon entering the fetch fault trap due to a bus-error exception.
Upon entering the trap handler, this bit must be read to find if the fetch fault is due to bus error.
This bit is cleared automatically upon MRET.
(R/W)

NMFT Configures whether to disable the memory fence on trap.
1: CPU will enter the trap handler upon exceptions/interrupts without waiting for pending memory load/store operations to finish (Default).
0: CPU will wait for pending memory load/store operations to finish before entering the trap handler.
(R/W)

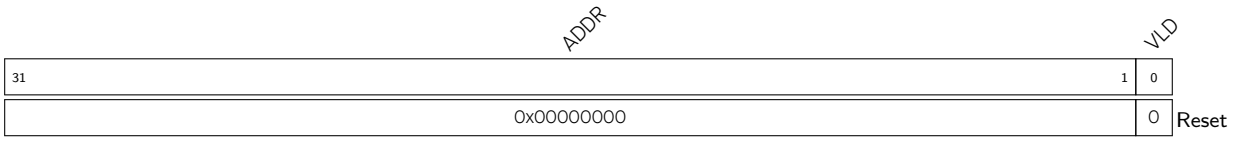
CLIC_INHV Configures whether to enable resuming faulting vector table fetch.
0: Disable
1: Enable (default). When it is set, mcause.minhv is used to decide if mepc holds an instruction address or a vector table address.
(R/W)

Register 1.68. mhint (0x7C5)



SBE Configures whether to enable the synchronous load/store bus-error exception.
0: Disable
1: Enable
(R/W)

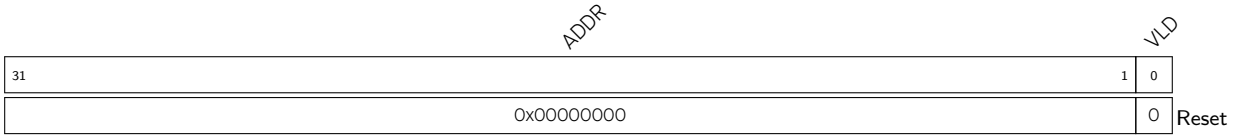
Register 1.69. ldpc0 (0xBE0)



VLD Configures whether the program-counter address value in this CSR is valid for a recent load bus-error exception.
0: Not valid
1: Valid
(R/W)

ADDR Configures the higher 31 bits of the half-word-aligned program counter corresponding to the same load bus-error exception. (R/W)

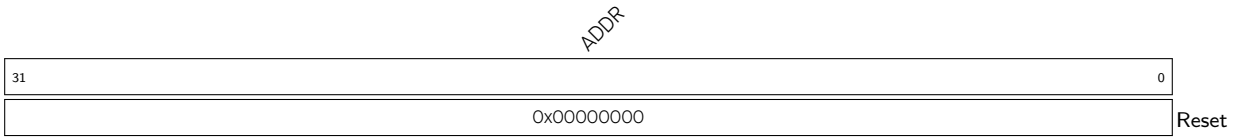
Register 1.70. ldpc1 (0xBE1)



VLD Configures whether the program-counter address value in this CSR is valid for a recent load bus-error exception.
0: Not valid
1: Valid
(R/W)

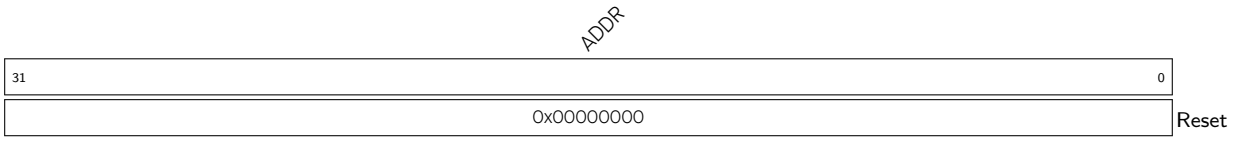
ADDR Configures the higher 31 bits of the half-word-aligned program counter corresponding to the same load bus-error exception. (R/W)

Register 1.71. ldtval0 (0xBE8)



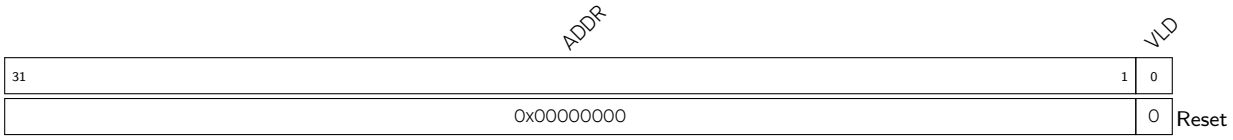
ADDR Configures the access address corresponding to the load bus-error exception recorded in ldpc0. (R/W)

Register 1.72. ldtval1 (0xBE9)



ADDR Configures the access address corresponding to the load bus-error exception recorded in ldpc1. (R/W)

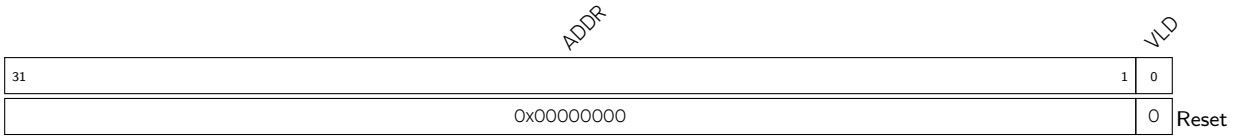
Register 1.73. stpc0 (0xBF0)



VLD Configures whether the program-counter address value in this CSR is valid for a recent store bus-error exception.
0: Not valid
1: Valid
(R/W)

ADDR Configures the higher 31 bits of the half-word-aligned program counter corresponding to the same store bus-error exception. (R/W)

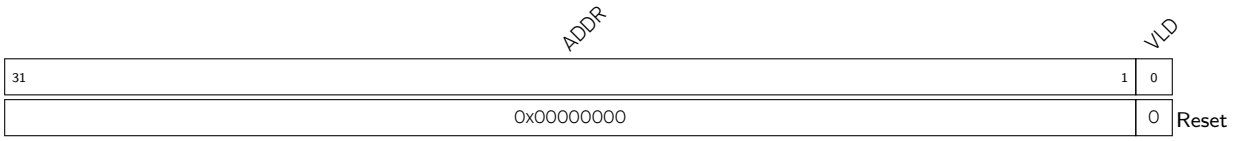
Register 1.74. stpc1 (0xBF1)



VLD Configures whether the program-counter address value in this CSR is valid for a recent store bus-error exception.
0: Not valid
1: Valid
(R/W)

ADDR Configures the higher 31 bits of the half-word-aligned program counter corresponding to the same store bus-error exception. (R/W)

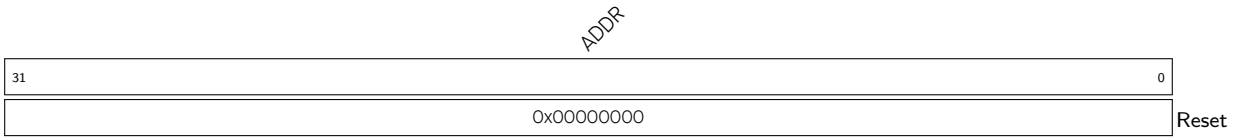
Register 1.75. stpc2 (0xBF2)



VLD Configures whether the program-counter address value in this CSR is valid for a recent store bus-error exception.
0: Not valid
1: Valid
(R/W)

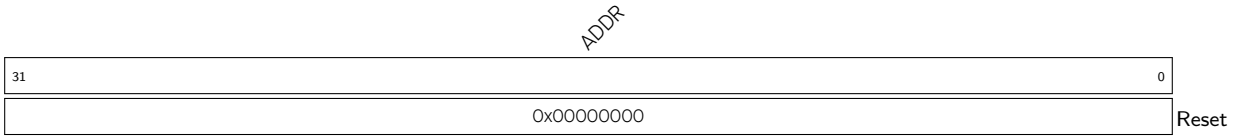
ADDR Configures the higher 31 bits of the half-word-aligned program counter corresponding to the same store bus-error exception. (R/W)

Register 1.76. sttval0 (0xBF8)



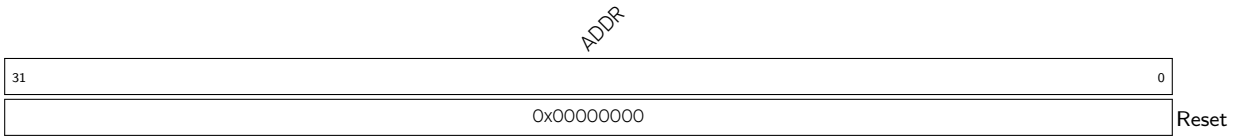
ADDR Configures the access address corresponding to the store bus-error exception recorded in stpc0. (R/W)

Register 1.77. sttval1 (0xBF9)



ADDR Configures the access address corresponding to the store bus-error exception recorded in stpc1. (R/W)

Register 1.78. sttval2 (0xBFA)



ADDR Configures the access address corresponding to the store bus-error exception recorded in stpc2. (R/W)

1.6 RISC-V Standard ISA Extensions Support

Each HP core implements mandatory base integer ISA and is compliant with version 2.0 of RV32I base integer instruction set specified in RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA, Version 2.2.

1.6.1 M Extension

1.6.1.1 Overview

In addition to base RV32I instruction set, each HP core also implements standard integer multiplication and division extension and is fully compliant with version 2.0 of standard extension "M" specified RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA, Version 2.2.

1.6.1.2 Functional Description

Instructions specified in "M" extension perform multiply and divide operations on the values held in two integer registers. Below are CPU cycle details taken by multiply and divide instructions in the absence of any data hazard in the pipeline.

- MUL operation takes 2 cycles
- DIV operation takes 1-19 cycles depending on operands

1.6.2 C Extension

1.6.2.1 Overview

Each HP core also implements RISC-V standard compressed instruction set extension named "C" and is fully compliant with version 2.0 of standard extension "C" specified in RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA, Version 2.2. RVC is generic term used to specify C extension support. RVC reduces static and dynamic code size by adding short 16-bit instruction encoding for common operations. Typically, 50-60% of the RISC-V instructions in a program can be replaced with RVC instructions, resulting in a 25-35% code-size reduction.

1.6.2.2 Functional Description

RVC uses a simple compression scheme that offers a shorter 16-bit version of 32-bit RISC-V instructions for any of the below scenarios:

1. Immediate and address offset is small
2. One of the registers is a zero register (x0), the ABI lint register (x1), or the ABI stack pointer (x2)
3. Destination register and the first source register are identical, or
4. The registers used are the 8 most popular ones i.e. x8-x15.

1.6.3 F Extension

1.6.3.1 Overview

Each HP core implements the RISC-V standard single-precision floating-point instruction set extension, "RV32F" extension in short. The RV32F extension also provides its own register file and various CSRs for configuration and flag checking. Please refer to the RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA, Version 2.2, F Standard Extension Version 2.0" for complete information.

1.6.3.2 Functional Description

- Various instructions to perform floating-point arithmetic operations, conversion/move operations to/from integer format, comparison operations, classification operations, and memory load/store operations.
- Single-precision 32-bit floating-point operations compliant with IEEE 754-2008 standard.
- Separate 32-bit wide floating-point register file with 32 registers.
- Configurable rounding modes.
- Flags for checking inexact, overflow, underflow, invalid and divide by zero outcomes.
- Subnormal and NaN (not a number) arithmetic
- Status bits to control the availability of the extension and track its usage in the context of a program execution.

1.6.3.3 Initialization

The RV32F extension is disabled by default and must be manually enabled using the mstatus.FS bits. Otherwise, an illegal instruction exception will be generated upon executing any of the instructions, or accessing any of the CSRs, which are part of the RV32F extension.

Please refer to the RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10, Extension Context Status in mstatus Register for complete information.

1.6.4 Zc (Z) Extension

1.6.4.1 Overview

Zc extension is useful for code size reduction. Each HP core implements this extension compatible with Zc-specification v1.0.4-3. The Zc-specification has multiple subset extensions, and the ESP32-P4HP CPU has implemented Zcb, Zcmp, and Zcmt extensions.

1.6.4.2 Functional Description

Zcb Extension

The Zcb extension depends on the C extension and provides more compressed instructions for the following scenarios:

- Immediate and address offset is small.
- Destination register and the first source register are identical.
- The registers used are the 8 most popular ones (x8-x15).

Zcmp Extension

The Zcmp extension is a set of instructions which may be executed as a series of existing 32-bit RISC-V instructions.

- PUSH, POP, POPRET operations used to reduce the size of function prologues and epilogues
  - The PUSH operation
    - adjusts the stack pointer to create the stack frame
    - pushes (stores) the registers specified in the register list to the stack frame
  - The POP operation
    - pops (loads) the registers in the register list from the stack frame
    - adjusts the stack pointer to destroy the stack frame
  - The POPRET operation
    - pops (loads) the register in the register list from the stack frame
    - cm.popretz also moves zero into a0 as the return value
    - adjusts the stack pointer to destroy the stack frame
    - executes a ret instruction to return from the function
- Move two a0-a1 into two registers of s0-s7
- Move two s0-s7 into a0-a1

Zcmt Extension

The Zcmt extension is also referred to as table jump. It reads the jump target address from the jump table and then jumps to it.

Table jump uses a 256-entry 32-bit wide table in instruction memory to contain function address. The table must be a minimum of 64-byte aligned. It is used as a form of dictionary compression to reduce the code size of jal, auipc+jalr, jr, auipc+jr instructions.

Table jump allows the linker to replace the following instruction sequences with a cm.jalt encoding, and an entry in the table:

- 32-bit j calls
- 32-bit jal ra calls

1.6.4.3 Zc Instructions

Instruction	Mnemonic	Description
Zcb extension		
c.lbu	c.lbu rd', uimm(rs1')	Loads a byte from the memory address formed by adding rs1' to the zero-extended immediate uimm. The resulting byte is zero-extended to 32 bits and is written to rd'
c.lhu	c.lhu rd', uimm(rs1')	Loads a halfword from the memory address formed by adding rs1' to the zero-extended immediate uimm. The resulting byte is zero-extended to 32 bits and is written to rd'
c.lh	c.lh rd', uimm(rs1')	Loads a halfword from the memory address formed by adding rs1' to the zero-extended immediate uimm. The resulting byte is sign-extended to 32 bits and is written to rd'
c.sb	c.sb rs2', uimm(rs1')	Stores the least significant byte of rs2' to the memory address formed by adding rs1' to the zero-extended immediate uimm
c.sh	c.sh rs2', uimm(rs1')	Stores the least significant halfword of rs2' to the memory address formed by adding rs1' to the zero-extended immediate uimm
c.zext.b	c.zext.b rsd'	Zero-extends the least significant byte of the operand to 32 bits by inserting zeros into all of the bits more significant than 7
c.sext.b	c.sext.b rsd'	Sign-extends the least significant byte of the operand to 32 bits by copying the most significant bit in the byte to all the more significant bits
c.zext.h	c.zext.h rsd'	Zero-extends the least significant halfword of the operand to 32 bits by inserting zeros into all of the bits more significant than 15
c.sext.h	c.sext.h rsd'	Sign-extends the least significant halfword of the operand to 32 bits by copying the most significant bit in the halfword to all the more significant bits
c.not	c.not rsd'	Takes the one's complement of rd'/rs1' and writes the result to the same register
c.mul	c.mul rsd'	Multiplies 32 bits of the source operands from rsd' and rs2' and writes the lowest 32 bits to the result to rsd'
Zcmp extension		
cm.push	cm.push {reg_list}, -stack_adj	Stores the registers in reg_list to the memory below the stack pointer, and then creates the stack frame by decrementing the stack pointer by stack_adj, including any additional stack space requested by the value of spimm

Instruction	Mnemonic	Description
cm.pop	cm.pop {reg_list}, stack_adj	Loads the registers in reg_list from stack memory, and then adjusts the stack pointer by stack_adj
cm.popret	cm.popret {reg_list}, stack_adj	Loads the registers in reg_list from stack memory, adjusts the stack pointer by stack_adj, then return to ra
cm.popretz	cm.popretz {reg_list}, stack_adj	Loads the registers in reg_list from stack memory, adjusts the stack pointer by stack_adj, moves zero to a0 and then return to ra
cm.mva01s	cm.mva01s rs1', rs2'	Moves r1s' into a0 and r2s' into a1
cm.mva01	cm.mva01 r1s', r2s'	Moves a0 into r1s' and a1 into r2s'
Zcmt extension		
cm.jt	cm.jt index	Reads an entry from the jump vector table in memory and jumps to the address that was read
cm.jalt	cm.jalt index	Reads an entry from the jump vector table in memory and jumps to the address that was read, linking to ra

For more details about the instructions please refer to Zc-specification v1.0.4-3.

1.6.4.4 Limitations

jvt adds architecture state to the context. Therefore, it must be saved and restored on context switches.

1.6.5 Bit Manipulation (Zb) Extension

1.6.5.1 Overview

The bit-manipulation (bitmanip) extension, also known as the Zb extension set, consists of several component extensions to the base RISC-V architecture. These extensions aim to improve code density, performance, and energy efficiency. The HP core fully supports the Zb extension. For Zb extension specifications, please refer to Zb-specification v1.0.0.

1.6.5.2 Functional Description

Although the bitmanip instructions are designed for general-purpose use, certain instructions are more applicable to specific domains. Hence, the bitmanip extension set is divided into several smaller extensions instead of a single large one. Each extension has its own Zb*-extension name, and comprises instructions that share related functionality, use cases, and often the same implementation logic. Some instructions are available in only one extension, while others are available in several. The instructions have mnemonics and encodings that are independent of the extensions in which they appear. Thus, when implementing extensions with overlapping instructions, there is no redundancy in logic or encoding.

1.6.6 A Extension

1.6.6.1 Overview

Support for atomic (A) extension is available in compliance with RISC-V Instruction Set Manual, Volume I: RISC-V User-Level ISA, Version 2.2, with an emphasis to guarantee forward progress, i.e. any situation that may cause data memory lock for an indefinite amount of time is prevented by their very functionality.

1.6.6.2 Functional Description

Atomic (A) Extension contains instructions that atomically read-modify-write memory to support synchronization between multiple RISC-V harts running in the same memory space. The two forms of atomic instruction provided are load-reserved/store-conditional instructions and atomic fetch-and-op memory instructions. The atomic instructions currently ignore the aq (acquire) and rl (release) bits as they are irrelevant to the current architecture, in which memory ordering is always guaranteed.

1.6.6.3 Load-Reserve (LR.W) Instruction

The LR.W instruction simply locks a 32-bit aligned memory address to which the load access is being performed. Once a 4-byte memory region is locked, it will remain locked, i.e. other harts won't be able to access this same memory location, until any of the following scenarios is encountered during execution:

- any load operation
- any store operation
- any interrupts/exceptions
- backward jump/taken backward branch
- JALR
- ECALL/EBREAK/MRET/URET
- FENCE/FENCE.I
- debug mode
- critical section exceeding 64 bytes
- data address in SC.W instruction not matching that in LR.W instruction

If any of the above happens, except SC.W, the memory lock will be released immediately. If an SC instruction is encountered instead, the lock will be released eventually (not immediately) in the manner described in Section 1.6.6.4.

If a misaligned address is encountered, it will cause an exception with mcause = 6.

1.6.6.4 Store-Conditional (SC.W) Instruction

The SC.W instruction first checks if the memory lock is still valid, and the address is the same as specified during the last LR.W instruction. If so, only then will it perform the store to memory, and later release the lock as soon as it gets an acknowledgement of operation completion from the memory.

On the other hand, if the lock is found to have been invalidated (due to any of the situations as described in Section 1.6.6.3), it will set a fail code (currently always 1) in the destination register rd.

If a misaligned address is encountered, it will cause an exception with mcause = 6.

1.6.6.5 AMO Instructions

An AMO instruction executes in three steps:

1. Read data from memory address given by rs1, and save it to destination register rd.
2. Combine the data in rd and rs2 according to the operation type and keep the result for Step 3 below.
3. Write the result obtained in Step 2 above to memory address given by rs1.

There are nine different AMO operations: SWAP, ADD, AND, OR, XOR, MAX, MIN, MAXU, and MINU.

During this whole process, the memory address is kept locked from being accessed by other harts. If a misaligned address is encountered, it will cause an exception with mcause = 6.

For AMO operations both load and store access faults (PMP/PMA) are checked in the 1st step itself. For such cases mcause = 7.

1.7 Custom Instruction Extensions

1.7.1 Hardware Loop

1.7.1.1 Overview

HWLP (hardware loop) is the implementation of "For" loop in hardware. A loop is configured with a start and an end address and the number of iterations before its execution. When instruction execution encounters the start address, the loop execution starts. When the end address is executed, it is considered as one loop count and the start address is fetched again. This procedure is repeated until the loop count is equal to the number of iterations configured.

1.7.1.2 Features

HWLP has the following features:

- Execution with no extra penalty compared to conditional branch instructions
- Support for two nested HWLPs
- Configurable using a single instruction (lp.setupi) in case of end address within 1 KB from the configuration instruction

1.7.1.3 Functional Description

The configuration of HWLP is done either through CSRs or through custom instructions of HWLP, which will be described in detail in subsequent sections. Any HWLP needs to be configured with a start, an end address, and an iteration count.

When a single loop is to be set up, either Loop0 or Loop1 CSRs can be configured. Before setting up a HWLP, configure STATE bits in mhwloop_state_reg CSR to any value other than 0 (OFF, default state). Otherwise, accessing HWLP configuration CSRs will cause an illegal instruction exception. To set up and execute HWLP in user mode, URW bit in mhwloop_state_reg CSR should be set to 1.

The core supports a maximum of two HWLPs, which can be executed in nested fashion. Each loop is configured separately in this case. Note that, the inner loop has higher priority than the outer loop in case the end instruction of both loops is common. Hardware Loop0 has to be considered as the inner loop and Loop1 as the outer loop while configuring nested loops, as Loop0 has a higher priority than Loop1 in hardware implementation.

The advantage of a HWLP over a conditional branch instruction is that it has zero penalty for its execution. Branch/JUMP instructions use prediction logic for evaluating branch condition and branch/jump address. If a miss-prediction happens, the core pipeline will be flushed and the correct address will be fetched, and thus a few cycles have already been wasted in fetch of the incorrect instructions before flush. As the HWLP does not need prediction of result or target address of conditional branch, it has no penalty, and thus has a higher throughput.

1.7.1.4 Instructions/Operations/Modes Supported in HWLP

- Load type instructions
- Store type instructions

- Arithmetic, logical, AIA instructions
- Zc-extension instructions
- Any interrupts/exceptions within the HWLP
- ECALL/EBREAK
- FENCE/FENCE.I/CSRx instructions
- Atomic instructions
- Debug mode
- Data address in SC.W instruction not matching that in LR.W instruction

1.7.1.5 HWLP Constraints

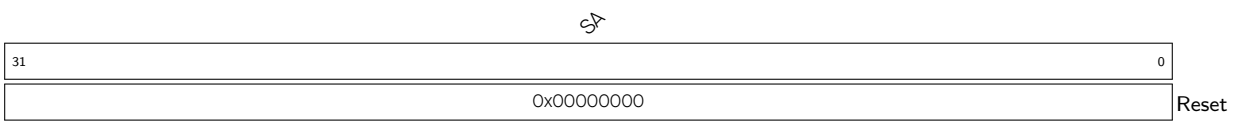
Please note that branch or jump instructions at the end of HWLP may cause unexpected behavior.

1.7.1.6 Register Summary

Name	Description	Address	Access
mhwloop0_start_addr	Machine Loop0 start address	0x7C6	R/W
mhwloop0_end_addr	Machine Loop0 end address	0x7C7	R/W
mhwloop0_count	Machine Loop0 count	0x7C8	R/W
mhwloop1_start_addr	Machine Loop1 start address	0x7C9	R/W
mhwloop1_end_addr	Machine Loop1 end address	0x7CA	R/W
mhwloop1_count	Machine Loop1 count	0x7CB	R/W
mext_ill_reg	Machine extension illegal register	0x7F0	R/W
mhwloop_state_reg	Machine HWLP state register	0x7F1	R/W
uhwloop0_start_addr	User Loop0 start address	0x8C0	R/W
uhwloop0_end_addr	User Loop0 end address	0x8C1	R/W
uhwloop0_count	User Loop0 count	0x8C2	R/W
uhwloop1_start_addr	User Loop1 start address	0x8C3	R/W
uhwloop1_end_addr	User Loop1 end address	0x8C4	R/W
uhwloop1_count	User Loop1 count	0x8C5	R/W
uhwloop_state_reg	User HWLP state register	0x8C6	R/W

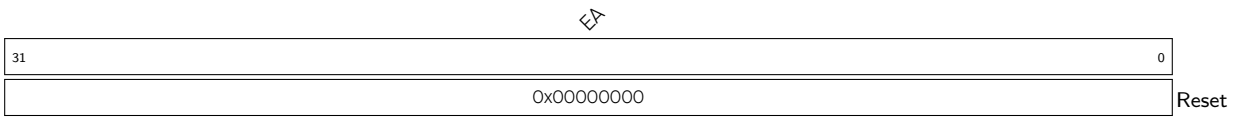
1.7.1.7 Register Description

Register 1.79. mhwloop0_start_addr (0x7C6)



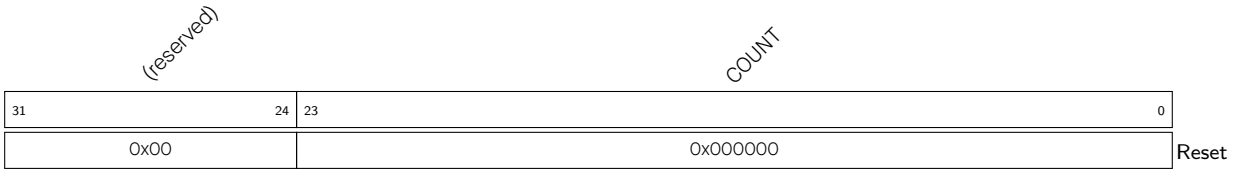
SA Configures the 32-bit address of the first instruction of HW Loop0. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. (R/W)

Register 1.80. mhwloop0_end_addr (0x7C7)



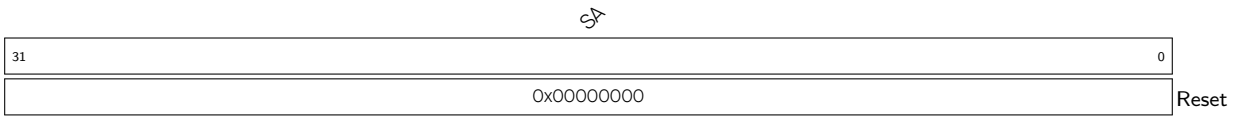
EA Configures the 32-bit address of the last instruction of HW Loop0. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. (R/W)

Register 1.81. mhwloop0_count (0x7C8)



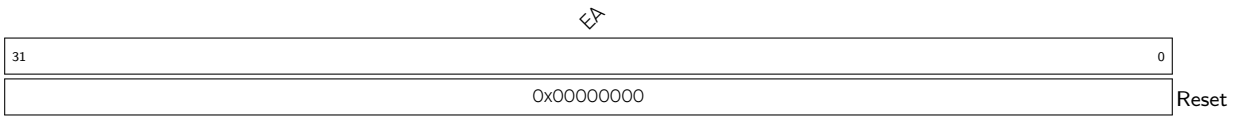
COUNT Configures the number of iterations of HW Loop0. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. (R/W)

Register 1.82. mhwloop1_start_addr (0x7C9)



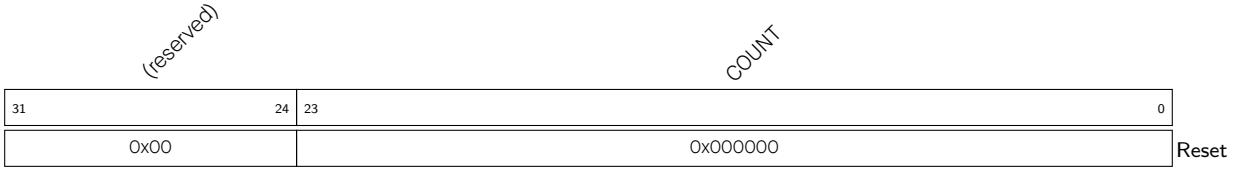
SA Configures the 32-bit address of the first instruction of HW Loop1. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. (R/W)

Register 1.83. mhwloop1_end_addr (0x7CA)



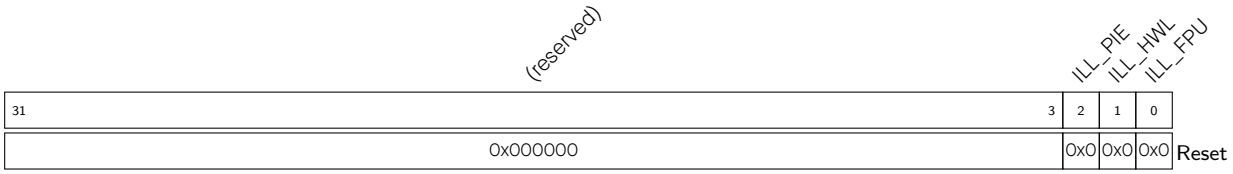
EA Configures the 32-bit address of the last instruction of HW Loop1. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. (R/W)

Register 1.84. mhwloop1_count (0x7CB)



COUNT Configures the number of iterations of HW Loop1. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. (R/W)

Register 1.85. mext_ill_reg (0x7F0)



ILL_FPU Represents whether an illegal instruction is triggered due to the disabled F extension.

0: Not triggered

1: Triggered

(RO)

ILL_HWL Represents whether the illegal instruction is triggered due to the disabled HWLP extension.

0: Not triggered

1: Triggered

(RO)

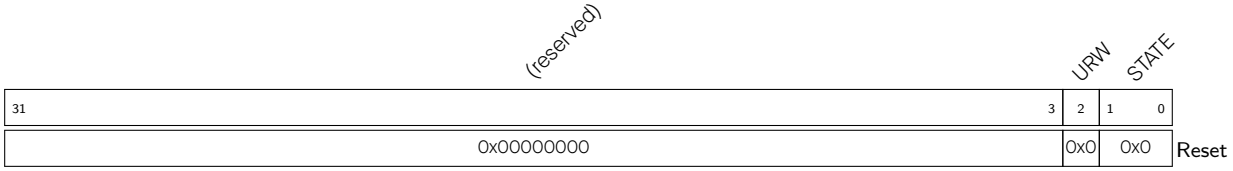
ILL_PIE Represents whether the illegal instruction is triggered due to the disabled PIE extension.

0: Not triggered

1: Triggered

(RO)

Register 1.86. mhwloop_state_reg (0x7F1)



STATE Configures the state of the HWLP extension.

0x0: OFF (Accessing HWLP instructions and CSRs will trigger an illegal instruction exception)

0x1: INITIAL

0x2: CLEAN

0x3: DIRTY

(R/W)

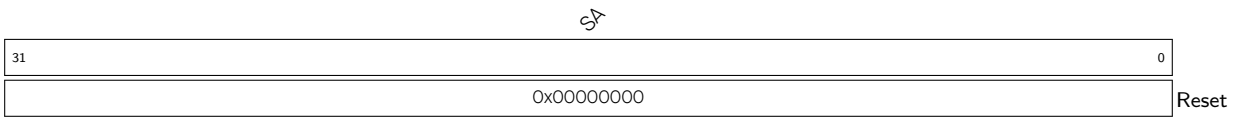
URW Configures user-mode access for user-mode HWLOOP CSRs.

0: User-mode access is disabled

1: User-mode access is enabled

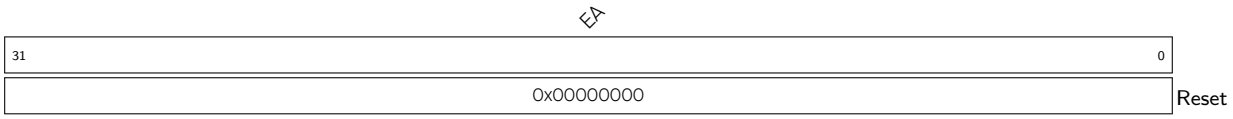
(R/W)

Register 1.87. uhwloop0_start_addr (0x8C0)



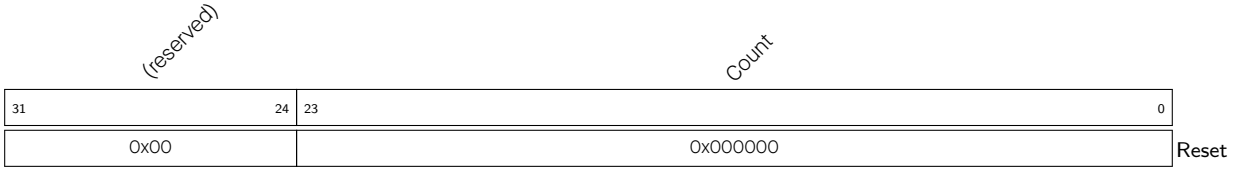
SA Configures the 32-bit address of the first instruction of HW Loop0 in user mode. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. It is read-writable only when mhwloop_state_reg.URW is 1. (R/W)

Register 1.88. uhwloop0_end_addr (0x8C1)



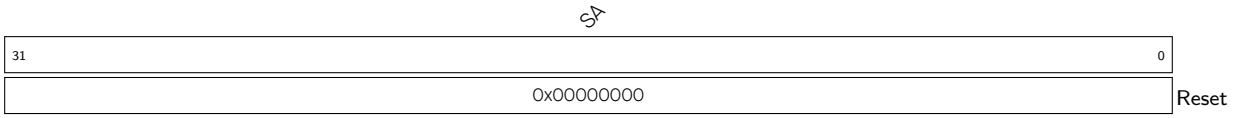
EA Configures the 32-bit address of the last instruction of HW Loop0 in user mode. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. It is read-writable only when mhwloop_state_reg.URW is 1. (R/W)

Register 1.89. uhwloop0_count (0x8C2)



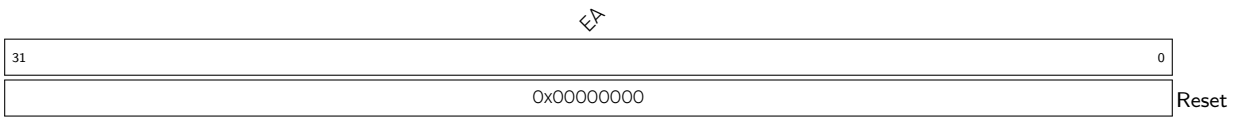
Count Configures the number of iterations of HW Loop0 in user mode. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. It is read-writable only when mhwloop_state_reg.URW is 1. (R/W)

Register 1.90. uhwloop1_start_addr (0x8C3)



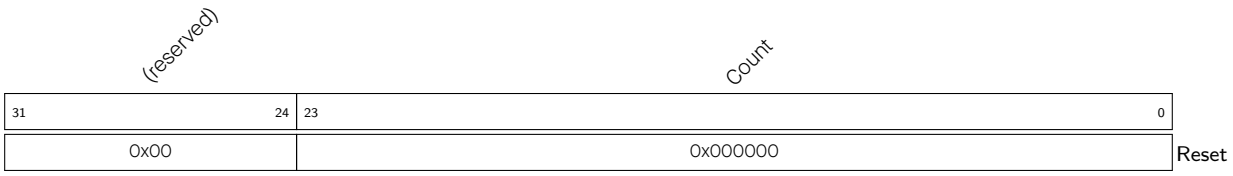
SA Configures the 32-bit address of the first instruction of HW Loop1 in user mode. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. It is read-writable only when mhwloop_state_reg.URW is 1. (R/W)

Register 1.91. uhwloop1_end_addr (0x8C4)



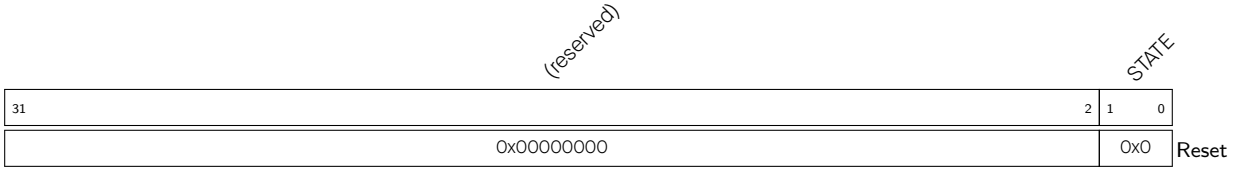
EA Configures the 32-bit address of the last instruction of HW Loop1 in user mode. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. It is read-writable only when mhwloop_state_reg.URW is 1. (R/W)

Register 1.92. uhwloop1_count (0x8C5)



Count Configures the number of iterations of HW Loop1 in user mode. Valid when mhwloop_state_reg.STATE is not 0x0, that is, not OFF. It is read-writable only when mhwloop_state_reg.URW is 1. (R/W)

Register 1.93. uhwloop_state_reg (0x8C6)



STATE Configures the state of the HWLP extension in user mode.

0x0: OFF (Accessing HWLP instructions and CSRs will trigger an illegal instruction exception)

0x1: INITIAL

0x2: CLEAN

0x3: DIRTY

Bits are physically the same as mhwloop_state_reg. Ensure the permission is enabled in mhwloop_state_reg.URW field.

Assuming state is not OFF, whenever HWLP CSRs are modified, or HWLP instructions are executed, the state will be updated to DIRTY.

(R/W)

1.7.1.8 HWLP Instructions

The table below contains the HWLP instruction set.

Table 1.7-2. HWLP Instructions

SN	Instruction Mnemonic	Description	31:20	19:15	14:12	11:8	7	6:0
1	lp.setupi L, uimm_Count, uimm_EndOffset	StartAddr = PC+4 EndAddr = PC+uimm_EndOffset Count = uimm_Count	uimm_Count [11:0]	uimm_EndOffset [5:1]	101	uimm_EndOffset [9:6]	L (index)	0101011
2	lp.setup L, rs1, uimm_EndOffset	StartAddr = PC+4 EndAddr = PC+uimm_EndOffset Count = GPR[rs1]	uimm_EndOffset [12:1]	rs1 [4:0] (count)	100	X	L (index)	0101011
3	lp.setup.beqz L, rs1, EndInstrSize, uimm_EndOffset	StartAddr = PC+4 EndAddr = PC+uimm_EndOffset Count = GPR[rs1] Size of HWLP end instruction = 32 bits if EndInstrSize is 1, 16 bits if EndInstrSize is 0	uimm_EndOffset [12:1]	rs1 [4:0] (count)	100	Size of end instruction (0000 for 16-bit instruction and 0010 for 32-bit instruction)	L (index)	0101011
4	lp.starti L, uimm_StartOffset	StartAddr = PC+uimm_StartOffset	uimm_StartOffset [12:1]	X	000	X	L (index)	0101011
5	lp.endi L, uimm_EndOffset	EndAddr = PC+uimm_EndOffset	uimm_EndOffset [12:1]	X	001	X	L (index)	0101011
6	lp.count L, rs1	Count = GPR[rs1]	X	rs1 [4:0] (count)	010	X	L (index)	0101011
7	lp.counti L, uimm_Count	Count = uimm_Count	uimm_Count [11:0]	X	011	X	L (index)	0101011

1.7.2 Processor Instruction Extension

1.7.2.1 Overview

The Processor Instruction Extension (PIE) ISA entails various custom 128-bit wide SIMD (Single Instruction Multiple Data) and complex instructions for accelerating edge inference and digital signal processing applications. These instructions are fully integrated into the CPU pipeline for seamless use with the standard RV32IMAFC ISA with which the HP core is compliant. PIE Extension support is compliant to Espressif's PIE V2.2.0 version.

1.7.2.2 Functional Description

- 8 x 128-bit wide integer vector register files
- Each 128-bit wide vector register can be used as 16 x 8-bit elements, 8 x 16-bit elements, or 4 x 32-bit elements for SIMD operations
- Up to 128-bit wide memory load/store operations directly to/from the vector registers
- Fused instructions for parallel execution of arithmetic and memory operations
- Vector and scalar MAC units capable of operating on 16 x 8-bit elements or 8 x 16-bit elements per cycle
- 512-bit wide vector register for accumulating results of 8/16-bit vector MAC operations
- 40-bit wide scalar register for accumulating results of 8/16-bit scalar MAC operations
- Instructions for parallel 16-bit FFT/complex math operations
- Final results are generated after rounding and saturation
- Configurable rounding modes and scaling (arithmetic right shift) parameter

For a complete description of all the available instructions and configurations, please refer to Chapter 2 *Processor Instruction Extensions [to be added later]*.

1.7.2.3 Initialization, Context Switching, and Exceptions

The PIE extension is disabled by default and must be manually enabled using the mext_pie_status.STATE field. Otherwise, an illegal instruction exception will be generated upon executing any of the instructions that are part of the PIE extension.

To indicate that an illegal instruction exception, i.e. exception with mcause = 0x2, has occurred due to disabled PIE extension, the HP core will set the mext_ill.PIE_ILL bit.

When the PIE extension is enabled, i.e. mext_pie_status.STATE bits are non-zero, the execution of any PIE instructions would always cause the mext_pie_status.STATE bits to change to DIRTY (11).

To enable the PIE extension, follow the steps below:

1. Set the mext_pie_status.STATE bits to the INITIAL (01) state.
2. Use the PIE instructions to initialize the PIE register file and accumulator registers to zero, which will cause the mext_pie_status.STATE bits to change to DIRTY (11) automatically.
3. Since the value of the PIE registers is known at this point, set the mext_pie_status.STATE bits to CLEAN (10).

4. Execute the application program.
5. During execution of the application code, if any PIE instruction is encountered, the mext_pie_status.STATE bits will automatically change to DIRTY (11).

During context switching, whether to save/restore the PIE extension registers can be decided based on the value of the mext_pie_status.STATE bits. If the value is DIRTY, the state of the PIE register must be saved to the stack. After that, the state would be updated to CLEAN or OFF, depending on whether the next context (thread) has the PIE extension enabled or not.

Some PIE instructions have constraints on the register fields or immediate fields. e.g. for fused instructions which are performing load and arithmetic, with the same destination register for the two parallel operations, it will be considered ambiguous and thus cause a PIE illegal exception with mcause = 0x1f.

1.8 Memory-Mapped Registers

1.8.1 Overview

The tightly-coupled memory-mapped registers below are present within the core complex:

- CLIC registers for each HP core
- Shared CLINT registers

1.8.2 Features

Each HP CPU core supports:

- Access to the other HP core's CLIC registers
- Bus-error exception generation on invalid address access
- Access to CLINT registers. However, only Core 0 can modify CLINT timer value

1.8.3 Functional Description

The table below shows the address decoding used by each HP core to access CLIC and CLINT address range.

Table 1.8-1. Address Decoding for Memory-Mapped Register Access

Bits	Description
31:28	Fixed as 0x2, i.e. base address
27:20	Identifies the sub module: 0x0: CLINT 0x08: CLIC 0xB: CLIC user Others: Reserved
19:16	Controls access type 0x0: Access own registers 0x1: Access the other core's registers
15:0	Register offset

Table 1.8-2. Internal Memory Address Map

Block	Start Address	End Address
CLINT (Self)	0x20000000	0x2000FFFF
CLIC (Self)	0x20800000	0x2080FFFF
CLINT (Other Core)	0x20010000	0x2001FFFF
CLIC (Other Core)	0x20810000	0x2081FFFF

1.9 Interrupt Controller

1.9.1 Overview

Each HP core uses the CLIC + CLINT scheme for implementing and supporting interrupts.

CLINT stands for core-local interrupts, which are sources local to the HP core subsystem for generating timer and software interrupts for each HP core.

CLIC stands for core-local interrupt controller, which provides low-latency, vectored, pre-emptive interrupts for each HP core. Each HP core CLIC unit supports 32 external interrupts and 2 CLINT interrupts.

1.9.2 CLIC

1.9.2.1 Overview

The HP core CLIC implementation follows the official specification: Core-Local Interrupt Controller (CLIC) RISC-V Privileged Architecture Extensions (10/10/2023).

Only the CLIC mode of operation is supported by the HP core, i.e. mtvec.MODE is hardwired to 0x3. Hence, the basic RISC-V interrupt handling scheme and the associated CSRs (such as mie, mip, mideleg, uie, and uip) are unavailable.

HP core supports machine mode interrupt as well as user mode interrupt delegation.

1.9.2.2 CLIC CSRs

The following machine mode CSRs are relevant for interrupt handling in the CLIC scheme:

- mstatus
- mtvec
- mtvt
- mscratch
- mepc
- mcause
- mnxti
- mintthresh
- mintstatus
- mclicbase
- mscratchcsw
- mscratchcswl

As mentioned earlier, the CLIC mode is decided by the mtvec.MODE field. In each HP core, these bits are hardwired to 0x3, indicating that only the CLIC mode is supported.

In CLIC mode, the global interrupt enable for machine mode is still dictated by the mstatus.MIE bit.

interrupt can be either vectored or non-vectored depending on the clicintattr[i].SHV bit.

For a non-vectored interrupt, the HP core jumps to the trap handler base address as configured in mtvec.BASE field.

For a vectored interrupt, the HP core jumps to the 32-bit word address stored at the corresponding index in the vector table. For example, for the "i"th machine mode interrupt, first the 32-bit address stored at the memory location "mtvt + 4*i" will be loaded, after which the HP core will treat the loaded data as an address and jump to it. The base address of the vector table is configured in mtvt.

Upon taking an interrupt, the HP core will store the address of the instruction that was interrupted in mepc. Also, the HP core will update the value of mcause based on the interrupt's ID.

1.9.2.3 Interrupt Mapping

CLIC in each HP core supports 32 external interrupts and 2 core-local interrupts.

The IDs 16-47 are mapped to the external interrupts, while IDs 15 and below are reserved for the core-local interrupts.

Table 1.9-1. CLIC Interrupt Mapping

ID	Description
0-2	NA (Reserved)
3	CLINT M mode software interrupt
4-6	NA (Reserved)
7	CLINT M mode timer interrupt
8-15	NA (Reserved)
16	External interrupt 0
17	External interrupt 1
18	External interrupt 2
...	...
45	External interrupt 29
46	External interrupt 30
47	External interrupt 31

For detailed description of the core-local interrupt sources, please refer to Section 1.9.3.

The mapping of the 32 external interrupts to the interrupt sources in SoC is managed using the Interrupt Matrix. Please refer to Chapter 12 *Interrupt Matrix* for details on how to configure and map to SoC interrupts to CLIC interrupts for each HP core.

1.9.2.4 CLIC Parameters

The available features of a CLIC implementation can be discovered using the registers mclicbase and clicinfo:

- mclicbase is a read-only CSR which holds the base address of the CLIC memory-mapped registers for machine mode.
- clicinfo is a memory-mapped read-only register. It consists of the following fields:
  - NUM_INTERRUPTS: Represents the number of interrupts supported by the current CLIC implementation
  - CLICINTCTLBITS: Represents the number of programmable higher bits in clicintctl[i] and mintthresh.TH for encoding the level and priority of each interrupt
- Once the value of the mclicbase is known, the full address of the clicinfo can be computed by adding the corresponding address offset of 0x0004 (shown in Section 1.9.2.6) to the mclicbase value.

1.9.2.5 Interrupt Level and Priority Encoding

Table 1.9-2 shows the possible ways that level and priority can be specified for CLIC interrupts, based on the configured values of mcliccfg.MNLBITS and clicintctl[i] registers, and the fixed value of the CLICINTCTLBITS

parameter (0x3 for this CLIC implementation).

Table 1.9-2. CLIC Level and Priority Encoding with CLICINTCTLBITS=3

mnlbits[3:0]	clicintctl[i][7:0]	Interrupt Levels	Interrupt Priorities
0	`ppp.....`	255	31, 63, 95, 127, 159, 191, 223, 255
1	`lpp.....`	127, 255	31, 63, 95, 127
2	`llp.....`	63, 127, 191, 255	31, 63
3-8	`lll.....`	31, 63, 95, 127, 159, 191, 223, 255	31

mcliccfg.MNLBITS decides the number of higher bits in the clicintctl[i] registers which are to be used to encode the level (indicated as `'l'` in the above table), while the remaining lower bits are used to encode the priority (indicated as `'p'` in the above table). Since only the higher bits of clicintctl[i] are writeable as per the CLICINTCTLBITS parameter, hence the remaining lower bits are tied to 1 (indicated as '.' in the above table).

Level and priority are both used to give preference to one interrupt over the other. A higher level interrupt can preempt a lower level interrupt (assuming that mstatus.MIE is manually enabled inside an interrupt handler for allowing nesting), but in the case of two interrupts with same level and different priorities, the higher priority interrupt cannot preempt the lower priority interrupt while it is being serviced by the CPU. Thus, priority is only used as a tie-breaker to decide between two pending interrupts with the same level. Finally, if both level and priority are same for two interrupts, the one with the higher ID value is given preference.

The CPU internally uses the effective interrupt level threshold, which is defined as the maximum of mintstatus.MIL and mintthresh.TH, to decide if a new pending machine mode interrupt will be allowed to preempt the execution of an ongoing machine mode interrupt handler.

1.9.2.6 CLIC Memory-Mapped Register Summary

Listed below are all the CLIC machine mode memory-mapped registers. The address of each register is specified as an offset to the value of mclicbase, as well as the absolute address.

Note that the clicintip[i], clicintie[i], clicintattr[i] and clicintctl[i] registers are all byte sized. Still word and half-word aligned accesses to these registers will be performed atomically.

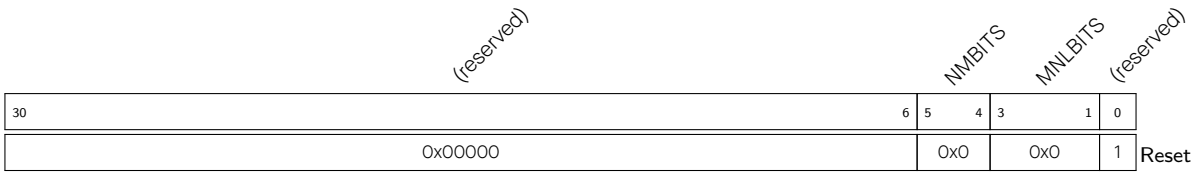
Machine Mode CLIC Registers				
Name	Description	MCLICBASE Offset	Absolute Address	Access
mcliccfg	CLIC machine mode global configuration register	0x0000	0x20800000	R/W
clicinfo	CLIC information register	0x0004	0x20800004	RO
mintthresh	CLIC machine mode interrupt threshold register	0x0008	0x20800008	R/W
clicintip[3]	Pending register for CLINT software interrupts	0x100C	0x2080100C	R/W
clicintie[3]	Enable register for CLINT software interrupts	0x100D	0x2080100D	R/W
clicintattr[3]	Attribute register for CLINT software interrupts	0x100E	0x2080100E	R/W
clicintctl[3]	Level control register for CLINT software interrupts	0x100F	0x2080100F	R/W
clicintip[7]	Pending register for CLINT timer interrupts	0x101C	0x2080101C	R/W
clicintie[7]	Enable register for CLINT timer interrupts	0x101D	0x2080101D	R/W
clicintattr[7]	Attribute register for CLINT timer interrupts	0x101E	0x2080101E	R/W

Name	Description	MCLICBASE Offset	Absolute Address	Access
clicintctl[7]	Level control register for CLINT timer interrupts	0x101F	0x2080101F	R/W
clicintip[16]	Pending register for external interrupt 0	0x1040	0x20801040	R/W
clicintie[16]	Enable register for external interrupt 0	0x1041	0x20801041	R/W
clicintattr[16]	Attribute register for external interrupt 0	0x1042	0x20801042	R/W
clicintctl[16]	Level control register for external interrupt 0	0x1043	0x20801043	R/W
clicintip[17]	Pending register for external interrupt 1	0x1044	0x20801044	R/W
clicintie[17]	Enable register for external interrupt 1	0x1045	0x20801045	R/W
clicintattr[17]	Attribute register for external interrupt 1	0x1046	0x20801046	R/W
clicintctl[17]	Level control register for external interrupt 1	0x1047	0x20801047	R/W
...	...	...	...	...
clicintip[n]	Pending register for nth external interrupt [6]	0x1040 + 4*n	0x20801040 + 4*n	R/W
clicintie[n]	Enable register for nth external interrupt [6]	0x1041 + 4*n	0x20801041 + 4*n	R/W
clicintattr[n]	Attribute register for nth external interrupt [6]	0x1042 + 4*n	0x20801042 + 4*n	R/W
clicintctl[n]	Level control register for nth external interrupt [6]	0x1043 + 4*n	0x20801043 + 4*n	R/W
....	....	....	....	....
clicintip[46]	Pending register for external interrupt 30	0x10B8	0x208010B8	R/W
clicintie[46]	Enable register for external interrupt 30	0x10B9	0x208010B9	R/W
clicintattr[46]	Attribute register for external interrupt 30	0x10BA	0x208010BA	R/W
clicintctl[46]	Level control register for external interrupt 30	0x10BB	0x208010BB	R/W
clicintip[47]	Pending register for external interrupt 31	0x10BC	0x208010BC	R/W
clicintie[47]	Enable register for external interrupt 31	0x10BD	0x208010BD	R/W
clicintattr[47]	Attribute register for external interrupt 31	0x10BE	0x208010BE	R/W
clicintctl[47]	Level control register for external interrupt 31	0x10BF	0x208010BF	R/W

[6] It must be noted that the external interrupts are assigned to CLIC interrupt IDs 16 and onwards. Thus, the MCLICBASE offset can also be written as 0x1000 + 4*i, where "i" represents the CLIC ID, and n = i + 16. On the other hand, the descriptions of these registers use the CLIC index "i" to specify the offset instead of the external interrupt index "n", which is used here just for a matter of convenience.

1.9.2.7 CLIC Memory-Mapped Register Description

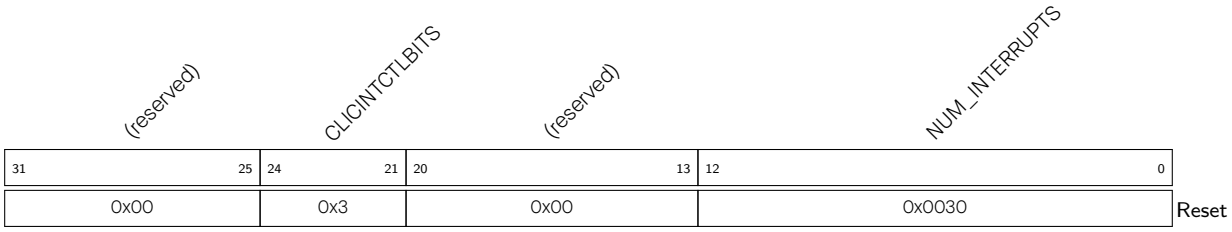
Register 1.94. mcliccfg (0x20800000)



MNLBITS Configures the number of upper bits that are used globally (for each HP core) to encode the interrupt level in all the 8-bit clicintctl[i] registers for each machine mode interrupt. The only allowed values are in the range 0-8. Any higher value will be clamped to 8. (R/W)

NMBITS Configures the number of bits in clicintattr[i].MODE to be used globally (for each HP core) to represent the mode of an interrupt. For systems with machine mode only, this is hardwired to 0. This indicates that the clicintattr[i].MODE field is hardwired to 0x3 and therefore all interrupts are in machine mode only. (RO)

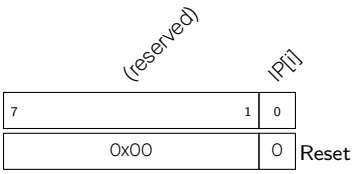
Register 1.95. clicinfo (0x20800004)



NUM_INTERRUPTS Represents the number of interrupts supported by this implementation of CLIC. Hardwired to 48. (RO)

CLICINTCTLBITS Represents the number of higher bits which are actually programmable in the 8-bit clicintctl[i] registers. Hardwired to 0x3. (RO)

Register 1.96. clicintip[i] (0x20801000 + 4*i)



IP[i] Represents the pending state for ith CLIC machine mode interrupt.

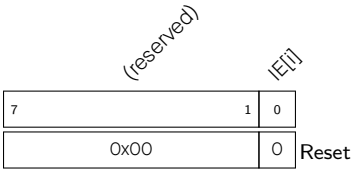
0: ith interrupt is not pending

1: ith interrupt is pending

- When the ith interrupt is configured as level type using clicintattr[i].TRIG, this bit is read-only, and to clear it the interrupt must be cleared from source.
- When the ith interrupt is configured as edge triggered using clicintattr[i].TRIG, this bit is R/W, and thus it is to be cleared by writing 0 to this register.

(R/W)

Register 1.97. clicintie[i] (0x20801001 + 4*i)



IE[i] Configures whether to enable the ith CLIC machine mode interrupt.

0: Disable

1: Enabled

(R/W)

Register 1.98. clicintattr[i] (0x20801002 + 4*i)

SHV Configures hardware vectoring for the ith CLIC machine mode interrupt.

0: ith interrupt is not hardware vectored. It means that upon taking this interrupt, the CPU will jump to the address configured in mtvec.

1: ith interrupt is hardware vectored. It means that upon taking this interrupt, the CPU will jump to the word address stored at (mtvt + 4*i) relative to the base address configured in mtvt.

(R/W)

TRIG Configures the trigger type and polarity of the ith CLIC machine mode interrupt.

0x0: interrupt is positive level-triggered

0x1: interrupt is positive edge-triggered

0x2: interrupt is negative level-triggered

0x3: interrupt is negative edge-triggered

(R/W)

MODE Configures the privilege mode of the ith CLIC interrupt.

0x0: User mode

0x3: Machine mode

Others: Reserved and not programmable

(R/W)

Register 1.99. clicintctl[i] (0x20801003 + 4*i)

CTL[i] Configures the level and priority of the ith CLIC machine mode interrupt.

- Since the CLICINTCTLBITS parameter is 3 in this implementation of CLIC, therefore the actual number of bits implemented in clicintctl[i] is the same, and thus the lower 5 bits of this register are hardwired to 1.
- The configured value of mcliccfg.MNLBITS determines the number of higher bits in clicintctl[i] which encode the level of the interrupt, while the remaining lower bits encode the priority.

(R/W)

1.9.3 Core-Local Interrupts (CLINT)

1.9.3.1 Overview

CLINT is the core-local interrupt (CLINT) block that includes software and timer interrupts and corresponding configuration registers. CLINT interrupt sources with IDs as shown below:

Table 1.9-4. Core-Local Interrupt Sources

ID	Description
3	M mode software interrupt
7	M mode timer interrupt

1.9.3.2 Features

- Two local level-type machine mode interrupt sources with programmable priorities
- Memory-mapped configuration and status registers
- 64-bit timer with interrupt functionality, overflow flags, and three sampling modes
- Software interrupt

1.9.3.3 Software Interrupt

An M mode software interrupt is controlled by setting or clearing the memory-mapped register MSIP.

The register bit clicintie[3] must be set for enabling the software interrupt at the core level.

Pending state of this interrupt can be checked by reading the corresponding bit of clicintip register of interrupt 3 in CLIC, that is clicintip[3] bit.

1.9.3.4 Timer Counter and Interrupt

The timer counter can be enabled by setting the mtime_EN bit in mtimectl.

mtimecmplo and mtimecmphi registers are set to the value for comparison with system timer with lower and higher 32 bits respectively.

The CPU provides two local memory-mapped 32-bit wide registers mtimeloadlo and mtimeloadhi, which has both read/write access, to load the system timer with a specific value.

Interrupt for M mode is asserted when 64-bit value of system timer value exceeds the 64-bit combined value of mtimecmplo and mtimecmphi.

Pending state of timer interrupt is reflected as IP bit of clicintip register of interrupt 7 in CLIC.

For de-asserting the pending timer interrupt in M mode, either IE bit of clicintie register of interrupt 7 in CLIC has to be cleared or the value of the mtimecmplo and mtimecmphi register needs to be updated.

1.9.3.5 Register Summary

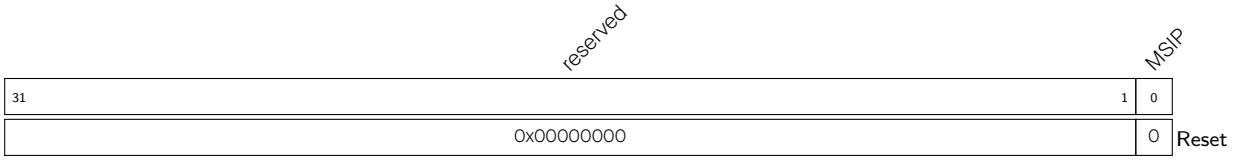
The addresses in this section are relative to CPU sub-system base address provided in Figure 7.3-1 in Chapter *7 System and Memory*.

Name	Description	Address	Access
msip	Core-local machine software interrupt pending register	0x0000	R/W
mtimecmplo	Lower 32 bits of the core-local machine timer compare value	0x4000	R/W
mtimecmphi	Higher 32-bit core-local machine timer compare value	0x4004	R/W
mtimeloadlo	Lower 32 bits of core-local machine timer load value	0x4008	R/W
mtimeloadhi	Higher 32 bits of core-local machine timer load value	0x400C	R/W
mtimectl	Core-local machine timer interrupt control/status register	0x4010	R/W
mtimelo	Lower 32 bits of core-local timer counter value	0xBFF8	RO
mtimehi	Higher 32 bits core-local timer counter value	0xBFFC	RO

1.9.3.6 Register Description

The addresses in this section are relative to CPU sub-system base address provided in Figure 7.3-1 in Chapter *7 System and Memory*.

Register 1.100. msip (0x0000)



MSIP Configures the pending status of the machine software interrupt.

0: Not pending

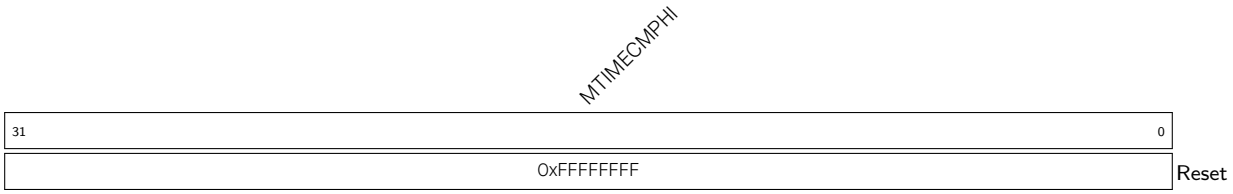
1: Pending

(R/W)

Register 1.101. mtimecmplo (0x4000)

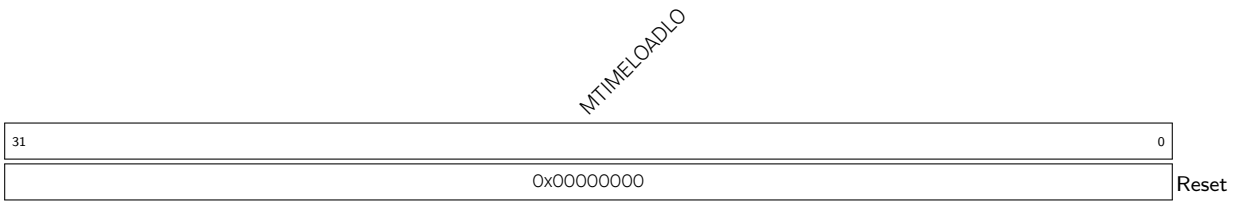
MTIMECMPLO Represents the value to compare with the lower 32 bits of the system counter. (R/W)

Register 1.102. mtimecmphi (0x4004)



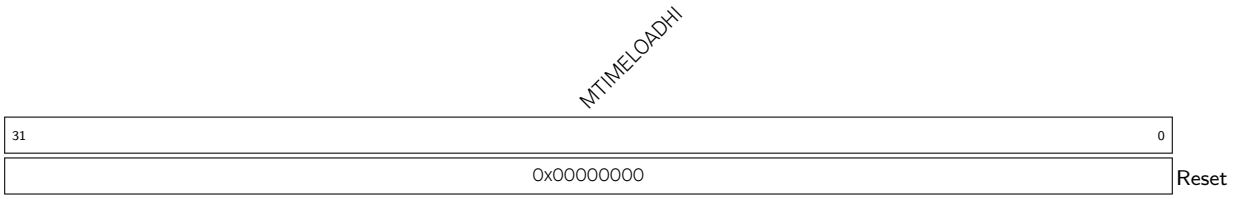
MTIMECMPHI Represents the value to compare with the higher 32 bits of the system counter. (R/W)

Register 1.103. mtimeloadlo (0x4008)



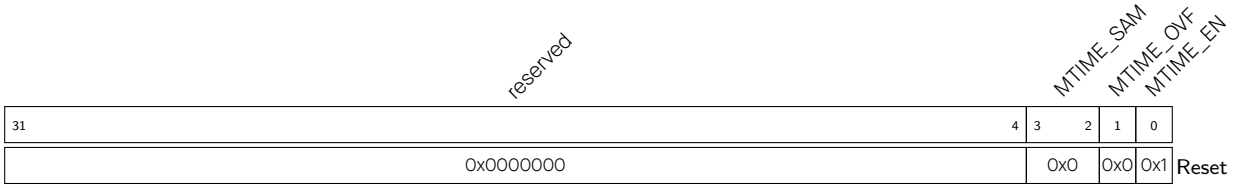
MTIME_LOADLO Represents the lower 32 bits to be loaded to the system counter. (R/W)

Register 1.104. mtimeloadhi (0x400C)



MTIME_LOADHI Represents the higher 32 bits to be loaded to the system counter. (R/W)

Register 1.105. mtimectl (0x4010)



MTIME_EN Configures whether to enable the system counter.

1: Enable

0: Disable

This bit is implemented only in Core 0 MTIMERCTL register. Writing the corresponding bit of Core 1's MTIMERCTL has no effect on the system timer. Since Core 1 can access Core 0's CLINT registers, it can run or pause the timer from MTIMERCTL register of Core 0.

(R/W)

MTIME_OVF Configures the overflow bit of the system counter.

1: Represents that system counter value is maximum, that is 0xFFFFFFFFFFFFFFFF

0: Otherwise

When the counter value is 0xFFFFFFFFFFFFFFFF, which is the maximum value, then this bit is set as 1 by hardware. Software can clear this bit by writing 0 to it. Writing 1 has no effect.

(R/W)

MTIME_SAM Configures the sampling mode of MTIMELO and MTIMEHI registers to allow accurate reading of the 64-bit system count. For the 64-bit system count, only one half of the read can be performed atomically. Therefore, the sampling mode allows the value of the other half to be sampled and stored in a buffer, and upon the next read of the other half, the sampled value from this buffer is retrieved instead of the actual counter.

0x0: No sampling (Default)

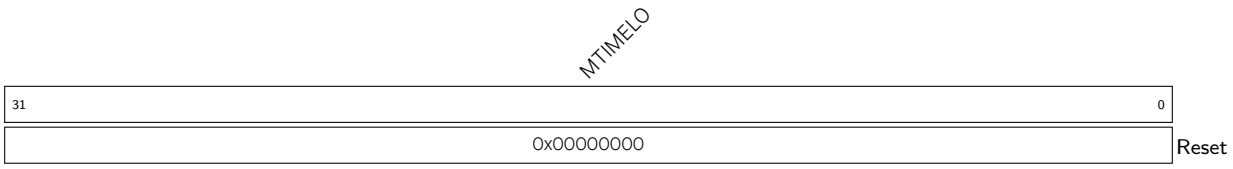
0x1: Sample upper half of the system count on reading MTIMELO

0x2: Sample lower half of the system count on reading MTIMEHI

0x3: Sample the other half of the system count on reading MTIMELO or MTIMEHI

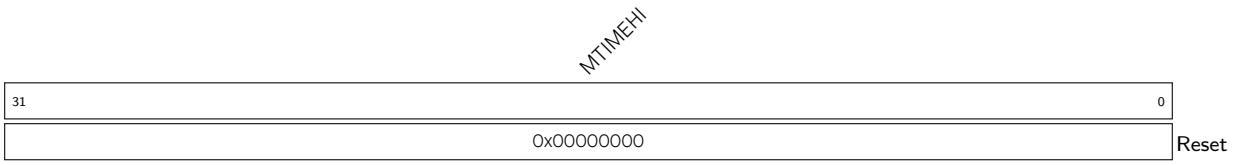
(R/W)

Register 1.106. mtimelo (0xBFF8)



MTIMELO Represents the current value of the lower 32 bits of the system counter. (RO)

Register 1.107. mtimehi (0xBFFC)



MTIMEHI Represents the current value of the higher 32 bits of the system counter. (RO)

1.10 Memory Protection Unit

Each HP core implements standard RISC-V physical memory protection (PMP) scheme along with custom physical memory attribute checker (PMAC) logic to prevent unauthorized memory access.

1.10.1 Standard Physical Memory Protection

1.10.1.1 Overview

The CPU core includes a physical memory protection (PMP) unit, which is fully compliant with RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10. It can be used by software to set memory access privileges (read, write and execute permissions) for required memory regions. In addition to standard PMP checks, the CPU core also implements custom physical memory attributes (PMA) checkers to provide additional permission checks based on pre-defined attributes.

1.10.1.2 Features

The PMP unit can be used to restrict access to physical memory. The main features are:

- Up to 32 PMP entries
- Minimum address granularity of 128 bytes
- Three address matching modes: OFF, TOR (Top of Range), and NAPOT (Naturally Aligned Power-of-Two). NA4 (Naturally Aligned 4-Byte Region) address matching mode is not supported
- Permission controls for read, write, and execute
- Lock function for each PMP entry
- Maximum physical space address of 4 GB

1.10.1.3 Functional Description

Software can program the PMP unit's configuration and address registers in order to contain faults and support secure execution. PMP CSRs can only be programmed in machine mode. Once the PMP unit is enabled by configuring PMP CSRs, write, read and execute permission checks are applied to all the accesses in user mode as per programmed values of the enabled 16 pmpcfgX and pmpaddrX registers (refer to Section 1.10.1.4).

By default, PMP grants permission to all accesses in machine mode and revokes permission of all access in user mode. This implies that it is mandatory to program the address range and valid permissions in pmpcfg and pmpaddr registers (refer to Section 1.10.1.4) for any valid access to pass through in user mode. However, it is not required for machine mode as PMP permits all accesses to go through by default. In cases where PMP checks are also required in machine mode, software can set the lock bit of required PMP entry to enable permission checks on it. Once the lock bit is set, it can only be cleared through CPU reset.

When any instruction is being fetched from a memory region without execute permissions, an exception is generated at processor level and exception cause is set as instruction access fault in mcause CSR. Similarly, any load/store access without valid read/write permissions will result in an exception generation with mcause updated as load access and store access fault respectively. In case of load/store access faults, violating address is captured in mtval CSR.

1.10.1.4 Register Summary

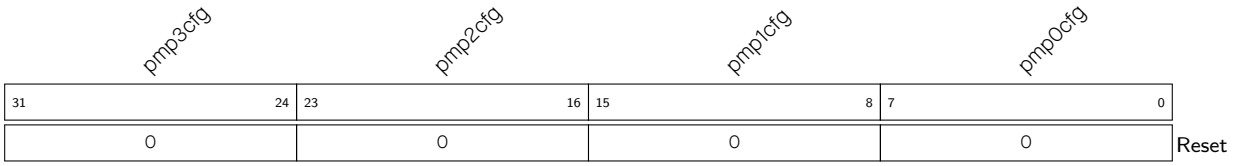
Below is a list of PMP CSRs supported by the CPU. These are only accessible from machine mode.

Name	Description	Address	Access
pmpcfg0	Physical memory protection configuration register	0x3A0	R/W
pmpcfg1	Physical memory protection configuration register	0x3A1	R/W
pmpcfg2	Physical memory protection configuration register	0x3A2	R/W
pmpcfg3	Physical memory protection configuration register	0x3A3	R/W
pmpcfg4	Physical memory protection configuration register	0x3A4	R/W
pmpcfg5	Physical memory protection configuration register	0x3A5	R/W
pmpcfg6	Physical memory protection configuration register	0x3A6	R/W
pmpcfg7	Physical memory protection configuration register	0x3A7	R/W
pmpaddr*n* (*n*: 0-31)	Physical memory protection address register	0x3B0+0x1**n*	R/W

1.10.1.5 Register Description

PMP unit implements all pmpcfg0-3 and pmpaddr0-15 CSRs as defined in RISC-V Instruction Set Manual, Volume II: Privileged Architecture, Version 1.10.

Register 1.108. pmpcfg0 (0x3A0)



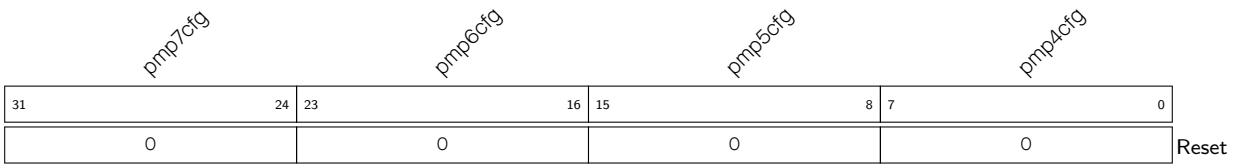
pmp3cfg Configuration register for PMP entry 3. (R/W)

pmp2cfg Configuration register for PMP entry 2. (R/W)

pmp1cfg Configuration register for PMP entry 1. (R/W)

pmp0cfg Configuration register for PMP entry 0. (R/W)

Register 1.109. pmpcfg1 (0x3A1)



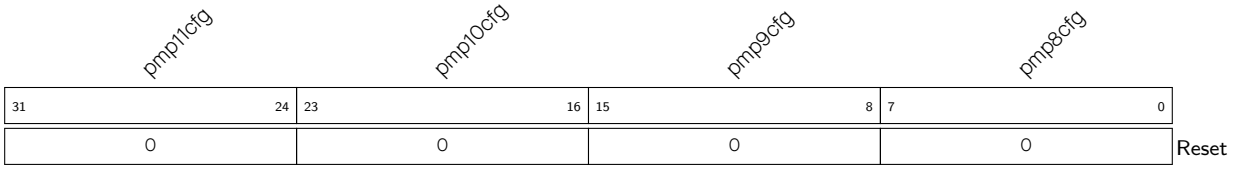
pmp7cfg Configuration register for PMP entry 7. (R/W)

pmp6cfg Configuration register for PMP entry 6. (R/W)

pmp5cfg Configuration register for PMP entry 5. (R/W)

pmp4cfg Configuration register for PMP entry 4. (R/W)

Register 1.110. pmpcfg2 (0x3A2)



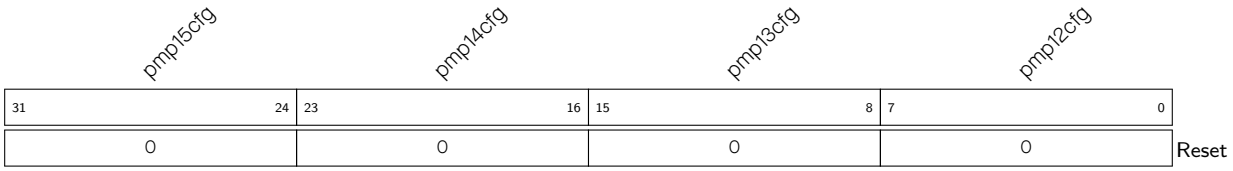
pmp11cfg Configuration register for PMP entry 11. (R/W)

pmp10cfg Configuration register for PMP entry 10. (R/W)

pmp9cfg Configuration register for PMP entry 9. (R/W)

pmp8cfg Configuration register for PMP entry 8. (R/W)

Register 1.111. pmpcfg3 (0x3A3)



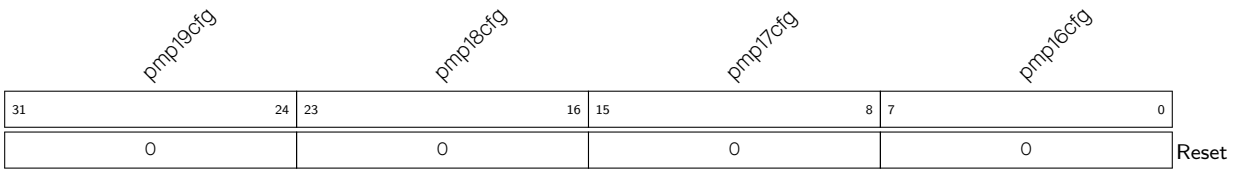
pmp15cfg Configuration register for PMP entry 15. (R/W)

pmp14cfg Configuration register for PMP entry 14. (R/W)

pmp13cfg Configuration register for PMP entry 13. (R/W)

pmp12cfg Configuration register for PMP entry 12. (R/W)

Register 1.112. pmpcfg4 (0x3A4)



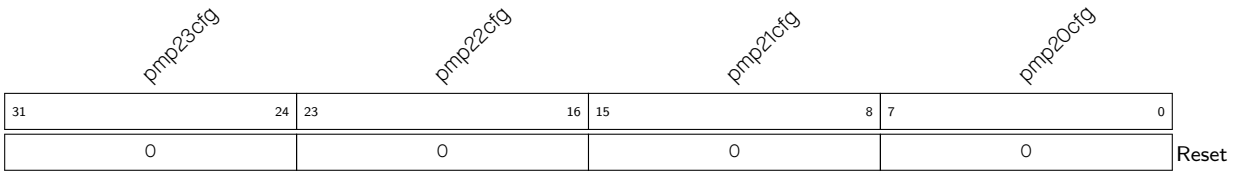
pmp19cfg Configuration register for PMP entry 19. (R/W)

pmp18cfg Configuration register for PMP entry 18. (R/W)

pmp17cfg Configuration register for PMP entry 17. (R/W)

pmp16cfg Configuration register for PMP entry 16. (R/W)

Register 1.113. pmpcfg5 (0x3A5)



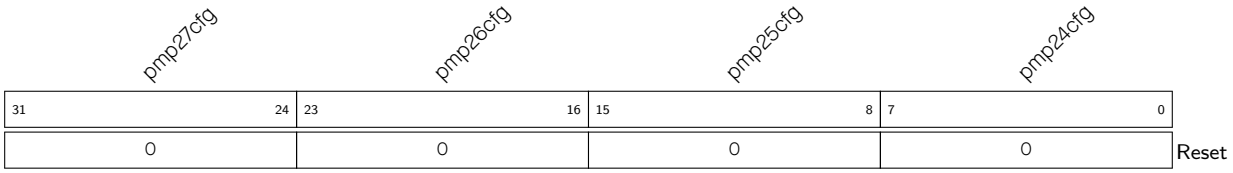
pmp23cfg Configuration register for PMP entry 23. (R/W)

pmp22cfg Configuration register for PMP entry 22. (R/W)

pmp21cfg Configuration register for PMP entry 21. (R/W)

pmp20cfg Configuration register for PMP entry 20. (R/W)

Register 1.114. pmpcfg6 (0x3A6)



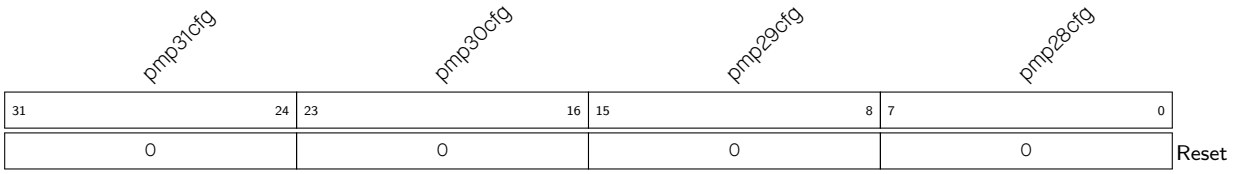
pmp27cfg Configuration register for PMP entry 27. (R/W)

pmp26cfg Configuration register for PMP entry 26. (R/W)

pmp25cfg Configuration register for PMP entry 25. (R/W)

pmp24cfg Configuration register for PMP entry 24. (R/W)

Register 1.115. pmpcfg7 (0x3A7)



pmp31cfg Configuration register for PMP entry 31. (R/W)

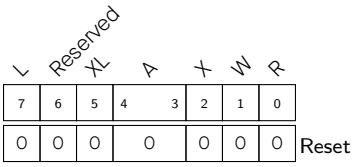
pmp30cfg Configuration register for PMP entry 30. (R/W)

pmp29cfg Configuration register for PMP entry 29. (R/W)

pmp28cfg Configuration register for PMP entry 28. (R/W)

The PMP configuration register format for any PMP entry is shown as follows.

Register 1.116. pmpXcfg



L Configures lock bit. Once set, it can only be cleared through a core reset. (R/W)

XL Configures custom-lock bit. Once set, it can only be cleared through a core reset.
0: PMP CSR accesses work normally
1: The corresponding pmpaddr and pmpcfg entry are locked and cannot be modified
(R/W)

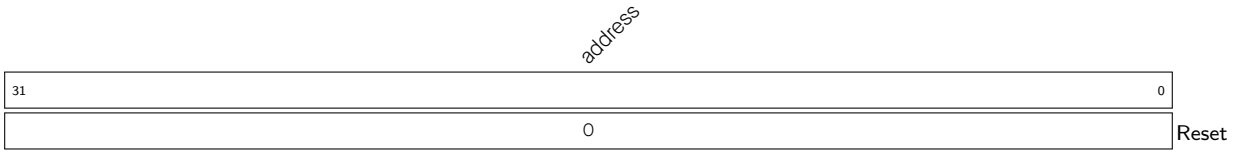
A Configures address matching mode.
0: OFF
1: TOR (Top of range)
2: Not Supported
3: NAPOT (Naturally aligned power-of-two region >= 128 Bytes)
(R/W)

X Configures execute permission. When set, execution from the address range is allowed. (R/W)

W Configures write permission. When set, memory writes to the address range are allowed. (R/W)

R Configures read permission. When set, memory reads from the address range are allowed. (R/W)

Register 1.117. pmpaddr*n* (*n*: 0-31) (0x3B0+0x1n*)**



address Configures the address for pmpaddr*n*. (R/W)

1.10.2 Custom Physical Memory Attribute (PMA) Checker

1.10.2.1 Overview

CPU core also implements a custom physical memory attributes checker (PMAC) to provide additional permission checks based on pre-defined memory types configured through custom CSRs. Please note that the PMA checks are applied irrespective of the core's privilege mode, i.e., it doesn't differentiate between machine and user mode.

1.10.2.2 Features

PMAC supports below features:

- 16 PMA entries
- Configurable memory type for defined memory regions
- Minimum address granularity of 128 bytes
- Three address matching mode: OFF, TOR (Top of Range) and NAPOT (Naturally Aligned Power-of-Two). NA4 (Naturally Aligned 4-Byte Region) address matching mode is not supported
- Permission controls for read, write and execute
- Lock function for each PMA entry
- Maximum physical space address of 4 GB
- Configurable attribute for defined memory regions

1.10.2.3 Functional Description

Software can program the PMAC unit's configuration and address registers in order to avoid faults due to access to invalid memory regions. PMAC CSRs can only be programmed in machine mode. Once enabled, write, read and execute permission checks are applied to all the accesses irrespective of privilege mode as per programmed values of enabled 16 pma_cfgX and pma_addrX registers (refer to Section 1.10.2.4). Access to entries marked as invalid memory types will result in fetch fault or load/store fault exception, as the case may be.

Exception generation and handling for PMAC related faults will be handled in similar way to PMP checks. When any instruction is being fetched from a memory region configured as a null or an invalid memory region, an exception is generated at the processor level and the exception cause is set as instruction access fault in mcause CSR. Similarly, any load/store access to a null or an invalid memory region will result in an exception generation with mcause updated as load access or store access fault respectively. In case of load/store access faults, violating address is captured in mtval CSR. For the PMAC entries configured as valid memory, the handling is same as for PMP checks.

A lock bit per entry is also provided in case software wants to disable programming of PMAC registers. Once the lock bit in any pma_cfgX register is set, respective pma_cfgX and pma_addrX registers can not be programmed further, unless a CPU reset cycle is applied.

A 4-bit field in PMAC CSRs is also provided to define attributes for memory regions. These bits are not used internally by the CPU core for any purpose. Based on address match, these attributes are provided on

load/store interface as side-band signals and are used by cache controller block for its internal operation.

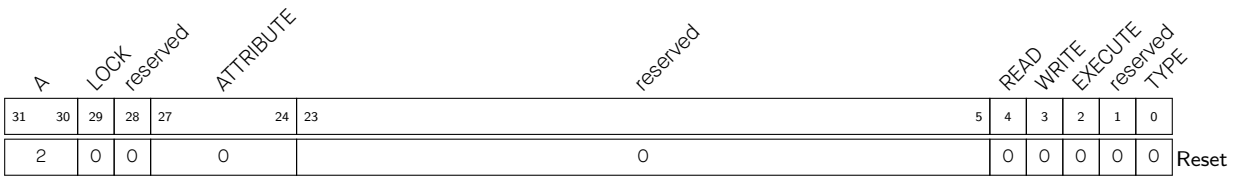
1.10.2.4 Register Summary

Below is a list of PMA CSRs supported by the CPU. These are only accessible from machine mode.

Name	Description	Address	Access
pma_cfg*n* (*n*: 0-15)	Physical memory attribute configuration register	0xBC0+0x1**n*	R/W
pma_addr*n* (*n*: 0-15)	Physical memory attribute address register	0xBD0+0x1**n*	R/W

1.10.2.5 Register Description

Register 1.118. pma_cfg*n* (*n*: 0-15) (0xBC0+0x1**n*)



A Configures address type. The functionality is the same as pmpcfg register's A field.

0x0: OFF

0x1: TOR

0x2: NA4

0x3: NAPOT

(R/W)

LOCK Configures whether to lock the corresponding pma_cfgX and pma_addrX.

0: Not lock

1: Lock. The write permission to the corresponding pma_cfgX and pma_addrX is revoked. It can only be unlocked by core reset.

(R/W)

ATTRIBUTE Configures the values to be driven on DRAM attribute ports.

Bit 27:

0: Cacheable

1: Non-cacheable

Bit 26:

0: Write-back

1: Write-through

Bit 25:

0: Write miss allocate

1: Write miss no allocate

Bit 24:

0: Read miss allocate

1: Read miss no allocate

(R/W)

READ Configures read-permission for the corresponding region.

0: Read not allowed

1: Read allowed

(R/W)

Continued on the next page...

Register 1.118. pma_cfg*n* (*n*: 0-15) (0xBC0+0x1**n*)

Continued from the previous page...

WRITE Configures write-permission for the corresponding region.

0: Write not allowed

1: Write allowed

(R/W)

EXECUTE Configures execute-permission for the corresponding region.

0: Execution not allowed

1: Execution allowed

(R/W)

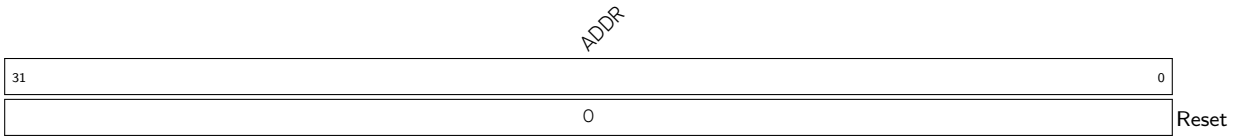
TYPE Configures region type.

0x0: Invalid memory region (RWX access will be treated as 0, even if programmed to 1)

0x1: Valid memory region (Programmed RWX access will be applicable)

(R/W)

Register 1.119. pma_addr*n* (*n*: 0-15) (0xBD0+0x1**n*)



ADDR Configures address. The functionality is same as pmpaddr register. (R/W)

1.11 Debug and Trace Support

1.11.1 Debug

1.11.1.1 Overview

This section describes how to debug software running on HP and LP CPU cores. Debug support is provided through standard JTAG pins and complies to the RISC-V External Debug Support Version 0.13.2 specification.

HP cores and LP core have their own dedicated JTAG DTM (debug transport module) in order to connect with respective Debug Modules. Both these JTAG DTM's are daisy-chained to provide debugger access to respective cores via a single JTAG interface. Depending on its debug requirements, Debugger can keep the unused DTM in a bypass state while debugging the required core. Both HP cores can be debugged through a single debug module (DM) connected to JTAG DTM (HP Cores), while the LP core is debugged through a separate debug module (DM) connected to JTAG DTM (LP Core).

Please note it is not possible to debug HP cores and LP core at same time as they are accessed through separate debug modules.

Figure 1.11-1 below shows the main components of External Debug Support.

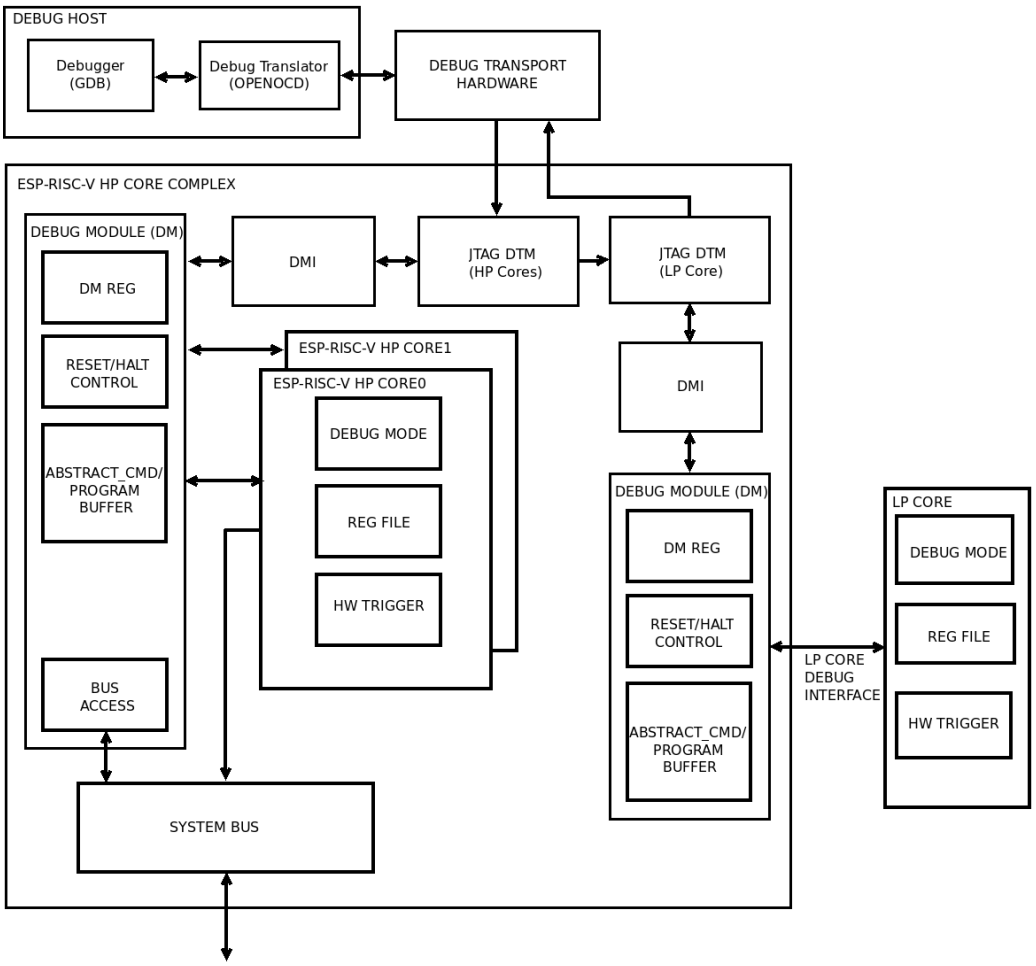


Figure 1.11-1. Debug System Overview

This section focuses on HP core debug features. For LP core debug features, please refer to Chapter 3 *Low-Power CPU*.

The user interacts with the Debug Host (e.g. laptop), which is running a debugger (e.g. GDB). The debugger communicates with a Debug Translator (e.g. OpenOCD, which may include a hardware driver) to communicate with Debug Transport Hardware (e.g. ESP-Prog adapter). The Debug Transport Hardware connects the Debug Host to the HP core's Debug Transport Module (DTM) through standard JTAG interface (bypassing LP core DTM). The DTM provides access to the Debug Module (DM) using the Debug Module Interface (DMI).

The DM allows the debugger to halt selected HP cores. Abstract commands provide access to GPRs (general-purpose registers). The Program Buffer allows the debugger to execute arbitrary code on the core, which allows access to additional CPU core state. Alternatively, additional abstract commands can provide access to additional CPU core state.

Each HP core contains Trigger Module supporting 3 triggers. When trigger conditions are met, respective core will halt spontaneously and inform the debug module that they have halted.

System bus access block allows memory and peripheral register access without affecting either HP core.

1.11.1.2 Features

The HP CPU supports the following basic debugging features for each HP core:

- Provides necessary information about the implementation to the debugger
- Allows the core to be halted and resumed
- Core registers (including CSRs) can be read/written by debugger
- Core can be debugged from the first instruction executed after reset
- Core can be reset through debugger
- Core can be halted on software breakpoint (planted breakpoint instruction)
- Hardware single-stepping
- Execute arbitrary instructions in the halted core by means of the program buffer. 2-word program buffer is supported
- System bus access is supported through word aligned address access
- Supports three Hardware Triggers (can be used as breakpoints/watchpoints) per HP core as described in Section 1.11.3
- Supports LP core debug through daisy chaining of its JTAG DTM

1.11.1.3 Functional Description

As mentioned earlier, Debug Scheme conforms to the RISC-V External Debug Support Version 0.13.2 specification. Please refer to it for functional operation details.

1.11.1.4 JTAG Control

Standard JTAG interface is the only way for DTM to access DM. The hardware provides two JTAG methods: PAD_to_JTAG and USB_to_JTAG.

- PAD_to_JTAG: The JTAG's signal source comes from IO.
- USB_to_JTAG: The JTAG's signal source comes from USB Serial/JTAG Controller.

Which JTAG method to use depends on many factors. The following table shows the configuration method.

Table 1.11-1. JTAG Control

Temporary disable JTAG 3,4	EFUSE_DIS_USB_SERIAL_JTAG 4	EFUSE_DIS_USB_JTAG 4	EFUSE_DIS_PAD_JTAG 4	EFUSE_JTAG_SEL_ENABLE 4	Strapping Pin GPIO34 5	USB JTAG Status	PAD JTAG Status
0	0	0	0	0	x 2	Available 1	Unavailable 1
0	0	0	0	1	1	Available	Unavailable
0	0	0	0	1	0	Unavailable	Available
0	0	0	1	x	x	Available	Unavailable
0	0	1	0	x	x	Unavailable	Available
0	0	1	1	x	x	Unavailable	Unavailable
0	1	0	0	0	x	Unavailable	Unavailable
0	1	0	0	1	0	Unavailable	Available
0	1	0	0	1	1	Unavailable	Unavailable
0	1	0	1	x	x	Unavailable	Unavailable
0	1	1	0	x	x	Unavailable	Available
0	1	1	1	x	x	Unavailable	Unavailable
1	x	x	x	x	x	Unavailable	Unavailable

Note:

1. Available: the corresponding JTAG function is available.
   Unavailable: the corresponding JTAG function is not available.
2. x: do not care.
3. "Temporary disable JTAG" means that if there are an even number of bits "1" in EFUSE_SOFT_DIS_JTAG[2:0], the JTAG function is turned on (the corresponding value in the table is 1), otherwise it is turned off (the corresponding value in the table is 0). However, under certain special conditions of the HMAC Accelerator in ESP32-P4, the JTAG function may be turned on even if there is an odd number of bits "1" in EFUSE_SOFT_DIS_JTAG[2:0]. For information on how HMAC affects JTAG functionality, please refer to Chapter HMAC Accelerator.
4. Please refer to Chapter eFuse Controller to get more information about eFuse.
5. Please refer to Chip Boot Control to get more information about the strapping pin GPIO34.

1.11.1.5 Register Summary

Below is the list of Debug CSRs supported by HP cores:

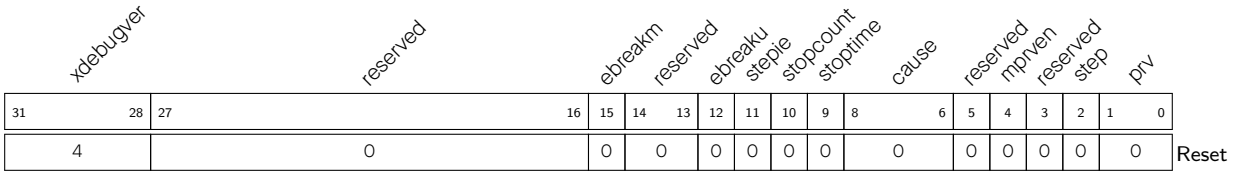
Name	Description	Address	Access
dcsr	Debug Control and Status Register	0x7B0	R/W
dpc	Debug PC Register	0x7B1	R/W
dscratch0	Debug Scratch Register 0	0x7B2	R/W
dscratch1	Debug Scratch Register 1	0x7B3	R/W

All the debug module registers are implemented in conformance to the specification RISC-V External Debug Support Version 0.13.2. Please refer to it for more details.

1.11.1.6 Register Description

Below are the details of Debug CSRs supported by HP cores:

Register 1.120. dcsr (0x7B0)



xdebugver Represents the debug version.

4: External debug support exists

(RO)

ebreakm When 1, ebreak instructions in Machine Mode enter debug mode.

(R/W)

ebreaku When 1, ebreak instructions in User/Application Mode enter debug mode.

(R/W)

stepie Configures interrupt control in debug mode. 0: Interrupts are disabled during single-stepping

1: Interrupts are enabled during single-stepping

(R/W)

Debugger must not change the value of this bit while the hart is running.

stopcount Configures mcycle counter control in debug mode. 0: Increment counters as usual.

1: Not increment any counters while in debug mode or on ebreak instructions that cause entry into debug mode. These counters include the cycle and instret CSRs.

(R/W)

stoptime Configures timer control in debug mode.

0: Increment timers as usual.

1: Not increment any hart-local timers while in debug mode.

(R/W)

cause Explains why debug mode was entered. When there are multiple reasons to enter debug mode in a single cycle, the cause with the highest priority number is the one written.

1: An ebreak instruction was executed. (priority 3)

2: The Trigger Module caused a halt. (priority 4, highest)

3: haltreq was set. (priority 1)

4: The CPU core single-stepped because the step was set. (priority 0, lowest)

5: The hart halted directory out of reset due to resethaltreq. (priority 2)

Other values are reserved for future use.

(RO)

mprven Configures whether to enable the functionality of MPRV bit in mstatus CSR during debug mode.

0: Disable

1: Enable

(R/W)

Continued on the next page...

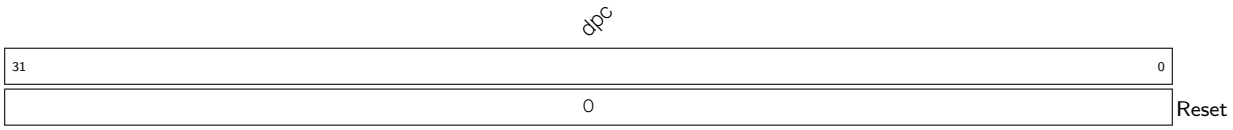
Register 1.120. dcsr (0x7B0)

Continued from the previous page...

step When set and not in debug mode, the core will only execute a single instruction and then enter debug mode.
If the instruction does not complete due to an exception, the core will immediately enter debug mode before executing the trap handler, with appropriate exception registers set.
Setting this bit does not mask interrupts. This is a deviation from the RISC-V External Debug Support Specification Version 0.13.
(R/W)

prv Contains the privilege level the core was operating in when debug mode was entered. A debugger can change this value to change the core's privilege level when exiting debug mode. Only **0x3** (machine mode) and **0x0** (user mode) are supported. (R/W)

Register 1.121. dpc (0x7B1)



dpc Upon entry to debug mode, dpc is written with the virtual address of the next instruction to be executed. When resuming, the CPU core's PC is updated to the virtual address stored in dpc. A debugger may write dpc to change where the CPU resumes. (R/W)

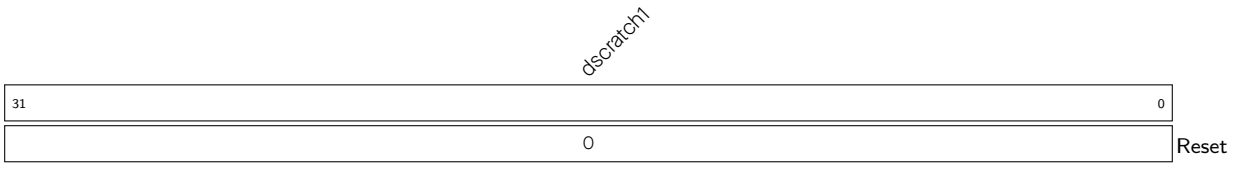
Table 1.11-3. Virtual address in DPC upon Debug Mode Entry

Cause	Virtual Address in DPC
ebreak	Address of ebreak instruction
Single Step	Address of the instruction that would be executed next if no debugging was going on i.e. pc+4 for 32-bit instruction that don't change program flow, the destination PC on taken jumps/branches, etc.
Trigger Module	If timing is 0, the address of the instruction which caused the trigger to fire. If timing is 1, the address of next instruction to be executed at the time that debug mode was entered.
Halt Request	Address of the next instruction to be executed at the time that debug mode was entered.

Register 1.122. dscratch0 (0x7B2)

dscratch0 Used by Debug Module internally. (R/W)

Register 1.123. dscratch1 (0x7B3)



dscratch1 Used by Debug Module internally. (R/W)

1.11.2 Debug Halt Groups

1.11.2.1 Overview

In a multi-core system, when the debugging software is running on a given core, it is useful that other cores do not change the state of the system. This requirement is addressed by synchronous halt and resume. It is important that halt/resume information is communicated as quickly as possible to other cores. So, it is better to do it based on chip infrastructure rather than commands through the debugger software running on the host.

1.11.2.2 Features

- Support cross-triggering between HP cores
- Overriding the RunStall functionality of a core

1.11.2.3 Functional Description

HP core debug module implements dmcs2 register specified in the upcoming specification, RISC-V External Debug Version 1.0-STABLE, to implement simultaneous halting/resuming based on hart groups.

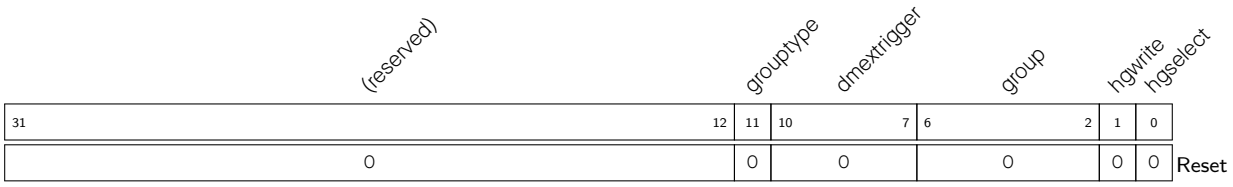
1.11.2.4 Register Summary

Below is a required control register implemented inside debug module.

Name	Description	Address	Access
dmcs2	Debug module control and status register 2	0x32	R/W

1.11.2.5 Register Description

Register 1.124. dmcs2 (0x32)



grouptype Configures the group type.

0 (halt group): The remaining fields in this register configure halt groups

1 (resume group): The remaining fields in this register configure resume groups

(R/W)

dmexttrigger Configures the currently selected DM external trigger. If a non-existent trigger value is written here, the hardware will change it to a valid value or 0 if no DM external triggers exist.

(R/W)

group When hgselect is 0, this field contains the group of the hart specified by hartsel.

When hgselect is 1, this field contains the group of the DM external trigger selected by dmexttrigger.

The value written to this field is ignored unless hgwrite is also written 1.

Group numbers are contiguous starting from 0, with the highest number being implementation-dependent, and possibly different between different group types. Debuggers should read back this field after writing to confirm they are using a supported hart group. If groups are not implemented, this entire field is 0.

(R/W)

hgwrite When 1 is written and hgselect is 0, for every selected hart, the DM will change its group to the value written to group, if the hardware supports that group for that hart. Implementations may also change the group of a minimal set of unselected harts in the same way, if that is necessary due to a hardware limitation. When 1 is written and hgselect is 1, the DM will change the group of the DM external trigger selected by dmexttrigger to the value written to group, if the hardware supports that group for that trigger. Writing 0 has no effect.

(R/W)

hgselect Tied to zero.

0: Operate on HART

(RO)

1.11.3 Hardware Trigger

1.11.3.1 Overview

1.11.3.2 Features

Each HP core implements a hardware trigger block to provide breakpoint and watchpoint capability for debugging. Each trigger has the following features:

- Three independent trigger units, each of which can be configured for matching the address of the program counter or load-store accesses
- Preempting execution by causing a breakpoint exception
- Halting execution and transferring control to debuggers
- Support for NAPOT (naturally aligned power-of-two regions) address encoding

1.11.3.3 Functional Description

The Hardware Trigger module provides seven CSRs, which are listed under Section register summary. Among these, tdata1 and tdata2 are abstract CSRs, which means they are shadow registers for accessing internal registers for each of the four trigger units, one at a time.

To choose a particular trigger unit write the index (0-2) of that unit into tselect CSR. When tselect is written with a valid index, the abstract CSRs tdata1 and tdata2 are automatically mapped to reflect internal registers of that trigger unit. Each trigger unit has two internal registers, namely mcontrol and maddress, which are mapped to tdata1 and tdata2, respectively.

Writing larger than allowed indexes to tselect will clip the written value to the largest valid index, which can be read back. This property may be used for enumerating the number of available triggers during initialization or when using a debugger.

Since software or debugger may need to know the type of the selected trigger to correctly interpret tdata1 and tdata2, the 4 bits (31-28) of tdata1 encodes the type of the selected trigger. This type field is read-only and always provides a value of 0x2 for every trigger, which stands for match type trigger, hence, it is inferred that tdata1 and tdata2 are to be interpreted as mcontrol and maddress. The information regarding other possible values can be found in the specification, RISC-V External Debug Support Version 0.13.2, but this trigger module only supports type 0x2.

Once a trigger unit has been chosen by writing its index to tselect, it will become possible to configure it by setting the appropriate bits in mcontrol CSR (tdata1) and writing the target address to maddress CSR (tdata2).

Each trigger unit can be configured to either cause breakpoint exception or enter debug mode, by writing to the action field of mcontrol. This bit can only be written from debugger, thus by default a trigger, if enabled, will cause breakpoint exception.

mcontrol for each trigger unit has a hit bit which may be read, after CPU halts or enters exception, to find out if this was the trigger unit that fired. This bit is set as soon as the corresponding trigger fires, but it has to be manually cleared before resuming operation. Although, failing to clear it does not affect normal execution in any way.

Each trigger unit only supports match on address, although this address could either be that of a load/store access or the virtual address of an instruction. The address and size of a region are specified by writing to maddress (tdata2) CSR for the selected trigger unit. Larger than 1 byte region sizes are specified through NAPOT (naturally aligned power-of-two) encoding (see Table 1.11-5) and enabled by setting match bit in mcontrol. Note that for NAPOT encoded addresses, by definition, the start address is constrained to be aligned to (i.e. an integer multiple of) the region size. The maximum size supported is 128 bytes.

Table 1.11-5. NAPOT encoding for maddress

maddress(31-0)	Start Address	Size (bytes)
aaa...aaaaaaaaa0	aaa...aaaaaaaaa0	2
aaa...aaaaaaaa01	aaa...aaaaaaaa00	4
aaa...aaaaaaa011	aaa...aaaaaaa000	8
aaa...aaaaaa0111	aaa...aaaaaa0000	16
	....	
aaa...0111111111	aaa...aaa0000000	2^8

tcontrol CSR is common to all trigger units. It is used for preventing triggers from causing repeated exceptions in machine mode while execution is happening inside a trap handler. This also disables breakpoint exceptions inside ISRs by default, although, it is possible to manually enable this right before entering an ISR, for debugging purposes. This CSR is not relevant if a trigger is configured to enter debug mode.

1.11.3.4 Trigger Execution Flow

When hart is halted and enters debug mode due to the firing of a trigger (action = 1):

- dpc is set to current PC (in decode stage)
- cause field in dcsr is set to 2, which means halt due to trigger
- hit bit is set to 1, corresponding to the trigger(s) which fired

When hart goes into trap due to the firing of a trigger (action = 0) :

- mepc is set to current PC (in decode stage)
- mcause is set to 3, which means breakpoint exception
- mpte is set to the value in mte right before trap
- mte is set to 0
- hit bit is set to 1, corresponding to the trigger(s) which fired

Note: If two different triggers fire at the same time, one with action = 0 and another with action = 1, then hart is halted and enters debug mode.

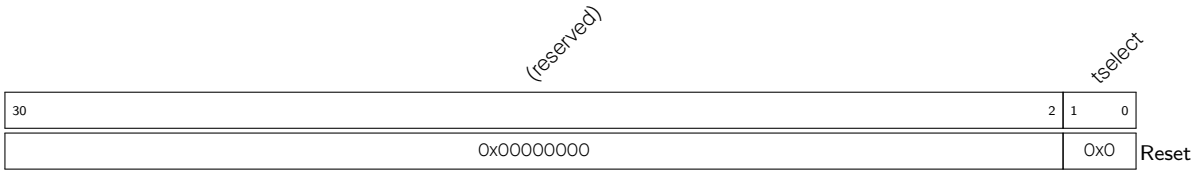
1.11.3.5 Register Summary

Below is a list of Trigger Module CSRs supported by the CPU. These are only accessible from machine mode.

Name	Description	Address	Access
tselect	Trigger select register	0x7A0	R/W
tdata1	Trigger abstract data 1	0x7A1	R/W
tdata2	Trigger abstract data 2	0x7A2	R/W
tdata3	Trigger abstract data 3	0x7A3	R/W
tinfo	Trigger information	0x7A4	RO
tcontrol	Global trigger control	0x7A5	R/W
mcontext	Trigger context	0x7A8	R/W

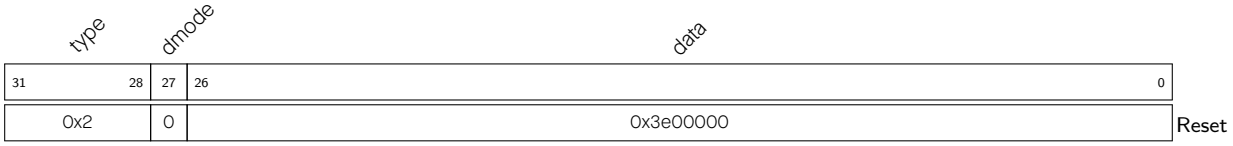
1.11.3.6 Register Description

Register 1.125. tselect (0x7A0)



tselect Configures the index (0-3) of the selected trigger unit. (R/W)

Register 1.126. tdata1 (0x7A1)

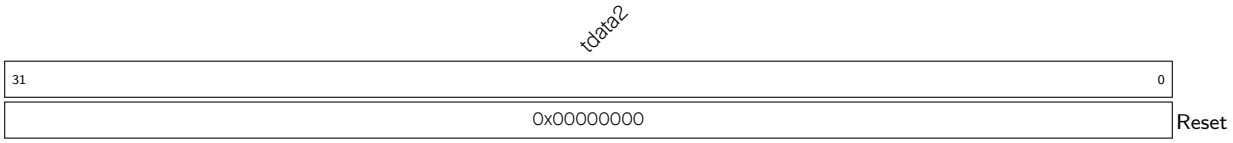


type Represents the trigger type. This field is reserved since only match type (0x2) triggers are supported. (RO)

dmode This is set to 1 if a trigger is being used by the debugger.
0: Both Debug and M mode can write the tdata1 and tdata2 registers at the selected tselect.
1: Only Debug Mode can write the tdata1 and tdata2 registers at the selected tselect. Writes from other modes are ignored.
Note: Only writable from debug mode.
(R/W)

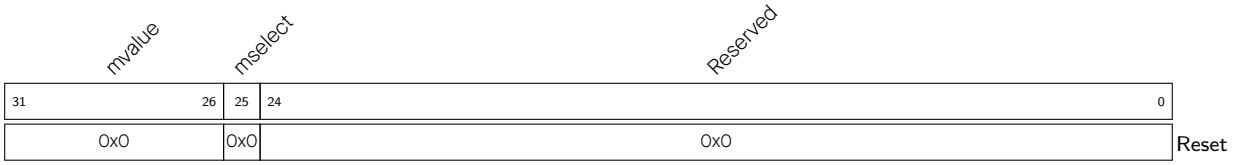
data Configures the abstract tdata1 content. This will always be interpreted as fields of mcontrol since only match type (0x2) triggers are supported. (R/W)

Register 1.127. tdata2 (0x7A2)



tdata2 Configures the abstract tdata2 content. This will always be interpreted as maddress since only match type (0x2) triggers are supported. (R/W)

Register 1.128. tdata3 (0x7A3)

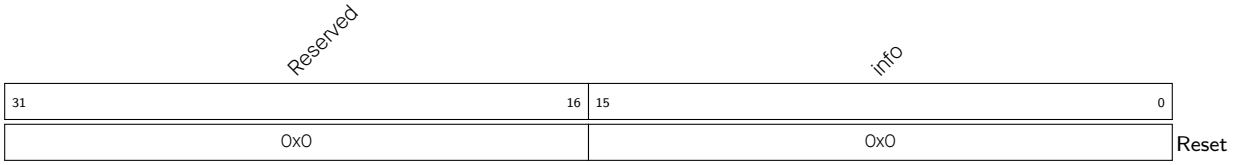


mvalue Data used together with mselect. (R/W)

mselect 0: Ignore mvalue.
1: This trigger will only match if the low bits of mcontext equal mvalue.
(R/W)

Reserved Read as 0. (RO)

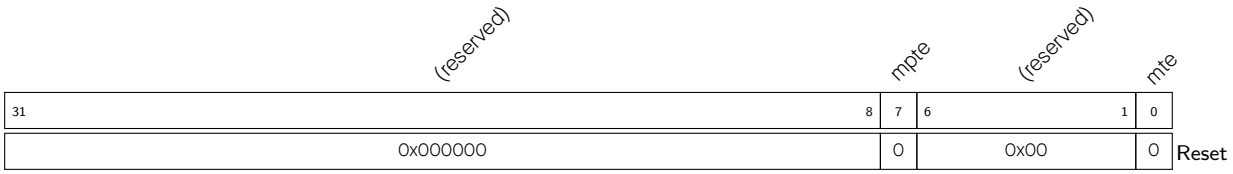
Register 1.129. tinfo (0x7A4)



Reserved Read as 0. (RO)

info One bit for each possible type enumerated in tdata1. Bit N corresponds to type N. If the bit is set, then that type is supported by the currently selected trigger. If the currently selected trigger does not exist, this field contains 1. If the type is not writable, this register may be unimplemented, in which case reading it causes an illegal instruction exception. In this case, the debugger can read the only supported type from tdata1. (RO)

Register 1.130. tcontrol (0x7A5)



mpte Configures whether to enable the machine mode previous trigger.

When CPU is taking a machine mode trap, the value of mte is automatically pushed into this.

When CPU is executing MRET, its value is popped back into mte, so this becomes 0.

(R/W)

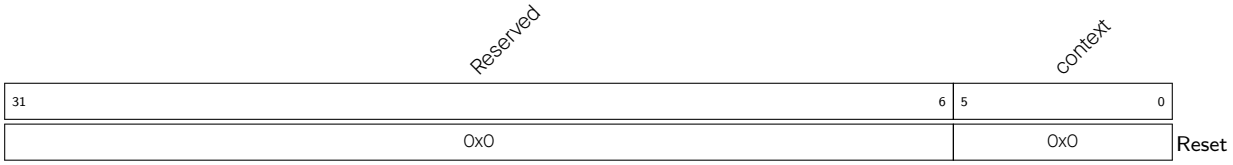
mte Configures whether to enable the machine mode trigger.

When CPU is taking a machine mode trap, its value is automatically pushed into mpte, so this becomes 0 and triggers with action=0 are disabled globally.

When CPU is executing MRET, the value of mpte is automatically popped back into this.

(R/W)

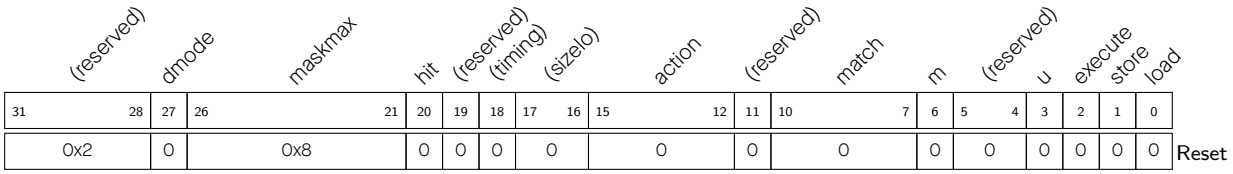
Register 1.131. MCONTEXT (0x7A8)



Reserved Read as 0. (RO)

context Writable in M mode and Debug Mode. Machine mode software can write a context number to this register, which can be used to set triggers that only fire in that specific context. (R/W)

Register 1.132. mcontrol (0x7A1)



dmode Same as dmode in tdata1. (RW*)

maskmax Represents the largest natural power-of-two (NAPOT) range supported when match is 1. It is always read as 0x8, which means only the lowest 8 bits can be masked. (RO)

hit This is found to be 1 if the selected trigger had fired previously. This bit needs to be cleared manually. (R/W)

timing Set this field to enable load/store trigger action timing.

0: Action for this trigger will be taken just before the instruction that triggered it is executed, but after all preceding instructions are committed.

1: Action for this trigger will be taken after the instruction that triggered it is executed. It should be taken before the next instruction is executed.

(R/W)

sizelo This field contains the lowest two bits of access address.

0x0: The trigger will attempt to match against an access of any size

0x1: The trigger will attempt to match against 8-bit memory accesses

0x2: The trigger will attempt to match against 16-bit memory accesses

0x3: The trigger will attempt to match against 32-bit memory accesses

(R/W)

action Configures the selected trigger to perform one of the available actions when firing. Valid options are:

0x0: cause breakpoint exception.

0x1: enter debug mode (only valid when dmode = 1)

0x2: Enable trace on trigger[0] hit

0x3: Enable trace on trigger[1] hit

0x4: Enable trace on trigger[2] hit

Note: Writing an invalid value will set this to the default value 0x0.

(R/W)

match Configures the selected trigger to perform one of the available matching operations on a data/instruction address. Valid options are:

0x0: exact byte match, i.e. address corresponding to one of the bytes in an access must match the value of maddress exactly.

0x1: NAPOT match, i.e. at least one of the bytes of an access must lie in the NAPOT region specified in maddress.

Note: Writing a larger value will clip it to the largest possible value 0x1.

(R/W)

Continued on the next page...

Register 1.132. mcontrol (0x7A1)

Continued from the previous page...

m Set this field to enable the selected trigger to operate in machine mode. (R/W)

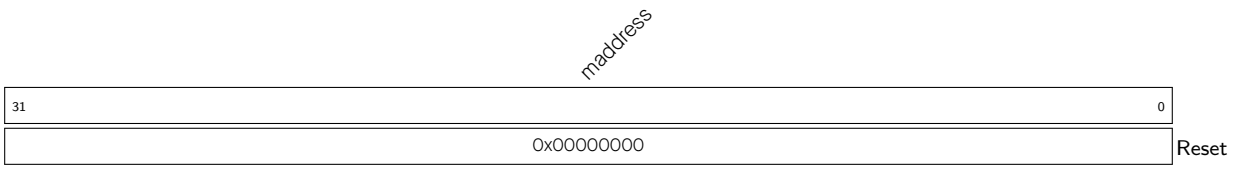
u Set this field to enable the selected trigger to operate in user mode. (R/W)

execute Set this field to enable the selected trigger to fire right before an instruction with the matching virtual address executed by the CPU. (R/W)

store Set this field to enable the selected trigger to fire right before a store operation with the matching data address executed by the CPU. (R/W)

load Set this field to enable the selected trigger to fire right before a load operation with the matching data address executed by the CPU. (R/W)

Register 1.133. maddress (0x7A2)



maddress Configures the address used by the selected trigger when performing a match operation. This is decoded as NAPOT when match=1 in mcontrol. (R/W)

1.11.4 Trace

1.11.4.1 Overview

In order to support non-intrusive software debug, the CPU core provides an instruction trace interface which provides relevant information for offline debug purpose. This interface provides relevant information to Trace Encoder block, which compresses the information and stores in memory allocated for it. Software decoders can read this information from trace memory without interrupting the CPU core and re-generate the actual program execution by the CPU core.

1.11.4.2 Features

The CPU core supports instruction trace feature and provides below information to Trace Encoder as mandated in RISC-V Processor Trace Version 1.0:

- Number of instructions being retired.
- Occurrence of exception and interrupt along with cause and trap values.
- Current privilege level of hart.
- Instruction type of retired instructions for jumps, branches and return.
- Instruction address for instructions retired before and after program counter changes.
- Trace enabling on trigger hit as per action bit in mcontrol.

1.11.4.3 Functional Description

Each HP core implements mandatory instruction delta tracing, also known as branch tracing. It works by tracking execution from a known start address by sending information about deltas taken by a program. Deltas are typically introduced by jump, call, return, branch type instructions and also by interrupts and exceptions. All such deltas along with additional details about cause and actual instructions/addresses are communicated over high bandwidth instruction trace interface output from the core. Trace Encoder operates on the information on this trace interface and compresses the information for storage in memory for offline debug by a decoder. More information about the encoding is available in Chapter *2 RISC-V Trace Encoder (TRACE)*.

The core does not have any internal registers to provide control over instruction trace interface. All register controls are available in *2 RISC-V Trace Encoder (TRACE)* block.

1.12 Performance

1.12.1 Branch Prediction

1.12.1.1 Overview

A core does branch prediction in terms of branch is taken or not taken and the target address of conditional/unconditional branch type instruction prior to its execution. Based on the prediction, the core starts fetching from the respective PC. Then, it validates the prediction on the execution of the instruction. If it is found to be miss-predicted, then the core flushes the pipeline and starts fetching from the correct PC. The prediction improves the throughput as the core continues fetching instructions based on prediction instead of waiting till the execution of branch instruction. The details about the prediction technique used by the core are described in the following section.

1.12.1.2 Features

- Prediction of both results and target address of conditional/unconditional branch type instructions
- Branch target prediction of instructions including BEQ, BNE, BLT, BLTU, BGE, BGEU, C.BEQZ, C.BNEZ, JAL, J, CJ
- Conditional branch prediction includes instructions such as: BEQ, BNE, BLT, BLTU, BGE, BGEU, C.BEQZ, C.BNEZ
- Uses 2-bit saturating counter for more accuracy
- Uses global history of conditional branches for prediction which improves performance
- Target prediction in one clock cycle for unconditional branch, that is, 0 cycle penalty when prediction or speculation is correct

1.12.1.3 Functional Description

The core uses Branch History Table (BHT) along with Branch Target Buffer (BTB) for prediction of branch and jump type instructions.

BHT is 512x16-bit SRAM memory for each core. It is indexed by Virtual Branch History Register (VBHR) while performing prediction and by Global Branch History Register (GBHR) while updating or correcting the result (taken or not taken) on execution of branch-type instructions. GBHR contains the results of the last 13 branch-type instructions executed. VBHR contains the results of the last 12 branch-type instructions and predicted results of the current branch instruction. Thus, both registers hold the trajectory of previously executed branches. Based on 13 bits of VBHR and GBHR, 2 bits of saturation counter value stored in BHT are either read or written respectively.

As shown in Figure 1.12-1, 2-bit saturation counter indicates if a branch is

- weakly not taken (00)
- strongly not taken (01)
- weakly taken (10)
- strongly taken (11)

If the result of prediction is weakly or strongly not taken, then the branch is predicted as not taken. Similar is the case for weakly/strongly taken.

While updating BHT with the actual result of branch execution, the following transition logic is used to update the respective saturation counter.

BTB stores the 32-bit branch target address. It can hold a maximum of 16 entries of target addresses. It is indexed by the last 16 bits of the PC of conditional/unconditional branch-type instruction. Thus, the core predicts the result of branch-type instructions based on BHT and jumps to the target obtained from BTB if the branch is predicted as taken. For unconditional branches, the core directly jumps to the target obtained from BTB, as it does not need prediction in this case. When the actual result and the target of branch-type instructions are obtained during execution, the core takes a corrective jump by flushing the pipeline in case of misprediction. BTB and BHT entries are corrected accordingly.

Figure 1.12-1 shows 2-bit saturation scheme details used in each HP core.

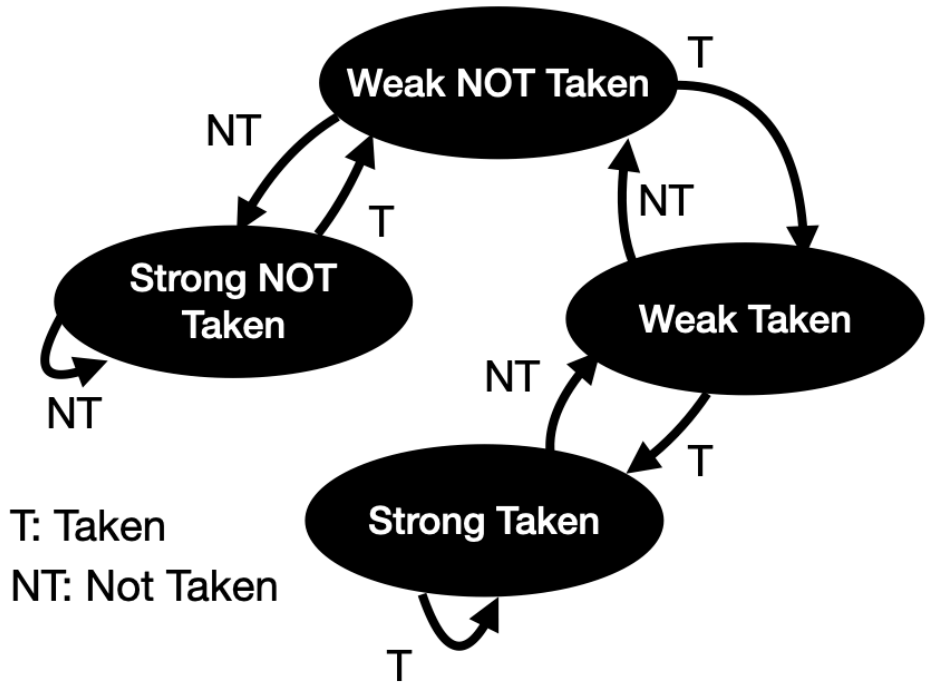


Figure 1.12-1. Two-bit Saturation Counter

1.12.2 RAS

1.12.2.1 Overview

Return Address Stack (RAS) is a stack maintained by the core to push the return addresses of jump and link (JAL/JALR) type instructions. Thus, the core can jump to the return address as soon as the RET instruction is pre-decoded, enhancing the performance of function call return.

1.12.2.2 Features

- Increased core performance by avoiding pipeline stall on return of function call

- Able to store up to four levels of function call nesting

1.12.2.3 Functional Description

When JAL/JALR/C.JALR instruction is pre-decoded by the core, then the 32-bit PC of the next instruction is pushed to RAS.

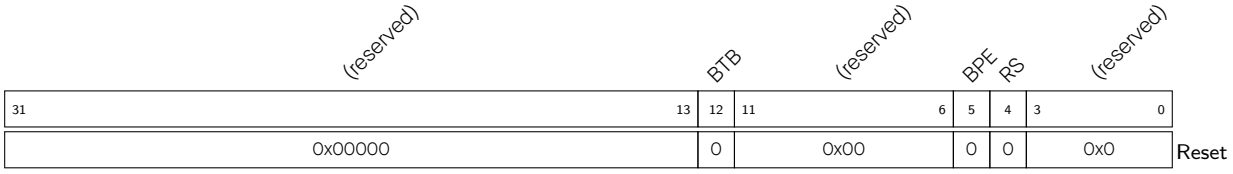
The address is popped out from RAS when the return type instruction (JALR, C.JR, C.JALR) is pre-decoded by the core. The core then starts fetching from the popped address. On execution of the return instruction, if the popped address does not match the address stored in the link register, then the core flushes the pipeline and starts to fetch from the address in the link register. RAS can store a maximum of four entries of return addresses, that is, four levels of function call nesting are supported.

RAS prediction depends on the following conditions:

Target Register (Rd)	Source Register (Rs)	RAS action
rd != x1	rs1 != x1	No action
rd != x1	rs1 == x1	POP
rd == x1	rs1 != x1	PUSH
rd == x1	rs1 == x1	PUSH and POP

1.12.3 Control Status Register for Performance Configuration

Register 1.134. mhcr (0x7C1)



RS Configures whether to enable the return address stack logic for prediction.
0: Disable
1: Enable
(R/W)

BPE Configures whether to enable conditional branch prediction.
0: Disable
1: Enable
(R/W)

BTB Configures whether to enable branch target buffer.
0: Disable
1: Enable
(R/W)

1.13 Custom Features

1.13.1 Bus Error Response

1.13.1.1 Overview

In the HP core, load/store bus errors are handled as asynchronous exceptions, which means that by the time such an exception is detected, the CPU may have executed some of the instructions after the actual load/store instruction that caused the exception.

Due to the asynchronous nature of these exceptions, handling them requires more context than that available via the standard CSRs mcause, mepc and mtval, which are otherwise sufficient for synchronous exceptions. Therefore, additional CSRs have been provided to allow software to gain all the necessary context required to handle and resume a program that has encountered an asynchronous bus-error exception.

In the following sections, these bus-error-specific CSRs are described and the recommended approach is to gain context about a bus-error exception.

1.13.1.2 Functional Description

There are two ways to handle load/store bus errors:

- Synchronous bus-errors mode:
  When this mode is enabled, (by setting mhint.SBE bit) bus-error exceptions can be handled just like other synchronous exceptions. Therefore, execution traps at the exact PC corresponding to the bus error causing load/store instruction, and mepc holds the same PC.
  The side effect of enabling this mode is that it increases the latency of all load/store instructions.
- Asynchronous bus-errors mode (default):
  - For handling bus errors in asynchronous mode, it is recommended (although not necessary) to clear the mexstatus.NMFT bit at startup.
  - In asynchronous mode, bus errors cause the CPU to enter trap handler with MCAUSE value 5 or 7 (depending upon the type of operation which caused the error), and mepc will hold the PC of the last instruction that was about to be executed when CPU detected the bus error(s) due to (one or more) previously executed load/store instruction(s).
  - Upon entering the exception handler for cause 5 or 7, the mexstatus.BUS_ERR bit must be checked to confirm if it is due to load/store bus-error. If so, this bit must be cleared to acknowledge.
  - Next, bits ldpc0.vld, ldpc1.vld, stpc0.vld, stpc1.vld and stpc2.vld must be checked to confirm if they are valid. If so, these must be cleared, and the corresponding address fields (ldpc0.addr, ldpc1.addr, stpc0.addr, stpc1.addr, and stpc2.addr) can be extracted from these CSRs to get the program counters at which these faulting load/store instructions were executed. The CSRs ldtval0, ldtval1, sttval0, sttval1 and sttval2 hold the memory access addresses corresponding to the respective loads/stores. Note that the lower indexed CSRs hold the newest bus errors, while the higher indexed CSRs hold the older bus errors.
  - Note that, even if the cause is either 5 or 7, both load and store bus error CSRs must be checked as sometimes there are more than one load/store instructions (or misaligned accesses which are split into multiple loads/stores) that have been executed and are in flight. Prior to entering the trap

handler for the first encountered bus error, the CPU will allow all pending memory operations to finish and record any bus errors caused by them, thus it is important to check all the CSRs to make sure that all such exceptions were caught and handled.

- Also note that recovering a normal program execution after encountering a bus-error exception is not as simple as for synchronous cases, since program execution may have proceeded quite far ahead of the bus-error-causing instructions.

1.13.1.3 Control Status Registers

Name	Description	Address	Access
mexstatus	Machine extension control and status register.	0x7E1	R/W
mhint	Machine implicit operation register	0x7C5	R/W
ldpc0	Load bus error PC register 0	0xBE0	R/W
ldpc1	Load bus error PC register 1	0xBE1	R/W
ldtval0	Load bus error access address register 0	0xBE8	R/W
ldtval1	Load bus error access address register 1	0xBE9	R/W
stpc0	Store bus error PC register 0	0xBF0	R/W
stpc1	Store bus error PC register 1	0xBF1	R/W
stpc2	Store bus error PC register 2	0xBF2	R/W
sttval0	Store bus error access address register 0	0xBF8	R/W
sttval1	Store bus error access address register 1	0xBF9	R/W
sttval2	Store bus error access address register 2	0xBFA	R/W

1.13.2 Dedicated IO

1.13.2.1 Overview

Normally, GPIOs are an APB peripheral, which means that changes to outputs and reads from inputs can get stuck in write buffers or behind other transfers. They are slower because the APB bus runs at a lower speed than the CPU core. As an alternative, the CPU core implements I/O processor-specific CPU registers (CSRs) which are directly connected to the GPIO matrix or IO pads. As these registers can be accessed in one instruction, the speed is fast.

1.13.2.2 Features

- 8 dedicated IOs directly mapped on GPIOs
- No latency for driving output ports
- Two CPU cycle latency for sensing input values

1.13.2.3 Functional Description

The CPU core has a set of 8 inputs and outputs (pin value + pin output enable). These input and output ports are directly connected to the GPIO matrix, through which they can be mapped on top-level pads. Please refer to Chapter 9 *GPIO Matrix and IO MUX* for more details.

The CPU implements three custom CSRs:

- GPIO_IN is read-only and reflects the input value.
- GPIO_OUT is R/W and reflects the output value for the GPIOs.
- GPIO_OEN is R/W and reflects the output enable state for the GPIOs. It controls the pad direction. Programming high would mean the pad should be configured in output mode. Programming low means it should be configured in input mode.

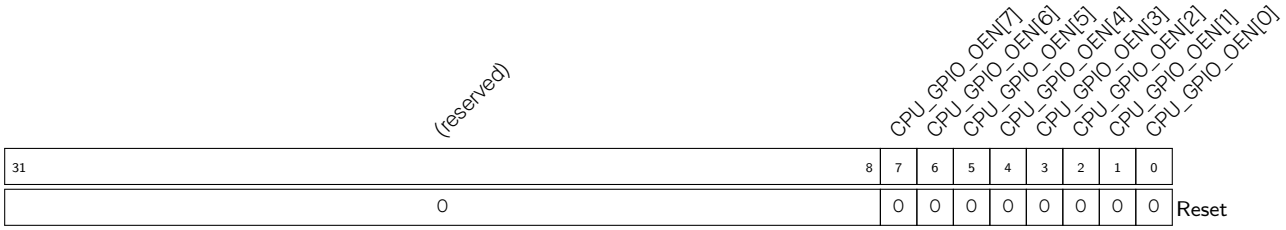
1.13.2.4 Register Summary

Below is a list of custom dedicated IO CSRs implemented inside the core.

Name	Description	Address	Access
cpu_gpio_oen	GPIO Output Enable	0x803	R/W
cpu_gpio_in	GPIO Input Value	0x804	RO
cpu_gpio_out	GPIO Output Value	0x805	R/W

1.13.2.5 Register Description

Register 1.135. cpu_gpio_oen (0x803)



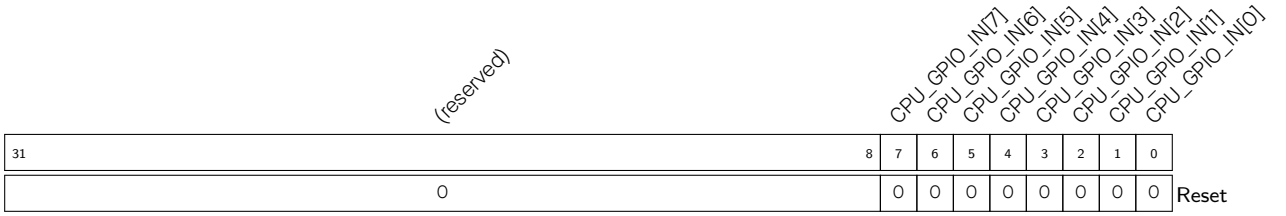
CPU_GPIO_OEN Configures whether to enable GPIOn (n=0 ~ 21) output. CPU_GPIO_OEN[7:0] correspond to output enable signals cpu_gpio_out_oen[7:0] in Table 9.12-1 *Peripheral Signals via GPIO Matrix*. CPU_GPIO_OEN value matches that of cpu_gpio_out_oen. CPU_GPIO_OEN is the enable signal of CPU_GPIO_OUT.

0: Disable GPIO output

1: Enable GPIO output

(R/W)

Register 1.136. cpu_gpio_in (0x804)



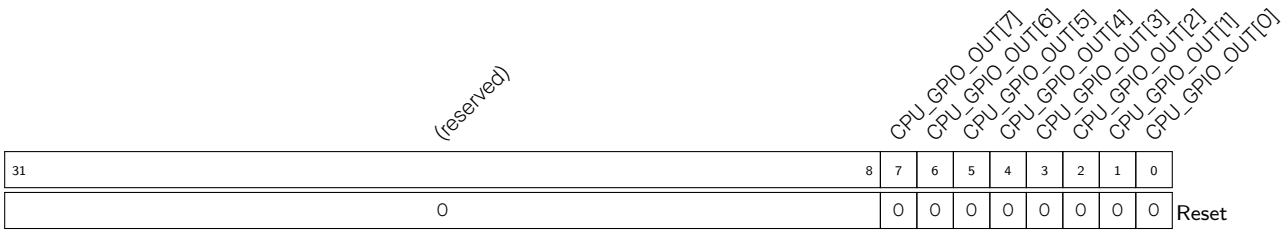
CPU_GPIO_IN Represents GPIOn (n=0 ~ 21) input value. It is a CPU CSR to read input value (1=high, 0=low) from SoC GPIO pin.

CPU_GPIO_IN[7:0] correspond to input signals cpu_gpio_in[7:0] in Table 9.12-1 *Peripheral Signals via GPIO Matrix* .

CPU_GPIO_IN[7:0] can only be mapped to GPIO pins through GPIO matrix. For details please refer to Section 9.4 in Chapter *GPIO Matrix and IO MUX*.

(RO)

Register 1.137. cpu_gpio_out (0x805)



CPU_GPIO_OUT Configures GPIOn (n=0 ~ 21) output value. It is a CPU CSR to write value (1=high, 0=low) to SoC GPIO pin. The value takes effect only when CPU_GPIO_OEN is set.

CPU_GPIO_OUT[7:0] correspond to output signals cpu_gpio_out[7:0] in Table 9.12-1 *Peripheral Signals via GPIO Matrix*.

CPU_GPIO_OUT[7:0] can only be mapped to GPIO pins through GPIO matrix. For details please refer to Section 9.5 in Chapter *GPIO Matrix and IO MUX*.

(R/W)

1.13.3 RunStall Support

1.13.3.1 Overview

The RunStall signal allows an external agent to stall the processor and shut off much of the clock tree to save operating power. The RunStall input allows external logic to stall the processor's internal pipeline. Any in-progress instructions in the pipeline will complete after the assertion of the RunStall input line. After the pipeline is stalled, the core outputs a handshake signal "corestalled" to indicate successful stalling of the CPU pipeline.

1.13.3.2 Functional Description

The RunStall input can be used in the following two scenarios:

- As a mechanism to initialize instruction and data RAMs after a reset. If the RunStall input is asserted during reset and remains asserted after the reset is removed, the processor will be in a stalled state. Instruction and data RAMs can be initialized and after these RAMs have been initialized, the RunStall can be released, allowing the processor to start code execution. This mechanism allows the processor's reset vector to point to an address in local instruction RAM, and the reset code can be written to the instruction RAM by an external agent before the processor starts executing code.
- To freeze the processor's internal pipeline. Assertion of RunStall reduces the processor's active power dissipation without using or requiring interrupts and the WAITI option. However, the power savings resulting from use of the RunStall signal will not be as great as for those using WAITI, because the RunStall logic allows some portions of the processor to be active even though the pipeline is stalled.

Figure 1.13-1 shows the timing diagram for RunStall and CoreStalled signals and CPU core state transition.

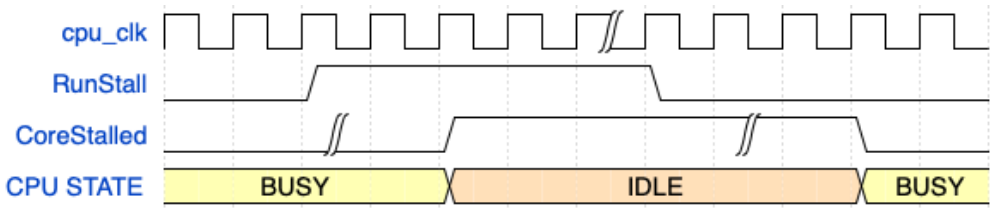


Figure 1.13-1. CPU Core State Transition Using RunStall

1.13.3.3 Register Summary

The SoC registers in Section 20.2.1.10 can be used to assert RunStall to HP cores.

1.13.4 Debug Assist Information

The HP CPU core complex provides additional debug information to the Debug Assistant module in ESP32-P4. For details, please refer to Chapter 21 *Debug Assistant*.

1.13.5 Core Lock-up

1.13.5.1 Overview

Currently, there is no exception enable register defined in the RISC-V programming model, and when returning from the exception service program to the normal program flow, the exception vector number in the MCAUSE register is not cleared, so software developers cannot easily know whether the CPU is currently processing an exception or running a normal program track by the registers defined in the programming model. To address this concern, a custom LOCKUP field has been implemented. It is readable through the custom CSR.

1.13.5.2 Functional Description

When the CPU responds to an exception, it will determine whether the current program flow is in the exception service program (through EXPT_VLD in MEXSTATUS field). If the CPU is processing an exception and triggers a new exception before returning from the exception service program, the CPU will be locked. When CPU is locked:

- the lock-up signal is set high to inform the SoC that the CPU is in a locked state
- the CPU stops instruction fetch and execution
- the CPU keeps the PC at the instruction PC that triggers the CPU lock
- the LOCKUP field in the MEXSTATUS register is set to 1

In a locked-up state, a debugger can still break in and execute debug commands. It is recommended to reset the core when a lock state is detected.

There are some exceptions to the lock-up rule. Exceptions caused by ECALL or EBREAK, when nested in any exception type, will not trigger CPU lock-up.

1.13.5.3 Register Summary

The mexstatus.LOCKUP bit provides lock-up status of individual core.

1.13.5.4 Register Description

Please refer to mexstatus CSR.

1.13.6 External Wakeup

1.13.6.1 Overview

Each core features a dedicated input port for wake-up via an external event. This functionality is controlled and configured through registers in the SoC.

1.13.6.2 Functional Description

To save power, each CPU core can enter a sleep state by executing the WFI instruction. It can be awakened by three types of events: an interrupt, a halt request, or a signal from its dedicated wake-up input port. A high-level pulse lasting a single clock cycle on this dedicated port is sufficient to wake the corresponding core.

For details on how to configure this port, please refer to Chapter *20 System Registers (SYSREG)*.